

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΕΠΙΚΟΙΝΩΝΙΩΝ, ΗΛΕΚΤΡΟΝΙΚΗΣ ΚΑΙ ΣΥΣΤΗΜΑΤΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ

Διαχείριση μνήμης σε συνομιλιακούς AI πράκτορες για την υποστήριξη ερωτημάτων πάνω σε γράφους γνώσης

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΟΛΓΑ Ε. ΓΙΑΝΝΟΥΧΟΥ

Επιβλέπων: Συμεών Παπαβασιλείου
Καθηγητής Ε.Μ.Π.

Αθήνα, Οκτώβριος 2026



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΕΠΙΚΟΙΝΩΝΙΩΝ, ΗΛΕΚΤΡΟΝΙΚΗΣ ΚΑΙ ΣΥΣΤΗΜΑΤΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ

Διαχείριση μνήμης σε συνομιλιακούς AI πράκτορες για την υποστήριξη ερωτημάτων πάνω σε γράφους γνώσης

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΟΛΓΑ Ε. ΓΙΑΝΝΟΥΧΟΥ

Επιβλέπων: Συμεών Παπαβασιλείου
Καθηγητής Ε.Μ.Π.

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 22η Οκτωβρίου 2026.

.....	.....	.....
Συμεών Παπαβασιλείου	Ιωάννα Ρουσσάκη	Γεώργιος Ματσόπουλος
Καθηγητής Ε.Μ.Π.	Καθηγήτρια Ε.Μ.Π.	Καθηγητής Ε.Μ.Π.

Αθήνα, Οκτώβριος 2026

Όλγα Ε. Γιαννούχου
Διπλωματούχος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών Ε.Μ.Π.

Copyright © Όλγα Ε. Γιαννούχου, 2026.
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας Εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της Εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

Περίληψη

Η ραγδαία εξέλιξη των Μεγάλων Γλωσσικών Μοντέλων (Large Language Models – LLMs) έχει επιτρέψει την ανάπτυξη συνομιλιακών και πρακτορικών συστημάτων Τεχνητής Νοημοσύνης που αλληλεπιδρούν με εξωτερικές πηγές γνώσης και εκτελούν σύνθετες ακολουθίες ενεργειών. Ωστόσο, η αποτελεσματική διαχείριση της μνήμης παραμένει πρόκληση, καθώς η εμπειρία από την επίλυση ενός προβλήματος δεν αξιοποιείται απαραίτητα σε μεταγενέστερες συναφείς αλληλεπιδράσεις. Η πρόκληση αυτή είναι κρίσιμη σε ροές Text-to-Cypher, όπου η παραγωγή ερωτημάτων για γράφους γνώσης απαιτεί συχνά διαδοχικούς ελέγχους και διορθώσεις λόγω συντακτικών σφαλμάτων, ασυμβατοτήτων με το σχήμα ή μη έγκυρης πρόσβασης σε ιδιότητες.

Στην παρούσα διπλωματική εργασία διερευνάται η ενσωμάτωση δομημένης επεισοδιακής μνήμης στην πρακτορική αρχιτεκτονική CyVerACT. Σχεδιάστηκε και υλοποιήθηκε στο Neo4j ένας προσαρμοσμένος γράφος μακροπρόθεσμης μνήμης, στον οποίο αποθηκεύονται προηγούμενες ερωτήσεις φυσικής γλώσσας, προσπάθειες παραγωγής Cypher, σφάλματα και τελικά ερωτήματα που εκτελέστηκαν επιτυχώς. Πάνω στη δομή αυτή υλοποιήθηκαν τρεις μηχανισμοί: η ανάκτηση παρόμοιων προηγούμενων ερωτήσεων και των αντίστοιχων επιτυχώς εκτελεσμένων ερωτημάτων πριν από την αρχική παραγωγή, η ανάκτηση σχετικών τροχιών διόρθωσης κατά την επαναληπτική διαδικασία και η αξιοποίηση του βραχυπρόθεσμου ιστορικού της τρέχουσας εκτέλεσης. Οι μηχανισμοί αξιολογήθηκαν τόσο μεμονωμένα όσο και σε συνδυασμούς.

Η προσέγγιση αξιολογήθηκε στον γράφο γνώσης Movies του Neo4j με τρία Μεγάλα Γλωσσικά Μοντέλα και επτά πειραματικές διαμορφώσεις, χρησιμοποιώντας 100 ερωτήματα αξιολόγησης και 50 επιπλέον ερωτήματα για την αρχική τροφοδότηση της μακροπρόθεσμης μνήμης. Η αξιοποίηση της μνήμης βελτίωσε την τεχνική εγκυρότητα των παραγόμενων ερωτημάτων Cypher, με τη μέγιστη αύξηση της μετρικής KG Valid να ανέρχεται σε 21 ποσοστιαίες μονάδες. Το μεγαλύτερο όφελος παρατηρήθηκε στο μικρότερο γλωσσικό μοντέλο, ενώ η βελτίωση στη συμφωνία των αποτελεσμάτων εκτέλεσης με τα αποτελέσματα αναφοράς ήταν πιο περιορισμένη. Η ανάκτηση προηγούμενων τροχιών διόρθωσης παρουσίασε την πιο συνεπή θετική επίδραση στην τεχνική εγκυρότητα, χωρίς ωστόσο κάποια διαμόρφωση μνήμης να υπερέχει σε όλα τα μοντέλα και σε όλες τις μετρικές.

Συνολικά, η επιλεκτική ανάκτηση προηγούμενης εμπειρίας ενίσχυσε την ικανότητα της πρακτορικής ροής να παράγει τεχνικά έγκυρα ερωτήματα Cypher. Η μνήμη δεν υποκαθιστά τις δυνατότητες του γλωσσικού μοντέλου ούτε εγγυάται τη σημασιολογική ορθότητα, αλλά επιτρέπει στο σύστημα να επαναχρησιμοποιεί εμπειρία από προηγούμενες εκτελέσεις.

Λέξεις-κλειδιά: συνομιλιακοί πράκτορες Τεχνητής Νοημοσύνης, πρακτορική Τεχνητή Νοημοσύνη, διαχείριση μνήμης, επεισοδιακή μνήμη, μακροπρόθεσμη μνήμη, γράφοι γνώσης, Text-to-Cypher, Cypher, Μεγάλα Γλωσσικά Μοντέλα, σημασιολογική ανάκτηση, Neo4j, CyVerACT

Abstract

The rapid development of Large Language Models (LLMs) has enabled conversational and agentic Artificial Intelligence systems to interact with external knowledge sources and execute complex sequences of actions. However, effective memory management remains a challenge, since experience acquired while solving one problem is not necessarily reused in subsequent related interactions. This challenge is critical in Text-to-Cypher workflows, where generating queries for knowledge graphs often requires successive validation and correction due to syntax errors, schema mismatches, or invalid property access.

This thesis investigates the integration of structured episodic memory into the CyVerACT agentic architecture. A custom long-term memory graph was designed and implemented in Neo4j to store previous natural-language questions, Cypher generation attempts, errors, and final queries that were successfully executed. Three memory mechanisms were implemented: retrieving similar previous questions and their corresponding successfully executed queries before initial generation, retrieving relevant correction trajectories during the iterative correction process, and using short-term history from the current execution. These mechanisms were evaluated both individually and in combination.

The approach was evaluated on the Neo4j Movies knowledge graph using three Large Language Models and seven experimental configurations, with 100 evaluation questions and 50 additional questions used to initialize the long-term memory. The use of memory improved the technical validity of the generated Cypher queries, with the maximum increase in the KG Valid metric reaching 21 percentage points. The greatest benefit was observed for the smallest language model, whereas improvements in agreement with the reference execution results were more limited. Correction-trajectory retrieval showed the most consistent positive effect on technical validity, although no single memory configuration outperformed the others across all models and metrics.

Overall, selectively retrieving previous experience enhanced the agentic workflow's ability to produce technically valid Cypher queries. Memory neither replaces the capabilities of the underlying language model nor guarantees semantic correctness, but enables the system to reuse experience from previous executions.

Keywords: conversational AI agents, agentic AI, memory management, episodic memory, long-term memory, knowledge graphs, Text-to-Cypher, Cypher, Large Language Models, semantic retrieval, Neo4j, CyVerACT

Ευχαριστίες

Με την ολοκλήρωση της παρούσας διπλωματικής εργασίας ολοκληρώνεται και ένα σημαντικό κεφάλαιο της ακαδημαϊκής μου πορείας. Για τον λόγο αυτό, θα ήθελα να εκφράσω την ειλικρινή μου ευγνωμοσύνη στους ανθρώπους που στάθηκαν δίπλα μου και με στήριξαν καθ' όλη τη διάρκεια των προπτυχιακών μου σπουδών.

Αρχικά, θα ήθελα να ευχαριστήσω θερμά τον Καθηγητή κ. Συμεών Παπαβασιλείου για την πολύτιμη βοήθεια και καθοδήγησή του στην επιλογή και τη διαμόρφωση του θέματος της παρούσας εργασίας. Επιπλέον, θα ήθελα να ευχαριστήσω την Ερευνήτρια του Ε.Μ.Π. κ. Ιωάννα Μανδηλαρά, τη Δόκτωρα κ. Χριστίνα-Μαρία Ανδρωνά και τον Δόκτωρα κ. Αναστάσιο Ζαφειρόπουλο για την ουσιαστική καθοδήγηση, τη συνεργασία και την υποστήριξή τους κατά την εκπόνηση της διπλωματικής μου εργασίας.

Ιδιαίτερη αναφορά θα ήθελα να κάνω στη φίλη μου Κατερίνα Σταύρου, την οποία ευχαριστώ για την υποστήριξη και τη συνεργασία μας σε τόσες εργασίες και εργαστήρια της Σχολής, αλλά κυρίως για το γέλιο, την αγάπη και τις όμορφες αναμνήσεις που δημιουργήσαμε όλα αυτά τα χρόνια. Θα ήθελα επίσης να ευχαριστήσω τη στενή μου φίλη Ζίνα Αμπντέλ-Αζίζ, φοιτήτρια στη Σχολή Ναυπηγών Μηχανολόγων Μηχανικών του Ε.Μ.Π., η οποία βρίσκεται δίπλα μου από τα χρόνια του Δημοτικού μέχρι και σήμερα, στηρίζοντάς με σε κάθε δυσκολία και ενθαρρύνοντάς με να κυνηγώ και να πραγματοποιώ τα όνειρά μου.

Η ολοκλήρωση των προπτυχιακών μου σπουδών δεν θα ήταν δυνατή χωρίς τη διαρκή στήριξη και την αγάπη της οικογένειάς μου. Ευχαριστώ μέσα από την καρδιά μου τη μητέρα μου και τον πατέρα μου για όλα όσα μου προσέφεραν και συνεχίζουν να μου προσφέρουν, καθώς και τις αδερφές μου, Ελένη και Χριστίνα, για την αγάπη και τη στήριξή τους. Τους είμαι βαθιά ευγνώμων και θα ήθελα να αφιερώσω την παρούσα διπλωματική εργασία στην οικογένειά μου.

Τέλος, θα ήθελα να ευχαριστήσω τον Θεό, που μου έδωσε την υπομονή στις στιγμές της αμφιβολίας και την έμπνευση στις στιγμές της σιωπής.

Όλγα Γιαννούχου
Οκτώβριος 2026

Περιεχόμενα

Περίληψη **5**

Abstract **7**

Ευχαριστίες **9**

Κατάλογος Σχημάτων **14**

Κατάλογος Πινάκων **15**

1 Εισαγωγή **16**
1.1 Κίνητρο Εργασίας 17
1.2 Στόχοι Διπλωματικής 19
1.3 Συνεισφορές 20
1.4 Δομή Εργασίας 21

2 Διαχείριση Μνήμης σε Πράκτορες Τεχνητής Νοημοσύνης **22**
2.1 Η Έννοια και ο Ρόλος της Μνήμης 22
2.2 Κατηγορίες Μνήμης 23
2.3 Επεισοδιακή Μνήμη 26
2.4 Βασικές Λειτουργίες Συστημάτων Μνήμης 27
2.5 Αναπαράσταση Μνήμης 31
2.6 Γραφική Αναπαράσταση Μνήμης με Γράφους Γνώσης 32
2.6.1 Βασικές Έννοιες και Μοντέλο Γράφου Ιδιοτήτων 32
2.6.2 Γράφοι Γνώσης ως Δομές Μνήμης 33
2.6.3 Διανυσματική, Γραφική και Υβριδική Αναπαράσταση Μνήμης . 34
2.7 Μεγάλα Γλωσσικά Μοντέλα και Γράφοι Γνώσης 36
2.7.1 Αλληλεπίδραση Μεγάλων Γλωσσικών Μοντέλων και Γράφων Γνώσης 36
2.7.2 Text-to-Cypher και η Αρχιτεκτονική CyVerACT 36
2.8 Σχετικές Αρχιτεκτονικές Μακροπρόθεσμης Μνήμης 38
2.8.1 MemGPT 38

2.8.2 AriGraph και A-MEM . . . 39
2.8.3 MIRIX και Memp . . . 40
2.9 Πλαίσια και Δομές Διαχείρισης Μνήμης . . . 43
2.9.1 Neo4j ως Υποδομή Γραφικής Μνήμης . . . 43
2.9.2 Zep και Graphiti . . . 44
2.9.3 Cognee . . . 44
2.9.4 Mem0 . . . 45
2.9.5 LangChain και LangGraph . . . 46

3 Μεθοδολογία **47**
3.1 Επισκόπηση της Προτεινόμενης Μεθόδου . . . 47
3.2 Βασική Πρακτορική Ροή CyVerACT . . . 48
3.2.1 Φιλτράρισμα του Σχήματος και Παραγωγή Cypher . . . 50
3.2.2 Αξιολόγηση και Επαναληπτική Διόρθωση . . . 50
3.2.3 Ανάκτηση Πλήρους Σχήματος και Ολοκλήρωση της Ροής . . . 51
3.3 Διερεύνηση Πλαισίων Μνήμης και Επιλογή Προσαρμοσμένης Λύσης . . . 51
3.3.1 Πειραματισμός με το Cognee . . . 52
3.3.2 Διερεύνηση Zep και Graphiti . . . 55
3.3.3 Επιλογή Προσαρμοσμένου Μηχανισμού Μνήμης . . . 56
3.4 Σχεδιασμός του Γράφου Μνήμης . . . 58
3.4.1 Κόμβοι και Σχέσεις του Γράφου Μνήμης . . . 58
3.4.2 Κατηγορίες Σφαλμάτων και Κανόνες Διόρθωσης . . . 60
3.4.3 Διανυσματικές Αναπαραστάσεις της Μνήμης . . . 61
3.5 Λειτουργία Αποθήκευσης και Οργάνωσης της Μνήμης . . . 62
3.5.1 Καταγραφή Προσπαθειών και Σφαλμάτων . . . 62
3.5.2 Αποθήκευση Επιτυχημένων Λύσεων . . . 63
3.6 Λειτουργία Ανάκτησης και Αξιοποίησης της Μνήμης . . . 64
3.6.1 Ανάκτηση Παρόμοιας Επιτυχημένης Ερώτησης . . . 65
3.6.2 Βραχυπρόθεσμο Ιστορικό της Τρέχουσας Εκτέλεσης . . . 66
3.6.3 Ανάκτηση Προηγούμενων Τροχιών Διόρθωσης . . . 67
3.6.4 Ανάκτηση Γενικών Κανόνων Διόρθωσης . . . 70
3.7 Αποθετήριο Κώδικα της Εργασίας . . . 70
3.8 Σύνοψη Κεφαλαίου . . . 71

4 Πειραματική Αξιολόγηση **72**
4.1 Πειραματική Διάταξη . . . 72
4.1.1 Βάση και Περιβάλλον Εκτέλεσης . . . 73
4.1.2 Σύνολο Δεδομένων και Διαχωρισμός Ερωτημάτων . . . 73
4.1.3 Γλωσσικά Μοντέλα και Παράμετροι Εκτέλεσης . . . 76
4.1.4 Πειραματικές Διαμορφώσεις Μνήμης . . . 77

4.2 Μετρικές Αξιολόγησης 78
4.2.1 CyVer Μετρικές 79
4.2.2 Μετρικές Αποτελέσματος Εκτέλεσης 79
4.2.3 Μετρικές Ομοιότητας Ερωτημάτων Cypher 81
4.2.4 Διορθωτικές Επαναλήψεις 81
4.3 Συνολικά Πειραματικά Αποτελέσματα 82
4.3.1 Llama-3.1-8B-Instruct 87
4.3.2 Qwen-2.5-7B-Instruct 90
4.3.3 Llama-3.2-3B-Instruct 93
4.4 Επίδραση των Μηχανισμών Μνήμης 96
4.4.1 Επίδραση στην Εγκυρότητα των Παραγόμενων Ερωτημάτων . . 96
4.4.2 Επίδραση στην Ορθότητα των Αποτελεσμάτων 97
4.4.3 Επίδραση στη Διαδικασία Διόρθωσης 99
4.4.4 Επίδραση στον Χρόνο Εκτέλεσης 100
4.4.5 Επίδραση ως προς τη Δυσκολία των Ερωτημάτων 102
4.5 Παρατηρήσεις επί των Αποτελεσμάτων 105
4.6 Περιορισμοί της Πειραματικής Αξιολόγησης 105
4.7 Σύνοψη Κεφαλαίου 107

5 Συμπεράσματα και Μελλοντικές Επεκτάσεις **108**
5.1 Συμπεράσματα 108
5.2 Μελλοντικές Επεκτάσεις 110

A Προτροπές των Πειραματικών και Τελικών Ροών **114**
A.1 Προτροπή του Cypher Generator 114
A.2 Προτροπή του Cypher Corrector 115
A.3 Δυναμικά Συστατικά Μνήμης των Προτροπών 116
A.3.1 Ανακτημένο Επιτυχημένο Παράδειγμα 116
A.3.2 Συμπιεσμένο Βραχυπρόθεσμο Ιστορικό 117
A.3.3 Ανακτημένη Τροχιά Διόρθωσης 117
A.3.4 Γενικοί Κανόνες Διόρθωσης (FixRules) 117
A.4 Πειραματικές Προτροπές της Ενσωμάτωσης Cognee 119
A.4.1 Προτροπή Κατασκευής του Γράφου Μνήμης 119
A.4.2 Προτροπή Αποθήκευσης Αποτυχημένης Προσπάθειας 120
A.4.3 Προτροπή Ανάκτησης Κανόνων 121
A.4.4 Ενσωμάτωση του Ανακτημένου Πλαισίου στη Ροή 121

Βιβλιογραφία **123**

Κατάλογος Σχημάτων

2.1 Βασικές κατηγορίες μνήμης σε πράκτορες Τεχνητής Νοημοσύνης . . . 24
2.2 Βασικές λειτουργίες συστήματος μακροπρόθεσμης μνήμης 30
2.3 Βασικές μορφές αναπαράστασης μνήμης 31

3.1 Βασική πρακτορική ροή του CyVerACT 49
3.2 Πειραματική ενσωμάτωση του Cognee 53
3.3 Επέκταση του CyVerACT με επίμονη μνήμη 57
3.4 Σχήμα του προσαρμοσμένου γράφου μνήμης 60
3.5 Ροή εγγραφής στη μακροπρόθεσμη μνήμη 64
3.6 Μορφές ανακτημένης πληροφορίας και αξιοποίησή τους 65
3.7 Παράδειγμα αξιοποίησης της μνήμης στην πρακτορική ροή 69

4.1 Συχνότητα χαρακτηριστικών όρων Cypher 75
4.2 Κατανομή δυσκολίας ερωτημάτων 76
4.3 Εγκυρότητα για το Llama-3.1-8B 87
4.4 Pass@1 για το Llama-3.1-8B 88
4.5 Θετική Jaccard για το Llama-3.1-8B 89
4.6 Εγκυρότητα για το Qwen-2.5-7B 90
4.7 Pass@1 για το Qwen-2.5-7B 91
4.8 Θετική Jaccard για το Qwen-2.5-7B 92
4.9 Εγκυρότητα για το Llama-3.2-3B 93
4.10 Pass@1 για το Llama-3.2-3B 94
4.11 Θετική Jaccard για το Llama-3.2-3B 95
4.12 KG Valid ανά επίπεδο δυσκολίας 103

Κατάλογος Πινάκων

2.1 Συγκριτική σύνοψη αρχιτεκτονικών μακροπρόθεσμης μνήμης 42

3.1 Αποτίμηση της διερεύνησης του Cognee 55
3.2 Βασικοί τύποι κόμβων του γράφου μνήμης 59
3.3 Διανυσματικές αναπαραστάσεις του μηχανισμού μνήμης 62

4.1 Παράμετροι σύνδεσης με τη βάση Movies 73
4.2 Κριτήρια ταξινόμησης ερωτημάτων 74
4.3 Κοινές παράμετροι πειραματικών εκτελέσεων 76
4.4 Πειραματικές διαμορφώσεις μνήμης 78
4.5 Σύνοψη μετρικών αξιολόγησης 82
4.6 Συγκεντρωτικά αποτελέσματα κύριων μετρικών 84
4.7 Συγκεντρωτικά αποτελέσματα μετρικών ομοιότητας Cypher 85
4.8 Μεταβολή της KG Valid 96
4.9 Μεταβολή της Pass@1 Output 98
4.10 Μέσος συνολικός αριθμός ενεργοποιήσεων του Cypher Corrector 100
4.11 Συνολικός χρόνος εκτέλεσης ανά διαμόρφωση 101
4.12 Μέσος χρόνος εκτέλεσης ανά ερώτημα 101

A.1 Προκαθορισμένοι κανόνες διόρθωσης 118

Κεφάλαιο 1

Εισαγωγή

Η ραγδαία εξέλιξη των Μεγάλων Γλωσσικών Μοντέλων (Large Language Models – LLMs) έχει επηρεάσει σημαντικά τον τρόπο με τον οποίο σχεδιάζονται σύγχρονα συστήματα Τεχνητής Νοημοσύνης. Πέρα από την παραγωγή και κατανόηση φυσικής γλώσσας, τα μοντέλα αυτά αξιοποιούνται πλέον ως βασικά δομικά στοιχεία συνομιλιακών και αυτόνομων πρακτόρων, οι οποίοι μπορούν να αλληλεπιδρούν με το περιβάλλον τους, να χρησιμοποιούν εξωτερικά εργαλεία και πηγές γνώσης και να εκτελούν σύνθετες ακολουθίες ενεργειών για την επίτευξη ενός στόχου. Στο πλαίσιο αυτό, η έννοια των συστημάτων Τεχνητής Νοημοσύνης που βασίζονται σε αυτόνομους πράκτορες (Agentic AI) επεκτείνει το παραδοσιακό μοντέλο ενός συστήματος ερώτησης-απάντησης προς πιο δυναμικές αρχιτεκτονικές, στις οποίες ο πράκτορας καλείται να αξιοποιήσει πληροφορίες από προηγούμενες αλληλεπιδράσεις και να προσαρμόσει τη συμπεριφορά του με βάση την αποκτηθείσα εμπειρία [1, 2].

Ένα από τα βασικά ζητήματα που προκύπτουν κατά την ανάπτυξη τέτοιων πρακτόρων είναι η αποτελεσματική διαχείριση της μνήμης. Η διατήρηση ολόκληρου του ιστορικού μιας αλληλεπίδρασης μέσα στο διαθέσιμο πλαίσιο συμφραζομένων (context window) ενός γλωσσικού μοντέλου δεν αποτελεί από μόνη της ολοκληρωμένο μηχανισμό μακροπρόθεσμης μνήμης. Καθώς ο όγκος των προηγούμενων αλληλεπιδράσεων αυξάνεται, καθίσταται αναγκαία η επιλογή, η οργάνωση και η ανάκτηση των πληροφοριών που σχετίζονται με την εκάστοτε εργασία. Προσεγγίσεις όπως το MemGPT εισάγουν μηχανισμούς διαχείρισης διαφορετικών επιπέδων μνήμης [1], ενώ η επεισοδιακή μνήμη (episodic memory) αντιμετωπίζει προηγούμενες αλληλεπιδράσεις και ενέργειες ως εμπειρίες που μπορούν να ανακληθούν και να αξιοποιηθούν σε μεταγενέστερα συναφή προβλήματα [3]. Για την αποτελεσματική λειτουργία ενός τέτοιου μηχανισμού απαιτούνται κατάλληλες μέθοδοι οργάνωσης και συσχέτισης των αποθηκευμένων πληροφοριών, με σύγχρονες προσεγγίσεις να διερευνούν δυναμικές δομές μνήμης που εξελίσσονται καθώς προστίθεται νέα γνώση [4].

Κεντρικό ρόλο στη δομημένη αναπαράσταση της γνώσης έχουν οι γράφοι γνώσης (Knowledge Graphs – KGs). Οι γράφοι γνώσης οργανώνουν την πληροφορία με βάση

οντότητες και τις μεταξύ τους σχέσεις, παρέχοντας έτσι έναν φυσικό τρόπο αναπαράστασης σύνθετης και έντονα συσχετισμένης γνώσης. Η ενσωμάτωσή τους σε συστήματα που βασίζονται σε Μεγάλα Γλωσσικά Μοντέλα μπορεί να συνδυάσει τις δυνατότητες κατανόησης και παραγωγής φυσικής γλώσσας των μοντέλων αυτών με μία πιο δομημένη αναπαράσταση της διαθέσιμης πληροφορίας. Η αποτελεσματική αλληλεπίδραση μεταξύ γλωσσικών μοντέλων και γράφων γνώσης αποτελεί, επομένως, σημαντικό ερευνητικό αντικείμενο, ιδιαίτερα σε εφαρμογές που απαιτούν σύνθετη συλλογιστική πάνω σε συσχετισμένες πληροφορίες [5].

Ο συνδυασμός μηχανισμών μνήμης με δομημένες αναπαραστάσεις γνώσης δημιουργεί τη δυνατότητα ανάπτυξης πρακτόρων που δεν αντιμετωπίζουν κάθε νέα αλληλεπίδραση ανεξάρτητα, αλλά αξιοποιούν πληροφορίες και εμπειρίες που έχουν αποκτηθεί σε προηγούμενα στάδια λειτουργίας. Προσεγγίσεις όπως το AriGraph διερευνούν τη σύνδεση γράφων γνώσης και επεισοδιακής μνήμης, με στόχο την αξιοποίηση προηγούμενων εμπειριών κατά τη μετέπειτα λειτουργία ενός πράκτορα [6]. Χαρακτηριστική κατηγορία εφαρμογών αποτελούν εκείνες στις οποίες ο χρήστης διατυπώνει ερωτήματα σε φυσική γλώσσα και το σύστημα καλείται να τα μετατρέψει σε κατάλληλα ερωτήματα προς έναν γράφο γνώσης, διαδικασία που συνήθως αναφέρεται ως παραγωγή ερωτημάτων από φυσική γλώσσα προς Cypher (Text-to-Cypher generation). Σε τέτοια συστήματα, η αξιοποίηση προηγούμενης εμπειρίας μπορεί να αποτελέσει πρόσθετη πηγή πληροφορίας για την παραγωγή ορθότερων ερωτημάτων. Στο πλαίσιο αυτό εντάσσεται η παρούσα διπλωματική εργασία, η οποία εξετάζει τη διαχείριση και την αξιοποίηση μνήμης σε συνομιλιακούς πράκτορες για την υποστήριξη ερωτημάτων πάνω σε γράφους γνώσης.

1.1 Κίνητρο Εργασίας

Η δυνατότητα υποβολής ερωτημάτων σε γράφους γνώσης μέσω φυσικής γλώσσας μπορεί να διευκολύνει σημαντικά την πρόσβαση σε σύνθετες δομές δεδομένων, καθώς απαλλάσσει τον χρήστη από την ανάγκη άμεσης γνώσης μιας εξειδικευμένης γλώσσας ερωτημάτων. Στην περίπτωση γράφων που αποθηκεύονται σε συστήματα όπως το Neo4j, η πρόθεση του χρήστη πρέπει να μετασχηματιστεί σε ένα εκτελέσιμο ερώτημα στη γλώσσα Cypher. Η διαδικασία αυτή, γνωστή ως παραγωγή ερωτημάτων φυσικής γλώσσας σε Cypher (Text-to-Cypher generation), μπορεί να υποστηριχθεί από Μεγάλα Γλωσσικά Μοντέλα, τα οποία αξιοποιούν τόσο το αρχικό ερώτημα του χρήστη όσο και πληροφορίες σχετικά με το σχήμα του γράφου για την παραγωγή του αντίστοιχου ερωτήματος.

Παρά τις σημαντικές δυνατότητες των Μεγάλων Γλωσσικών Μοντέλων, η παραγωγή ενός έγκυρου και σημασιολογικά ορθού ερωτήματος Cypher δεν είναι εγγυημένη. Το παραγόμενο ερώτημα μπορεί να περιέχει συντακτικά σφάλματα, να αναφέρεται σε ετικέτες, τύπους σχέσεων ή ιδιότητες που δεν υπάρχουν στον γράφο, να χρησιμο-

ποιεί λανθασμένη κατεύθυνση στις σχέσεις μεταξύ των οντοτήτων ή να διατυπώνει ένα μονοπάτι που δεν είναι συμβατό με το διαθέσιμο σχήμα του γράφου. Ακόμη και ένα συντακτικά έγκυρο ερώτημα μπορεί να αποτύχει να εκφράσει σωστά την πρόθεση του χρήστη ή να μην επιστρέψει το αναμενόμενο αποτέλεσμα. Κατά συνέπεια, η παραγωγή ερωτημάτων προς γράφους γνώσης αποτελεί διαδικασία στην οποία η αρχική έξοδος του γλωσσικού μοντέλου συχνά χρειάζεται έλεγχο και ενδεχόμενη διόρθωση [7, 8].

Για την αντιμετώπιση τέτοιων περιπτώσεων μπορούν να χρησιμοποιηθούν πρακτορικές ροές εργασίας (agentic workflows), στις οποίες το αρχικά παραγόμενο ερώτημα δεν εκτελείται άμεσα ως τελικό αποτέλεσμα, αλλά ελέγχεται ως προς την εγκυρότητά του και, όταν εντοπίζονται σφάλματα, επιστρέφεται σε έναν μηχανισμό διόρθωσης. Με αυτόν τον τρόπο είναι δυνατή η πραγματοποίηση διαδοχικών προσπαθειών μέχρι να παραχθεί ένα ερώτημα που είναι συμβατό με το σχήμα του γράφου και μπορεί να εκτελεστεί επιτυχώς. Η επαναληπτική αυτή διαδικασία αυξάνει την ανθεκτικότητα του συστήματος απέναντι σε λανθασμένες αρχικές προβλέψεις του γλωσσικού μοντέλου, δεν εξαλείφει όμως από μόνη της έναν σημαντικό περιορισμό: η γνώση που αποκτάται κατά την επίλυση ενός ερωτήματος δεν αξιοποιείται απαραίτητα κατά την επεξεργασία ενός επόμενου.

Έτσι, ένας πράκτορας μπορεί να αντιμετωπίσει σε διαφορετικές χρονικές στιγμές παρόμοια προβλήματα χωρίς να γνωρίζει ότι ένα αντίστοιχο σφάλμα έχει ήδη παρουσιαστεί και διορθωθεί στο παρελθόν. Για παράδειγμα, μία προηγούμενη προσπάθεια μπορεί να έχει αποκαλύψει τη σωστή κατεύθυνση μιας σχέσης, έναν έγκυρο τρόπο σύνταξης ενός ερωτήματος ή μία επιτυχημένη δομή Cypher για την αντιμετώπιση ενός συγκεκριμένου τύπου προβλήματος. Εάν η πληροφορία αυτή δε διατηρείται και δεν ανακτάται σε μεταγενέστερα συναφή ερωτήματα, το σύστημα ενδέχεται να επαναλάβει το ίδιο ή ένα παρόμοιο λάθος και να χρειαστεί εκ νέου πολλαπλές προσπάθειες για την εύρεση της σωστής λύσης. Η συμπεριφορά αυτή περιορίζει τόσο την αποδοτικότητα όσο και τη δυνατότητα του πράκτορα να βελτιώνεται μέσα από την προηγούμενη λειτουργία του.

Το παραπάνω πρόβλημα δημιουργεί το βασικό κίνητρο για την εισαγωγή μηχανισμών μνήμης στη διαδικασία παραγωγής και διόρθωσης ερωτημάτων Cypher. Αντί κάθε νέο ερώτημα να αντιμετωπίζεται ως μία πλήρως ανεξάρτητη διαδικασία, ένας πράκτορας με επεισοδιακή μνήμη μπορεί να διατηρεί πληροφορίες σχετικά με προηγούμενες ερωτήσεις, τις παραγόμενες ενδιάμεσες προσπάθειες, τα σφάλματα που παρουσιάστηκαν και τις τελικές επιτυχημένες λύσεις. Στη συνέχεια, όταν εμφανιστεί ένα νέο ερώτημα ή σφάλμα με παρόμοια χαρακτηριστικά, σχετικές προηγούμενες εμπειρίες μπορούν να ανακτηθούν και να χρησιμοποιηθούν ως πρόσθετο πλαίσιο κατά την παραγωγή ή τη διόρθωση του νέου ερωτήματος. Η λογική αυτή συνδέεται με τη γενικότερη αξιοποίηση επεισοδιακής μνήμης σε πράκτορες Τεχνητής Νοημοσύνης, όπου οι προηγούμενες αλληλεπιδράσεις δεν αντιμετωπίζονται ως παθητικό ιστορικό αλλά ως εμπειρίες που

μπορούν να καθοδηγήσουν μελλοντικές ενέργειες [3, 6].

Ένα ακόμη ζήτημα αφορά τη δυνατότητα γενίκευσης της αποκτηθείσας εμπειρίας. Η χρησιμότητα μιας προηγούμενης διόρθωσης δεν περιορίζεται απαραίτητα στο ίδιο ακριβώς ερώτημα ή στον ίδιο γράφο γνώσης. Ένα σφάλμα που σχετίζεται, για παράδειγμα, με τη λανθασμένη κατεύθυνση μιας σχέσης ή με έναν μη έγκυρο τρόπο διατύπωσης ενός μοτίβου στη γλώσσα Cypher μπορεί να εμφανιστεί ξανά σε διαφορετικό ερώτημα και σε διαφορετικό σχήμα. Η ανάκτηση προηγούμενων αλληλεπιδράσεων μπορεί επομένως να παρέχει στον πράκτορα παραδείγματα του τρόπου με τον οποίο παρόμοια προβλήματα αντιμετωπίστηκαν στο παρελθόν, χωρίς να απαιτείται η παραγωγή και ενσωμάτωση ειδικών κανόνων για κάθε πιθανή περίπτωση.

Με βάση τα παραπάνω, το κεντρικό κίνητρο της παρούσας διπλωματικής εργασίας είναι η διερεύνηση του κατά πόσο η ενσωμάτωση ενός δομημένου μηχανισμού μνήμης σε μία υπάρχουσα πρακτορική ροή παραγωγής ερωτημάτων Cypher μπορεί να περιορίσει την επανάληψη προηγούμενων σφαλμάτων και να υποστηρίξει αποτελεσματικότερα τη δημιουργία έγκυρων ερωτημάτων σε γράφους γνώσης. Η βασική υπόθεση είναι ότι η συστηματική αποθήκευση και ανάκτηση σχετικών προηγούμενων εμπειριών μπορεί να μετατρέψει μία κατά βάση επαναληπτική διαδικασία παραγωγής και διόρθωσης σε ένα σύστημα που αξιοποιεί τη συσσωρευμένη εμπειρία του και προσαρμόζει τη μελλοντική του συμπεριφορά βάσει αυτής.

1.2 Στόχοι Διπλωματικής

Πρώτος στόχος της παρούσας διπλωματικής εργασίας είναι η μελέτη και η παρουσίαση της υφιστάμενης βιβλιογραφίας σχετικά με τη διαχείριση μνήμης σε πράκτορες Τεχνητής Νοημοσύνης, με έμφαση σε προσεγγίσεις μακροπρόθεσμης και επεισοδιακής μνήμης, καθώς και στη χρήση γράφων γνώσης για τη δομημένη αναπαράσταση και ανάκτηση προηγούμενων εμπειριών. Παράλληλα, διερευνώνται υφιστάμενες αρχιτεκτονικές και πλαίσια διαχείρισης μνήμης, με στόχο την κατανόηση των δυνατοτήτων και των περιορισμών τους και την ανάδειξη των χαρακτηριστικών που είναι πιο κατάλληλα για την υποστήριξη πρακτορικών ροών εργασίας παραγωγής ερωτημάτων προς γράφους γνώσης.

Βασικός στόχος της εργασίας είναι η ανάπτυξη και ενσωμάτωση ενός δομημένου μηχανισμού επεισοδιακής μνήμης σε μία υπάρχουσα πρακτορική ροή παραγωγής και διόρθωσης ερωτημάτων Cypher. Ο μηχανισμός αυτός σχεδιάζεται ώστε να διατηρεί και να ανακτά πληροφορίες από προηγούμενες αλληλεπιδράσεις του πράκτορα και να τις παρέχει στα κατάλληλα στάδια της ροής εργασίας. Η εργασία εστιάζει επίσης στη σημασιολογική ανάκτηση σχετικών εμπειριών και στη διερεύνηση διαφορετικών τρόπων αξιοποίησής τους κατά την παραγωγή και τη διόρθωση νέων ερωτημάτων.

Τέλος, εξίσου σημαντική είναι η πειραματική αξιολόγηση της προτεινόμενης προ-

σέγγισης και η διερεύνηση της πραγματικής συμβολής της μνήμης στην απόδοση του συστήματος. Για τον σκοπό αυτό εξετάζονται διαφορετικοί τρόποι αξιοποίησης της αποθηκευμένης εμπειρίας και συγκρίνονται με μία βασική εκδοχή της πρακτορικής ροής εργασίας χωρίς μηχανισμό μνήμης. Η αξιολόγηση αποσκοπεί στη μελέτη της επίδρασης της μνήμης στην εγκυρότητα και την ορθότητα των παραγόμενων ερωτημάτων Cypher, στη διαδικασία διορθωτικών επαναλήψεων και στον χρόνο εκτέλεσης του συστήματος.

1.3 Συνεισφορές

Οι κύριες συνεισφορές της παρούσας διπλωματικής εργασίας συνοψίζονται στα ακόλουθα σημεία:

- Μελέτη της σχετικής βιβλιογραφίας και διερεύνηση υφιστάμενων προσεγγίσεων και πλαισίων για τη διαχείριση μνήμης σε πράκτορες Τεχνητής Νοημοσύνης, με στόχο την αποτίμηση των δυνατοτήτων και των περιορισμών τους και την αξιοποίηση των σχετικών συμπερασμάτων κατά τον σχεδιασμό της προτεινόμενης λύσης.
- Σχεδιασμός και ανάπτυξη ενός προσαρμοσμένου μηχανισμού επεισοδιακής μνήμης για την αποθήκευση και αξιοποίηση πληροφοριών που προκύπτουν κατά την παραγωγή και τη διόρθωση ερωτημάτων Cypher.
- Ενσωμάτωση του μηχανισμού μνήμης σε υπάρχουσα πρακτορική ροή εργασίας παραγωγής ερωτημάτων από φυσική γλώσσα προς Cypher, με δυνατότητα αξιοποίησης της προηγούμενης εμπειρίας σε διαφορετικά στάδια της διαδικασίας παραγωγής και διόρθωσης.
- Ανάπτυξη μηχανισμών σημασιολογικής ανάκτησης σχετικών προηγούμενων εμπειριών, τόσο από επιτυχημένες αλληλεπιδράσεις όσο και από προηγούμενες αποτυχημένες προσπάθειες και τα αντίστοιχα σφάλματα.
- Διερεύνηση διαφορετικών τρόπων αξιοποίησης της μνήμης και κατασκευή επιμέρους παραλλαγών της πρακτορικής ροής εργασίας, με στόχο τη μελέτη της επίδρασης διαφορετικών μορφών προηγούμενης εμπειρίας στην παραγωγή και τη διόρθωση ερωτημάτων Cypher.
- Πειραματική αξιολόγηση της προτεινόμενης προσέγγισης και σύγκρισή της με βασική εκδοχή του συστήματος χωρίς μνήμη, με τη χρήση διαφορετικών Μεγάλων Γλωσσικών Μοντέλων και ενός κοινού συνόλου ερωτημάτων αξιολόγησης, ώστε να αποτιμηθεί η επίδραση της μνήμης στην εγκυρότητα και την ορθότητα των παραγόμενων ερωτημάτων Cypher, στη διαδικασία διορθωτικών επαναλήψεων και στον χρόνο εκτέλεσης του συστήματος.

1.4 Δομή Εργασίας

Η παρούσα διπλωματική εργασία οργανώνεται ως εξής. Στο Κεφάλαιο 1 παρουσιάζεται το γενικό πλαίσιο της εργασίας και αναλύονται το κίνητρο, οι βασικοί στόχοι και οι κύριες συνεισφορές της. Στο Κεφάλαιο 2 παρουσιάζεται το απαραίτητο θεωρητικό υπόβαθρο και εξετάζεται η σχετική βιβλιογραφία γύρω από τη διαχείριση μνήμης σε πράκτορες Τεχνητής Νοημοσύνης, τους γράφους γνώσης και τις υφιστάμενες προσεγγίσεις για τη δομημένη αναπαράσταση και μακροπρόθεσμη διαχείριση της μνήμης τους. Επιπρόσθετα, παρουσιάζονται σχετικές αρχιτεκτονικές και εργαλεία, καθώς και η σύνδεση Μεγάλων Γλωσσικών Μοντέλων και γράφων γνώσης για την παραγωγή δομημένων ερωτημάτων.

Στο Κεφάλαιο 3 περιγράφεται αναλυτικά η μεθοδολογία της εργασίας. Αρχικά παρουσιάζεται η πρακτορική ροή παραγωγής και διόρθωσης ερωτημάτων Cypher και στη συνέχεια αναλύεται ο σχεδιασμός του προτεινόμενου μηχανισμού μνήμης, ο τρόπος αποθήκευσης της προηγούμενης εμπειρίας και οι μηχανισμοί ανάκτησης και αξιοποίησής της στα διαφορετικά στάδια της ροής εργασίας. Στο Κεφάλαιο 4 παρουσιάζεται η πειραματική διαδικασία και πραγματοποιείται η αξιολόγηση των διαφορετικών παραλλαγών της προτεινόμενης προσέγγισης. Περιγράφονται τα χρησιμοποιούμενα σύνολα δεδομένων, τα γλωσσικά μοντέλα και οι μετρικές αξιολόγησης, παρουσιάζονται και αναλύονται τα αποτελέσματα των πειραμάτων και η σύγκριση με τη βασική εκδοχή του συστήματος χωρίς μνήμη, ενώ συζητούνται και οι περιορισμοί της προτεινόμενης προσέγγισης.

Τέλος, στο Κεφάλαιο 5 συνοψίζονται τα κυριότερα συμπεράσματα της εργασίας και παρουσιάζονται πιθανές κατευθύνσεις για μελλοντική έρευνα και επέκταση του συστήματος.

Κεφάλαιο 2

Διαχείριση Μνήμης σε Πράκτορες Τεχνητής Νοημοσύνης

2.1 Η Έννοια και ο Ρόλος της Μνήμης

Η μνήμη αποτελεί βασικό συστατικό των πρακτόρων Τεχνητής Νοημοσύνης που βασίζονται σε Μεγάλα Γλωσσικά Μοντέλα, καθώς επιτρέπει τη διατήρηση και επαναχρησιμοποίηση πληροφορίας που έχει αποκτηθεί κατά τη διάρκεια προηγούμενων αλληλεπιδράσεων. Στο πλαίσιο αυτό, ως σύστημα μνήμης μπορεί να θεωρηθεί το σύνολο των μηχανισμών μέσω των οποίων ένας πράκτορας αποθηκεύει, οργανώνει, ενημερώνει και ανακτά πληροφορίες που είναι δυνατό να επηρεάσουν τη μελλοντική του συμπεριφορά [2, 9]. Η λειτουργία αυτή αποκτά μεγάλη σημασία σε πράκτορες που εκτελούν ακολουθίες ενεργειών ή λειτουργούν σε βάθος χρόνου, καθώς η αποτελεσματική αξιοποίηση της προηγούμενης εμπειρίας απαιτεί κάτι περισσότερο από την απλή διατήρηση ενός ιστορικού αλληλεπίδρασης.

Το διαθέσιμο πλαίσιο συμφραζομένων (context window) ενός Μεγάλου Γλωσσικού Μοντέλου παρέχει έναν προσωρινό χώρο στον οποίο μπορούν να τοποθετηθούν πληροφορίες σχετικές με την τρέχουσα αλληλεπίδραση. Η παρουσία μιας πληροφορίας στο πλαίσιο αυτό δεν ισοδυναμεί, ωστόσο, με μακροπρόθεσμη αποθήκευσή της. Καθώς η αλληλεπίδραση εξελίσσεται και ο όγκος της διαθέσιμης πληροφορίας αυξάνεται, απαιτούνται μηχανισμοί που αποφασίζουν ποια πληροφορία είναι χρήσιμο να διατηρηθεί, με ποια μορφή θα αποθηκευτεί και υπό ποιες συνθήκες θα ανακτηθεί στο μέλλον. Κατά συνέπεια, η διαχείριση μνήμης (memory management) δεν αφορά απλώς την αύξηση της ποσότητας του διαθέσιμου ιστορικού, αλλά τη συστηματική διαχείριση της πληροφορίας που μπορεί να αξιοποιήσει ο πράκτορας [1, 2].

Χαρακτηριστικό παράδειγμα αποτελεί το MemGPT, το οποίο αντιμετωπίζει τη μνήμη ως έναν διαχειριζόμενο πόρο και οργανώνει την πληροφορία σε διαφορετικά επίπεδα, μεταφέροντάς την επιλεκτικά ανάλογα με τις ανάγκες του γλωσσικού μοντέλου [1]. Η

αρχιτεκτονική του παρουσιάζεται αναλυτικότερα στην Ενότητα 2.8.

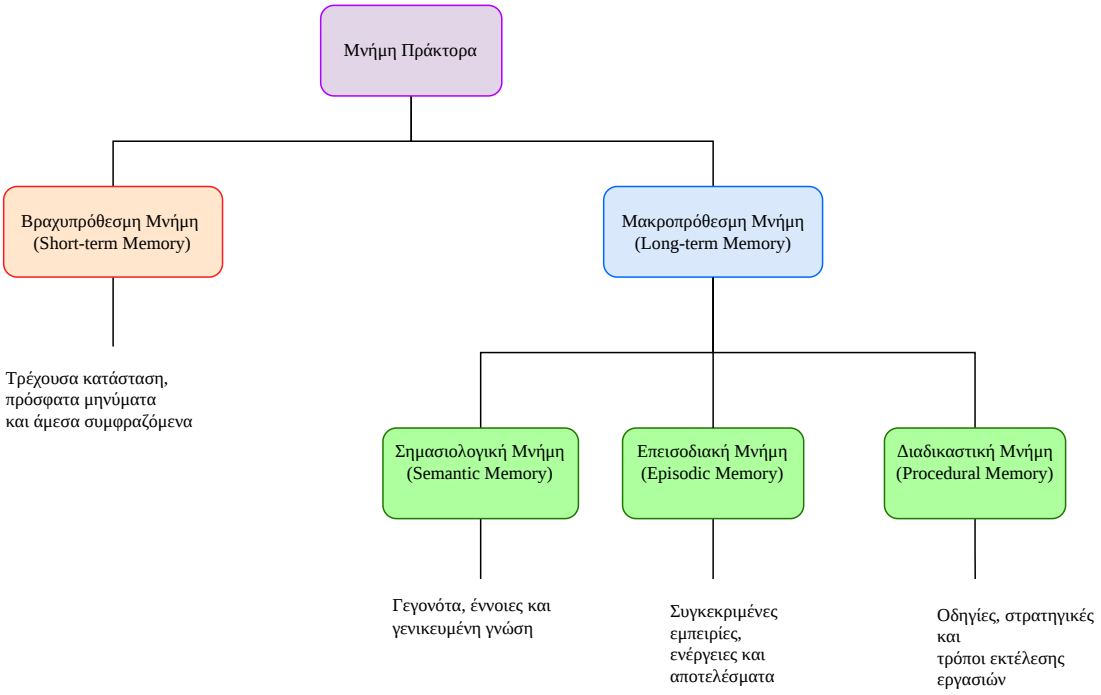
Μια άλλη χρήση της μνήμης αφορά την επεξεργασία κειμένων που υπερβαίνουν το διαθέσιμο πλαίσιο συμφραζομένων του γλωσσικού μοντέλου. Ενδεικτικά, το MemAgent επεξεργάζεται το κείμενο σε διαδοχικά τμήματα, διατηρώντας μια μνήμη περιορισμένου μεγέθους με πληροφορίες χρήσιμες για την απάντηση στο ερώτημα. Σε κάθε βήμα, το επόμενο τμήμα επεξεργάζεται μαζί με την προηγούμενη μνήμη, η οποία ενημερώνεται με βάση τις νέες πληροφορίες [10].

Πέρα από την αποθήκευση της πληροφορίας, σύγχρονες προσεγγίσεις αντιμετωπίζουν τη μνήμη ως ένα δυναμικό υποσύστημα του πράκτορα. Η πληροφορία μπορεί να ενοποιείται με προηγούμενη γνώση, να ενημερώνεται όταν προκύπτουν νεότερα δεδομένα, να οργανώνεται σε κατάλληλες δομές για αποτελεσματικότερη αναζήτηση, να ανακτάται με βάση το εκάστοτε πλαίσιο και, όταν παύει να είναι χρήσιμη, να συμπιέζεται ή να απομακρύνεται. Υπό αυτή την οπτική, η ποιότητα ενός συστήματος μνήμης εξαρτάται όχι μόνο από το περιεχόμενο που διατηρεί, αλλά και από τις λειτουργίες που καθορίζουν τον κύκλο ζωής και την αξιοποίηση της αποθηκευμένης πληροφορίας [2].

Η ύπαρξη αυτών των μηχανισμών επιτρέπει σε έναν πράκτορα να διατηρεί συνέχεια μεταξύ διαδοχικών αλληλεπιδράσεων και να αξιοποιεί πληροφορίες που αποκτήθηκαν σε προηγούμενα στάδια της λειτουργίας του. Ανάλογα με τη φύση της αποθηκευμένης πληροφορίας και τον τρόπο με τον οποίο αυτή χρησιμοποιείται, έχουν προταθεί διαφορετικές κατηγορίες μνήμης. Οι κυριότερες από αυτές παρουσιάζονται στην επόμενη ενότητα, με ιδιαίτερη έμφαση στην επεισοδιακή μνήμη, η οποία βρίσκεται στον πυρήνα της παρούσας εργασίας.

2.2 Κατηγορίες Μνήμης

Η μνήμη σε πράκτορες που βασίζονται σε Μεγάλα Γλωσσικά Μοντέλα μπορεί να ταξινομηθεί με διαφορετικούς τρόπους, ανάλογα με το χαρακτηριστικό που εξετάζεται. Μία βασική διάκριση αφορά τη διάρκεια και το εύρος διατήρησης της πληροφορίας και διαχωρίζει τη βραχυπρόθεσμη μνήμη (short-term memory) από τη μακροπρόθεσμη μνήμη (long-term memory). Αντίστοιχα, η μακροπρόθεσμη μνήμη μπορεί να διακριθεί με βάση το είδος της πληροφορίας που αποθηκεύεται σε σημασιολογική (semantic memory), επεισοδιακή (episodic memory) και διαδικαστική μνήμη (procedural memory) [2, 11, 12]. Οι κατηγορίες αυτές δεν είναι απαραίτητα αμοιβαία αποκλειόμενες, αλλά αποτελούν διαφορετικές όψεις της μνήμης που μπορούν να συνυπάρχουν στο ίδιο σύστημα. Η βασική αυτή ταξινόμηση συνοψίζεται στο Σχήμα 2.1.



Σχήμα 2.1: Βασικές κατηγορίες μνήμης σε πράκτορες Τεχνητής Νοημοσύνης, βάσει των [2, 11, 12].

Η βραχυπρόθεσμη μνήμη αφορά κυρίως πληροφορίες που είναι άμεσα διαθέσιμες κατά την εκτέλεση μιας τρέχουσας εργασίας ή στο πλαίσιο μιας συγκεκριμένης αλληλεπίδρασης. Σε έναν συνομιλιακό πράκτορα, για παράδειγμα, μπορεί να περιλαμβάνει τα πρόσφατα μηνύματα μιας συνομιλίας, ενδιάμεσα αποτελέσματα και άλλες πληροφορίες κατάστασης που είναι απαραίτητες για τη διατήρηση της συνοχής μεταξύ διαδοχικών βημάτων. Η πληροφορία αυτή είναι άμεσα προσβάσιμη από τον πράκτορα, όμως το μέγεθός της περιορίζεται από το διαθέσιμο πλαίσιο συμφραζομένων και από την ανάγκη να διατηρείται σχετικό και χρήσιμο το περιεχόμενο που παρέχεται στο γλωσσικό μοντέλο [11, 1, 12].

Αντίθετα, η μακροπρόθεσμη μνήμη αποσκοπεί στη διατήρηση πληροφορίας πέρα από τα όρια μιας μεμονωμένης αλληλεπίδρασης ή συνεδρίας. Η πληροφορία αυτή συνήθως αποθηκεύεται σε εξωτερικές δομές και ανακτάται επιλεκτικά όταν κρίνεται σχετική με μία μεταγενέστερη εργασία. Με τον τρόπο αυτό, ο πράκτορας δεν χρειάζεται να διατηρεί το σύνολο της προηγούμενης εμπειρίας του μέσα στο ενεργό πλαίσιο συμφραζομένων, αλλά μπορεί να προσπελαύνει συγκεκριμένα τμήματα προηγούμενης πληροφορίας ανάλογα με τις ανάγκες της εκάστοτε αλληλεπίδρασης [1, 11, 13]. Ο σχεδιασμός ενός τέτοιου μηχανισμού δημιουργεί επομένως δύο διακριτά προβλήματα: αφενός την επιλογή και οργάνωση των πληροφοριών που πρέπει να διατηρούνται και αφετέρου την αποτελεσματική ανάκτηση των κατάλληλων αναμνήσεων όταν αυτές απαιτούνται.

Ως προς το περιεχόμενο της μακροπρόθεσμης μνήμης, η σημασιολογική μνήμη αφορά την αποθήκευση γνώσεων, γεγονότων και εννοιών που δεν συνδέονται απαραί-

τητα με μία συγκεκριμένη χρονική εμπειρία. Σε ένα σύστημα πράκτορα μπορεί, για παράδειγμα, να περιλαμβάνει πληροφορίες που έχουν εξαχθεί από προηγούμενες αλληλεπιδράσεις και έχουν διατηρηθεί ως γενικευμένη γνώση. Αντίθετα, η επεισοδιακή μνήμη επικεντρώνεται σε συγκεκριμένα γεγονότα, εμπειρίες και ακολουθίες ενεργειών που έχουν πραγματοποιηθεί στο παρελθόν, διατηρώντας το πλαίσιο μέσα στο οποίο αυτές συνέβησαν [3]. Η διάκριση αυτή είναι σημαντική για πράκτορες που καλούνται όχι μόνο να θυμούνται τι γνωρίζουν, αλλά και να αξιοποιούν τον τρόπο με τον οποίο αντιμετώπισαν συγκεκριμένες καταστάσεις στο παρελθόν.

Η διαδικαστική μνήμη αφορά, αντίστοιχα, τη γνώση του τρόπου εκτέλεσης μιας εργασίας. Αντί να αποθηκεύει ένα συγκεκριμένο γεγονός ή μία μεμονωμένη εμπειρία, μπορεί να αναπαριστά επαναχρησιμοποιήσιμες οδηγίες, στρατηγικές ή διαδικασίες που καθοδηγούν τη συμπεριφορά του πράκτορα. Πρόσφατες προσεγγίσεις, όπως το Memp, διερευνούν τη δημιουργία διαδικαστικής μνήμης μέσω της μετατροπής προηγούμενων τροχιών εκτέλεσης (execution trajectories) σε λεπτομερείς οδηγίες ή πιο αφαιρετικά επαναχρησιμοποιήσιμα πρότυπα ενεργειών [14]. Με αυτόν τον τρόπο, η εμπειρία ενός πράκτορα μπορεί να μετασχηματίζεται σταδιακά από καταγραφή μεμονωμένων περιστατικών σε πιο γενικευμένη γνώση σχετικά με τον τρόπο επίλυσης μιας κατηγορίας προβλημάτων.

Στην πράξη, σύγχρονα συστήματα μνήμης συχνά συνδυάζουν περισσότερες από μία από τις παραπάνω κατηγορίες. Το MIRIX, για παράδειγμα, οργανώνει τη μνήμη σε έξι διακριτά συστατικά, μεταξύ των οποίων επεισοδιακή, σημασιολογική και διαδικαστική μνήμη, μαζί με επιπλέον δομές για διαφορετικούς τύπους πληροφορίας [15]. Η συνύπαρξη διαφορετικών μορφών μνήμης αναδεικνύει ότι ένα σύστημα μακροπρόθεσμης μνήμης δεν χρειάζεται να περιορίζεται σε έναν μόνο τύπο αποθηκευμένης πληροφορίας, αλλά μπορεί να οργανώνει την προηγούμενη εμπειρία με διαφορετικούς τρόπους ανάλογα με τον σκοπό για τον οποίο πρόκειται να αξιοποιηθεί.

Η παρούσα εργασία εστιάζει στην επεισοδιακή μνήμη, καθώς το ζητούμενο δεν είναι μόνο η διατήρηση γενικευμένης γνώσης, αλλά η επαναχρησιμοποίηση συγκεκριμένων προηγούμενων εκτελέσεων του πράκτορα. Μία τέτοια εκτέλεση μπορεί να θεωρηθεί ως επεισόδιο που περιλαμβάνει το αρχικό ερώτημα, τις ενδιάμεσες προσπάθειες, τα σφάλματα ή τις διορθώσεις που προέκυψαν και την τελική έκβαση. Η επιλεκτική ανάκτηση συναφών επεισοδίων επιτρέπει σε μία νέα εκτέλεση να αξιοποιεί τόσο προηγούμενες επιτυχείς λύσεις όσο και πληροφορία από αποτυχημένες προσπάθειες, χωρίς οι εμπειρίες αυτές να χρειάζεται να μετατραπούν εξαρχής σε γενικούς κανόνες. Για τον λόγο αυτό, η επεισοδιακή μνήμη αποτελεί την πιο άμεση θεωρητική βάση για τον μηχανισμό μνήμης που εξετάζεται στην παρούσα εργασία και αναλύεται χωριστά στην επόμενη ενότητα.

2.3 Επεισοδιακή Μνήμη

Η επεισοδιακή μνήμη (episodic memory) αναφέρεται στη διατήρηση συγκεκριμένων εμπειριών μαζί με το πλαίσιο μέσα στο οποίο αυτές πραγματοποιήθηκαν. Σε αντίθεση με τη σημασιολογική μνήμη, η οποία αποτυπώνει πιο γενικευμένη γνώση, η επεισοδιακή μνήμη διατηρεί πληροφορίες που συνδέονται με μεμονωμένα περιστατικά, τις ενέργειες που πραγματοποιήθηκαν και τα αποτελέσματά τους. Η δυνατότητα αυτή είναι πολύτιμη για πράκτορες που λειτουργούν σε βάθος χρόνου, καθώς επιτρέπει την αξιοποίηση προηγούμενων εμπειριών κατά την αντιμετώπιση νέων καταστάσεων [3].

Ένα επεισόδιο δεν περιορίζεται απαραίτητα σε μία απλή καταγραφή εισόδου και εξόδου. Μπορεί να περιλαμβάνει την αρχική κατάσταση, την είσοδο που έλαβε ο πράκτορας, τις ενέργειες που πραγματοποίησε, τα ενδιάμεσα αποτελέσματα ή σφάλματα που προέκυψαν, καθώς και την τελική έκβαση μιας εργασίας. Σε επαναληπτικές διαδικασίες επίλυσης προβλημάτων, η ακολουθία αυτών των ενεργειών και αποτελεσμάτων μπορεί να αναπαρασταθεί ως τροχιά εκτέλεσης (execution trajectory). Η αποθήκευση μιας τέτοιας τροχιάς διατηρεί όχι μόνο το τελικό αποτέλεσμα, αλλά και την πορεία μέσω της οποίας αυτό επιτεύχθηκε [3, 14].

Κεντρική λειτουργία της επεισοδιακής μνήμης αποτελεί η ανάκτηση (retrieval) προηγούμενων εμπειριών που σχετίζονται με την τρέχουσα κατάσταση. Αντί να παρέχεται κάθε φορά ολόκληρο το ιστορικό λειτουργίας του πράκτορα, μπορούν να επιλέγονται συγκεκριμένα επεισόδια που εμφανίζουν μεγαλύτερη συνάφεια με το τρέχον πρόβλημα. Οι ανακτημένες εμπειρίες ενσωματώνονται στη διαθέσιμη πληροφορία του πράκτορα και μπορούν να επηρεάσουν την επιλογή της επόμενης ενέργειας. Με τον τρόπο αυτό, η εμπειρία που αποκτάται κατά τη λειτουργία του συστήματος μπορεί να αξιοποιείται σε μελλοντικές αλληλεπιδράσεις χωρίς να απαιτείται μεταβολή των παραμέτρων του ίδιου του γλωσσικού μοντέλου [3].

Η χρησιμότητα της συγκεκριμένης μορφής μνήμης γίνεται πιο εμφανής όταν δύο προβλήματα δεν είναι ταυτόσημα αλλά παρουσιάζουν κοινή δομή ή απαιτούν παρόμοια στρατηγική επίλυσης. Η ανάκτηση ενός συναφούς προηγούμενου επεισοδίου μπορεί τότε να λειτουργήσει ως παράδειγμα του τρόπου με τον οποίο αντιμετωπίστηκε μια ανάλογη κατάσταση. Ιδιαίτερη αξία μπορεί να έχει όχι μόνο η προηγούμενη επιτυχής λύση, αλλά και η πορεία που προηγήθηκε αυτής. Αποτυχημένες ενέργειες, παρατηρούμενα σφάλματα και μεταγενέστερες διορθώσεις μπορούν να παρέχουν χρήσιμη πληροφορία για την αποφυγή αντίστοιχων επιλογών στο μέλλον. Υπό αυτή την έννοια, η επεισοδιακή μνήμη υποστηρίζει μία μορφή μάθησης από την εμπειρία, όπου η συμπεριφορά του πράκτορα σε μία νέα εργασία επηρεάζεται από προηγούμενες συναφείς περιπτώσεις [3, 14].

Η συγκεκριμένη αντίληψη της μνήμης δεν προϋποθέτει ότι οι αποθηκευμένες εμπειρίες μετασχηματίζονται εξαρχής σε γενικούς κανόνες. Μία τροχιά παραμένει επεισο-

διακή όσο διατηρεί το πλαίσιο της αρχικής περίπτωσης και ανακτάται ως συγκεκριμένη προηγούμενη εμπειρία. Αντίθετα, όταν πληροφορία από πολλαπλές τροχιές μετατραπεί σε γενικευμένες οδηγίες ή στρατηγικές ανεξάρτητες από τα περιστατικά από τα οποία προέκυψε, η αναπαράσταση αποκτά πιο διαδικαστικό χαρακτήρα [14]. Επεισοδιακή και διαδικαστική μνήμη μπορούν επομένως να λειτουργούν συμπληρωματικά: η πρώτη διατηρεί συγκεκριμένες εμπειρίες, ενώ η δεύτερη μπορεί να κωδικοποιεί γενικότερες μεθόδους δράσης που προκύπτουν από την επεξεργασία τους [16].

Στην κατεύθυνση της επαναχρησιμοποίησης στρατηγικών κινείται το Dynamic Cheatsheet, το οποίο διατηρεί μια εξελισσόμενη μνήμη με στρατηγικές επίλυσης και χρήσιμα αποσπάσματα από προηγούμενες προσπάθειες. Η πληροφορία αυτή αξιοποιείται κατά την αντιμετώπιση νέων εργασιών, επιτρέποντας στο μοντέλο να επωφελείται από τη συσσωρευμένη εμπειρία χωρίς μεταβολή των παραμέτρων του [17].

Η δυνατότητα επαναχρησιμοποίησης προηγούμενης εμπειρίας δημιουργεί και την ανάγκη αξιολόγησης του τρόπου με τον οποίο ο πράκτορας ανακτά και αξιοποιεί την αποθηκευμένη πληροφορία. Το LongMemEval εξετάζει, μεταξύ άλλων, την ανάκληση πληροφοριών από διαφορετικές συνεδρίες, τον χρονικό συλλογισμό και την αξιοποίηση ενημερωμένης γνώσης [18]. Το LongMemEval-V2 επεκτείνει την αξιολόγηση σε εμπειρίες από περιβάλλοντα πρακτόρων, εξετάζοντας τη γνώση ροών εργασίας, μεταβολών της κατάστασης και επαναλαμβανόμενων προβλημάτων [19]. Οι διαφορετικές αυτές απαιτήσεις αναδεικνύουν ότι η χρησιμότητα της μνήμης εξαρτάται τόσο από την ακριβή ανάκληση προηγούμενων πληροφοριών όσο και από τη δυνατότητα αξιοποίησης της εμπειρίας στο πλαίσιο μιας νέας εργασίας.

Η αποτελεσματικότητα της επεισοδιακής μνήμης εξαρτάται τελικά από τον τρόπο με τον οποίο οι εμπειρίες αποθηκεύονται, οργανώνονται και ανακτώνται. Η απλή συσσώρευση προηγούμενων αλληλεπιδράσεων δεν επαρκεί, καθώς ένα συνεχώς αυξανόμενο αποθετήριο μνήμης πρέπει να παρέχει μηχανισμούς για τον εντοπισμό των κατάλληλων πληροφοριών την κατάλληλη χρονική στιγμή. Για τον λόγο αυτό, η αναπαράσταση της μνήμης και οι λειτουργίες που εφαρμόζονται πάνω σε αυτή αποτελούν κεντρικό ζήτημα στον σχεδιασμό συστημάτων μακροπρόθεσμης μνήμης.

2.4 Βασικές Λειτουργίες Συστημάτων Μνήμης

Ένα σύστημα μακροπρόθεσμης μνήμης δεν καθορίζεται μόνο από το περιεχόμενο που διατηρεί, αλλά και από τις λειτουργίες μέσω των οποίων η πληροφορία εισάγεται, οργανώνεται, ενημερώνεται και ανακτάται κατά τη διάρκεια του κύκλου ζωής της. Η σχετική βιβλιογραφία αντιμετωπίζει τη μνήμη ως ένα δυναμικό σύστημα, στο οποίο η αποθηκευμένη εμπειρία εξελίσσεται και αξιοποιείται επιλεκτικά ανάλογα με τις ανάγκες του πράκτορα [2].

Η πρώτη βασική λειτουργία αφορά την εγγραφή ή αποθήκευση (memory writing)

νέας πληροφορίας. Συνήθως δεν είναι επιθυμητή η αδιάκριτη αποθήκευση κάθε στοιχείου που εμφανίζεται κατά τη λειτουργία ενός πράκτορα, καθώς αυτό οδηγεί σε ταχεία αύξηση του μεγέθους της μνήμης και σε συσσώρευση περιττής ή επαναλαμβανόμενης πληροφορίας. Ένας μηχανισμός εγγραφής μπορεί επομένως να επιλέγει ποια στοιχεία είναι αρκετά σημαντικά ώστε να διατηρηθούν και να καθορίζει τη μορφή με την οποία θα αποθηκευτούν [20]. Ανάλογα με τον σκοπό του συστήματος, η μνήμη μπορεί να περιέχει ακατέργαστο κείμενο, συνοπτικές περιγραφές, δομημένες εγγραφές, προηγούμενες ενέργειες και αποτελέσματα ή συνδυασμούς αυτών.

Η εγγραφή της μνήμης μπορεί επίσης να πραγματοποιείται σε διαφορετικά χρονικά σημεία της λειτουργίας του συστήματος. Μία δυνατότητα είναι η άμεση εγγραφή κατά την εκτέλεση (hot-path memory writing), ώστε η νέα πληροφορία να είναι αμέσως διαθέσιμη σε επόμενες ενέργειες. Εναλλακτικά, η επεξεργασία και αποθήκευση μπορούν να πραγματοποιούνται στο παρασκήνιο (background memory writing), περιορίζοντας την επιβάρυνση της κύριας ροής λειτουργίας, αλλά καθυστερώντας τη διαθεσιμότητα της νέας μνήμης [12].

Ο διαχωρισμός της άμεσης αποθήκευσης από τη μεταγενέστερη επεξεργασία της μνήμης εφαρμόζεται στο LightMem. Οι νέες εγγραφές εισάγονται αρχικά στη μακροπρόθεσμη μνήμη μαζί με χρονικές σημάνσεις. Η αναδιοργάνωση και η ενοποίησή τους πραγματοποιούνται σε μεταγενέστερο στάδιο, μέσω μηχανισμού sleep-time update, ο οποίος αξιοποιεί τη σημασιολογική ομοιότητα και τη χρονική σειρά των εγγραφών. Με αυτόν τον τρόπο, η πιο απαιτητική συντήρηση της μνήμης αποσυνδέεται από την άμεση παραγωγή της απόκρισης, περιορίζοντας την καθυστέρηση της αλληλεπίδρασης [21].

Μετά την αποθήκευση ακολουθεί η οργάνωση και ευρετηρίαση (indexing) της πληροφορίας. Σκοπός της ευρετηρίασης είναι να καταστήσει δυνατή την αποδοτική αναζήτηση σε ένα αποθετήριο του οποίου το μέγεθος αυξάνεται με την πάροδο του χρόνου. Διαφορετικές αναπαραστάσεις οδηγούν και σε διαφορετικούς μηχανισμούς πρόσβασης. Η αναζήτηση μπορεί, για παράδειγμα, να στηρίζεται σε αντιστοίχιση λέξεων ή όρων, σε σημασιολογική ομοιότητα μεταξύ διανυσμάτων ή στην αξιοποίηση ρητών σχέσεων μιας δομημένης αναπαράστασης. Σύνθετα συστήματα μπορούν επίσης να συνδυάζουν περισσότερους μηχανισμούς αναζήτησης ώστε να αξιοποιούν τόσο τη σημασιολογική συνάφεια όσο και ακριβέστερα λεξιλογικά ή δομικά χαρακτηριστικά [2].

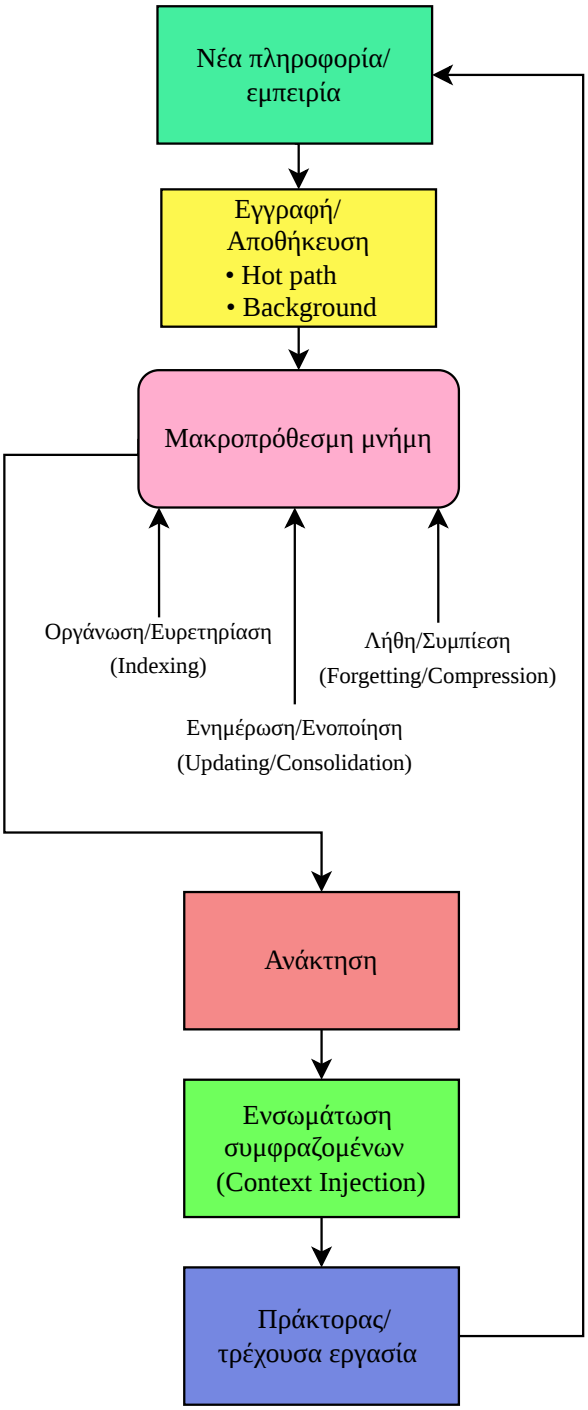
Η ανάκτηση αποτελεί τη λειτουργία μέσω της οποίας επιλέγεται από τη μακροπρόθεσμη μνήμη ένα περιορισμένο σύνολο πληροφοριών που θεωρείται χρήσιμο για την τρέχουσα εργασία. Η ποιότητα της ανάκτησης είναι κρίσιμη, καθώς τόσο η απουσία μιας σημαντικής προηγούμενης εμπειρίας όσο και η εισαγωγή άσχετης πληροφορίας μπορούν να επηρεάσουν αρνητικά τη συμπεριφορά του πράκτορα. Η συνάφεια μιας αποθηκευμένης εμπειρίας μπορεί να εκτιμηθεί με διαφορετικά κριτήρια, όπως η σημασιολογική ομοιότητα, η χρονική εγγύτητα, η δομική συνάφεια ή η σημασία που έχει αποδοθεί στην πληροφορία. Στην πράξη, οι μηχανισμοί αυτοί μπορούν να χρησιμο-

ποιούνται μεμονωμένα ή συνδυαστικά, ανάλογα με το είδος της αποθηκευμένης γνώσης και τον σκοπό του συστήματος [2].

Η ανακτημένη πληροφορία πρέπει στη συνέχεια να καταστεί διαθέσιμη στο μοντέλο που πρόκειται να εκτελέσει την επόμενη ενέργεια. Η διαδικασία αυτή μπορεί να θεωρηθεί ως ενσωμάτωση συμφραζομένων (context injection), κατά την οποία τα επιλεγμένα στοιχεία μνήμης ενσωματώνονται στην είσοδο του μοντέλου. Επειδή ο διαθέσιμος χώρος συμφραζομένων παραμένει πεπερασμένος, η ανάκτηση δεν πρέπει να μεγιστοποιεί απλώς την ποσότητα της παρεχόμενης πληροφορίας. Αντίθετα, επιδιώκεται ένα μικρό και σχετικό σύνολο αναμνήσεων που μπορεί να προσφέρει χρήσιμη καθοδήγηση χωρίς να επιβαρύνει υπερβολικά το πλαίσιο εισόδου. Η ανάγκη αυτή συνδέει άμεσα την ποιότητα της αναζήτησης με την αποτελεσματική αξιοποίηση της μνήμης [2].

Εξίσου σημαντικές είναι οι λειτουργίες που μεταβάλλουν την αποθηκευμένη πληροφορία με την πάροδο του χρόνου. Η ενημέρωση (update) επιτρέπει την τροποποίηση μιας υπάρχουσας μνήμης όταν εμφανίζεται νεότερη ή ακριβέστερη πληροφορία, ενώ η ενοποίηση (consolidation) μπορεί να συνδυάζει σχετιζόμενες εμπειρίες σε πιο συμπαγείς αναπαραστάσεις [22]. Αντίστοιχα, η λήθη (forgetting) ή η συμπίεση (compression) περιορίζουν την άσκοπη διόγκωση της μνήμης αφαιρώντας, συγχωνεύοντας ή συνοψίζοντας πληροφορίες που έχουν πάψει να είναι χρήσιμες [2, 23]. Οι λειτουργίες αυτές είναι αναγκαίες σε συστήματα που λειτουργούν συνεχώς, όπου η απεριόριστη συσσώρευση εμπειριών θα οδηγούσε σταδιακά σε αυξημένο υπολογιστικό κόστος και δυσκολότερη ανάκτηση σχετικής πληροφορίας.

Οι βασικές λειτουργίες και η μεταξύ τους αλληλεπίδραση συνοψίζονται στο Σχήμα 2.2.



Σχήμα 2.2: Βασικές λειτουργίες και κύκλος ζωής ενός συστήματος μακροπρόθεσμης μνήμης, βάσει των [2, 12, 23].

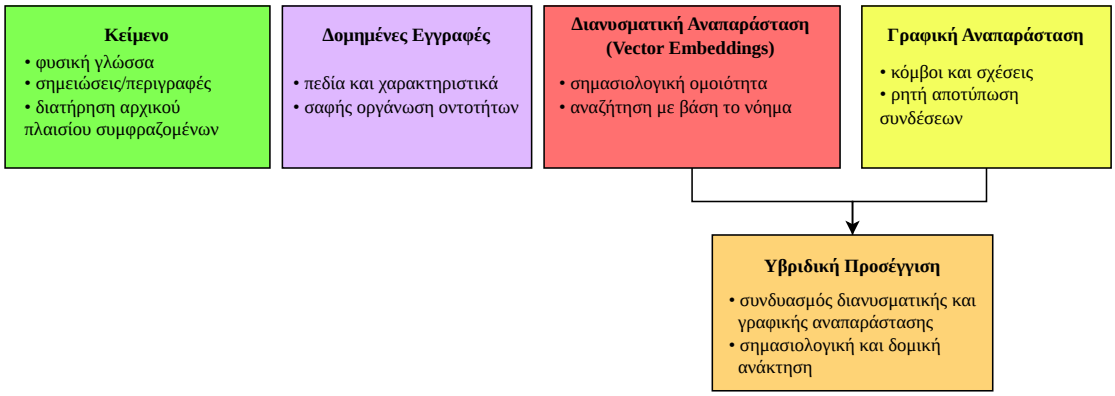
Όπως απεικονίζεται στο Σχήμα 2.2, οι παραπάνω λειτουργίες σχηματίζουν έναν επαναλαμβανόμενο κύκλο: μια νέα εμπειρία αξιολογείται και αποθηκεύεται, οργανώνεται, ανακτάται όταν εμφανιστεί μία συναφής ανάγκη και επηρεάζει τη νέα συμπεριφορά του πράκτορα. Το αποτέλεσμα της νέας αλληλεπίδρασης μπορεί με τη σειρά του να αποτελέσει νέα μνήμη, επιτρέποντας στο αποθετήριο να εξελίσσεται με την πάροδο του χρόνου. Η αποτελεσματικότητα των λειτουργιών αυτών εξαρτάται, ωστόσο, και από τη

μορφή με την οποία αναπαρίσταται η αποθηκευμένη πληροφορία. Οι βασικές επιλογές αναπαράστασης εξετάζονται στην επόμενη ενότητα.

2.5 Αναπαράσταση Μνήμης

Πέρα από τις λειτουργίες που καθορίζουν τον κύκλο ζωής της πληροφορίας, κρίσιμη σχεδιαστική επιλογή αποτελεί ο τρόπος με τον οποίο η ίδια η μνήμη αναπαρίσταται. Η επιλεγμένη αναπαράσταση επηρεάζει τόσο τον τρόπο οργάνωσης της αποθηκευμένης εμπειρίας όσο και τους μηχανισμούς μέσω των οποίων αυτή μπορεί να αναζητηθεί και να ανακτηθεί.

Όπως είναι γνωστό, το κείμενο μπορεί να διατηρεί μεγάλο μέρος του αρχικού πλαισίου μιας εμπειρίας, αλλά παρουσιάζει περιορισμούς όταν απαιτείται ρητή αποτύπωση των σχέσεων μεταξύ διαφορετικών πληροφοριών. Οι δομημένες εγγραφές επιτρέπουν σαφέστερο διαχωρισμό οντοτήτων και χαρακτηριστικών, ενώ οι διανυσματικές αναπαραστάσεις (vector embeddings) αντιστοιχίζουν το περιεχόμενο σε έναν διανυσματικό χώρο στον οποίο μπορούν να εντοπιστούν σημασιολογικά συναφείς πληροφορίες. Αντίστοιχα, οι γραφικές αναπαραστάσεις επιτρέπουν τη ρητή μοντελοποίηση των σχέσεων μεταξύ διαφορετικών στοιχείων της μνήμης. Οι μορφές αυτές δεν είναι αμοιβαία αποκλειόμενες και μπορούν να συνδυάζονται σε υβριδικές αρχιτεκτονικές [2]. Η βασική αυτή ταξινόμηση συνοψίζεται στο Σχήμα 2.3.



Σχήμα 2.3: Βασικές μορφές αναπαράστασης μνήμης και δυνατότητα συνδυασμού διανυσματικών και γραφικών προσεγγίσεων σε υβριδικά σχήματα ανάκτησης, βάσει των [2, 12].

Μεταξύ των παραπάνω επιλογών, η γραφική αναπαράσταση αποτελεί κατάλληλη επιλογή για συστήματα μακροπρόθεσμης μνήμης, καθώς επιτρέπει τη ρητή αποτύπωση όχι μόνο των αποθηκευμένων στοιχείων αλλά και των σχέσεων που τα συνδέουν. Η αξιοποίηση γράφων γνώσης για τον σκοπό αυτό εξετάζεται στην επόμενη ενότητα.

2.6 Γραφική Αναπαράσταση Μνήμης με Γράφους Γνώσης

Οι γράφοι γνώσης (Knowledge Graphs – KGs) αποτελούν δομημένες αναπαραστάσεις πληροφορίας στις οποίες οι οντότητες και οι μεταξύ τους συσχετίσεις μοντελοποιούνται ρητά. Σε αντίθεση με αναπαραστάσεις όπου οι επιμέρους πληροφορίες αποθηκεύονται ως ανεξάρτητες εγγραφές, ένας γράφος επιτρέπει την οργάνωσή τους ως ένα δίκτυο αλληλοσυνδεόμενων στοιχείων. Η δομή αυτή καθιστά δυνατή όχι μόνο την αποθήκευση γνώσης, αλλά και την αξιοποίηση των σχέσεων που συνδέουν διαφορετικές οντότητες κατά την αναζήτηση και επεξεργασία της πληροφορίας [5, 24].

Η ιδιότητα αυτή παρουσιάζει ιδιαίτερο ενδιαφέρον για συστήματα μακροπρόθεσμης μνήμης. Μία αποθηκευμένη εμπειρία μπορεί να περιλαμβάνει περισσότερα από ένα ανεξάρτητα στοιχεία, όπως την κατάσταση στην οποία βρισκόταν ένας πράκτορας, τις ενέργειες που πραγματοποίησε, τα αποτελέσματα που προέκυψαν και τις οντότητες που συμμετείχαν σε αυτά. Η γραφική αναπαράσταση επιτρέπει τα στοιχεία αυτά να αποθηκεύονται ως διακριτές αλλά συνδεδεμένες μονάδες, διατηρώντας έτσι πληροφορία τόσο για το περιεχόμενό τους όσο και για τον τρόπο με τον οποίο σχετίζονται μεταξύ τους [6, 12].

2.6.1 Βασικές Έννοιες και Μοντέλο Γράφου Ιδιοτήτων

Ένας γράφος αποτελείται από ένα σύνολο κόμβων (nodes) και ακμών (edges). Οι κόμβοι αναπαριστούν συνήθως αντικείμενα, έννοιες ή γεγονότα, ενώ οι ακμές εκφράζουν τις σχέσεις που συνδέουν δύο κόμβους. Σε έναν γράφο γνώσης, η πληροφορία αποκτά επομένως τόσο οντολογική όσο και σχεσιακή διάσταση: δεν καταγράφεται μόνο η ύπαρξη μιας οντότητας, αλλά και ο τρόπος με τον οποίο αυτή συνδέεται με άλλες οντότητες του ίδιου γνωσιακού χώρου [5].

Μία ευρέως χρησιμοποιούμενη μορφή γραφικής αναπαράστασης είναι το μοντέλο γράφου ιδιοτήτων (property graph model). Στο μοντέλο αυτό, τόσο οι κόμβοι όσο και οι σχέσεις μπορούν να συνοδεύονται από επιπλέον πληροφορία υπό τη μορφή ιδιοτήτων (properties), οι οποίες συνήθως εκφράζονται ως ζεύγη κλειδιού–τιμής. Οι κόμβοι μπορούν επιπλέον να χαρακτηρίζονται από μία ή περισσότερες ετικέτες (labels), οι οποίες δηλώνουν τον τύπο ή τον ρόλο της οντότητας που αναπαριστούν, ενώ οι σχέσεις είναι συνήθως κατευθυνόμενες και διαθέτουν συγκεκριμένο τύπο [25].

Για παράδειγμα, σε έναν γράφο που αναπαριστά πληροφορίες σχετικά με κινηματογραφικές ταινίες, ένας κόμβος με ετικέτα Movie μπορεί να διαθέτει ιδιότητες όπως ο τίτλος και το έτος κυκλοφορίας, ενώ ένας δεύτερος κόμβος με ετικέτα Person μπορεί να αναπαριστά έναν ηθοποιό. Μία κατευθυνόμενη σχέση τύπου ACTED_IN μεταξύ των δύο κόμβων αποτυπώνει ρητά το γεγονός ότι το συγκεκριμένο πρόσωπο συμμετείχε στην

αντίστοιχη ταινία. Με τον τρόπο αυτό, πληροφορία που σε μία σχεσιακή ή μη δομημένη αναπαράσταση θα απαιτούσε συνδυασμό πολλαπλών εγγραφών, αποτυπώνεται άμεσα στη δομή του γράφου.

Το σύνολο των επιτρεπόμενων τύπων κόμβων, σχέσεων και ιδιοτήτων καθορίζει το σχήμα (schema) του γράφου. Η γνώση του σχήματος είναι αναγκαία κατά την υποβολή δομημένων ερωτημάτων, καθώς ένα ερώτημα πρέπει να χρησιμοποιεί οντότητες, ιδιότητες και διαδρομές που είναι συμβατές με τη δομή της υποκείμενης βάσης [7, 8].

Το μοντέλο γράφου ιδιοτήτων χρησιμοποιείται από συστήματα διαχείρισης βάσεων γράφων όπως το Neo4j, στα οποία η προσπέλαση της πληροφορίας πραγματοποιείται μέσω μονοπατιών και σχέσεων μεταξύ των αποθηκευμένων οντοτήτων. Η συγκεκριμένη αναπαράσταση είναι κατάλληλη για περιπτώσεις στις οποίες η σχεσιακή δομή της πληροφορίας αποτελεί ουσιώδες μέρος της σημασίας της και όχι απλώς έναν μηχανισμό αποθήκευσης [25, 8].

2.6.2 Γράφοι Γνώσης ως Δομές Μνήμης

Η χρήση γράφων για την αναπαράσταση μνήμης επιτρέπει τη μετάβαση από ένα σύνολο ανεξάρτητων καταγραφών σε μία συνδεδεμένη δομή εμπειριών και γνώσης. Αντί κάθε αποθηκευμένο στοιχείο να αποτελεί μία απομονωμένη εγγραφή, μπορεί να συνδέεται ρητά με τις οντότητες, τα γεγονότα ή τις προηγούμενες εμπειρίες με τις οποίες σχετίζεται. Με αυτόν τον τρόπο διατηρείται όχι μόνο η πληροφορία που περιέχει κάθε μνήμη, αλλά και το πλαίσιο των σχέσεών της με το υπόλοιπο αποθετήριο.

Η ιδιότητα αυτή είναι αρκετά χρήσιμη για την επεισοδιακή μνήμη. Ένα επεισόδιο μπορεί να αναλυθεί σε επιμέρους στοιχεία και να συνδεθεί με τις ενέργειες, τα αποτελέσματα και τις οντότητες που συμμετείχαν σε αυτό. Αντίστοιχα, μια ακολουθία γεγονότων μπορεί να αποτυπωθεί μέσω σχέσεων που διατηρούν τη μεταξύ τους σειρά ή άλλη μορφή εξάρτησης. Η αναπαράσταση αυτή καθιστά δυνατή την αναζήτηση όχι μόνο με βάση το περιεχόμενο μιας συγκεκριμένης μνήμης, αλλά και με βάση τη θέση της μέσα σε μια ευρύτερη δομή συσχετισμένων εμπειριών [6].

Χαρακτηριστικό παράδειγμα αποτελεί το AriGraph, το οποίο συνδυάζει έναν σημασιολογικό γράφο γνώσης με επεισοδιακή μνήμη. Οι εμπειρίες του πράκτορα χρησιμοποιούνται για την ενημέρωση μιας γραφικής αναπαράστασης του περιβάλλοντος, ενώ η ανάκτηση μπορεί να αξιοποιήσει τόσο προηγούμενα επεισόδια όσο και τη σημασιολογική γνώση που έχει προκύψει από αυτά [6, 12]. Η προσέγγιση αυτή αναδεικνύει τη δυνατότητα των γράφων να λειτουργούν ως δυναμικές δομές μνήμης που εξελίσσονται καθώς ο πράκτορας αποκτά νέες εμπειρίες.

Επιπλέον, η γραφική αναπαράσταση μπορεί να υποστηρίξει τη διατήρηση χρονικής πληροφορίας. Συστήματα όπως το Zep μοντελοποιούν τη μακροπρόθεσμη μνήμη μέσω χρονικού γράφου γνώσης (temporal knowledge graph), ώστε οι αποθηκευμένες πληρο-

φορίες να συνδέονται όχι μόνο μέσω σημασιολογικών σχέσεων αλλά και με τη χρονική περίοδο κατά την οποία θεωρούνται έγκυρες [26]. Η δυνατότητα αυτή είναι σημαντική σε περιβάλλοντα όπου η γνώση μεταβάλλεται και μία νεότερη πληροφορία μπορεί να συμπληρώνει ή να αναιρεί μία προηγούμενη.

Ένα επιπλέον πλεονέκτημα της γραφικής αναπαράστασης είναι η δυνατότητα δομικής ανάκτησης. Εφόσον οι σχέσεις αποθηκεύονται ρητά, η αναζήτηση μπορεί να ακολουθήσει συγκεκριμένα μονοπάτια, γειτονικούς κόμβους ή υπογράφους, αντί να στηρίζεται αποκλειστικά στην ομοιότητα του περιεχομένου δύο εγγραφών. Έτσι, μία αρχικά ανακτημένη οντότητα μπορεί να οδηγήσει σε πρόσθετη σχετική πληροφορία μέσω των σχέσεών της, επιτρέποντας την κατασκευή πλουσιότερου πλαισίου για την τρέχουσα εργασία [12, 6].

Η χρήση γράφων δεν εξαλείφει, ωστόσο, την ανάγκη για άλλους μηχανισμούς αναπαράστασης και αναζήτησης. Σε ένα μεγάλο αποθετήριο, η επιλογή του κατάλληλου αρχικού σημείου του γράφου μπορεί να είναι δύσκολη όταν η τρέχουσα είσοδος εκφράζεται σε φυσική γλώσσα ή δεν αντιστοιχεί ακριβώς σε κάποια αποθηκευμένη οντότητα. Για τον λόγο αυτό, οι γραφικές δομές συνδυάζονται συχνά με διανυσματικές αναπαραστάσεις, ώστε η σημασιολογική αναζήτηση να λειτουργεί συμπληρωματικά προς τη δομική πληροφορία [23, 12].

2.6.3 Διανυσματική, Γραφική και Υβριδική Αναπαράσταση Μνήμης

Η διανυσματική αναπαράσταση αποτελεί έναν από τους συνηθέστερους μηχανισμούς υποστήριξης σημασιολογικής αναζήτησης. Μέσω ενός μοντέλου ενσωματώσεων (embedding model), ένα τμήμα κειμένου, μία ερώτηση ή μία αποθηκευμένη εμπειρία αντιστοιχίζεται σε ένα διάνυσμα πραγματικών αριθμών. Αντικείμενα με παρόμοιο σημασιολογικό περιεχόμενο αναμένεται να βρίσκονται σε κοντινές περιοχές του διανυσματικού χώρου, επιτρέποντας την ανάκτηση σχετικών εγγραφών μέσω κατάλληλων μετρικών ομοιότητας [6, 12, 27].

Το βασικό πλεονέκτημα της προσέγγισης αυτής είναι ότι δεν απαιτεί ακριβή λεξιλογική αντιστοίχιση μεταξύ της τρέχουσας εισόδου και της αποθηκευμένης μνήμης. Δύο διαφορετικά διατυπωμένες περιπτώσεις μπορούν να θεωρηθούν συναφείς εφόσον οι διανυσματικές τους αναπαραστάσεις βρίσκονται αρκετά κοντά. Αυτό καθιστά τη διανυσματική αναζήτηση κατάλληλη για την ανάκτηση προηγούμενων εμπειριών με παρόμοιο σημασιολογικό περιεχόμενο. Σύγχρονα μοντέλα, όπως η οικογένεια Qwen3 Embedding, παρέχουν αναπαραστάσεις για ανάκτηση κειμένων, καθώς και μοντέλα επανακατάταξης των υποψήφιων αποτελεσμάτων [28].

Ωστόσο, μία διανυσματική αναπαράσταση δεν αποτυπώνει κατ' ανάγκη ρητά τη δομή των σχέσεων που συνδέουν τις επιμέρους πληροφορίες. Η υψηλή σημασιολογική

ομοιότητα δύο εγγραφών δεν συνεπάγεται ότι διαθέτουν την ίδια σχεσιακή ή τοπολογική δομή, ενώ πληροφορίες που συνδέονται σημαντικά μέσω μιας ακολουθίας σχέσεων δεν είναι απαραίτητο να παρουσιάζουν υψηλή διανυσματική ομοιότητα. Η παρατήρηση αυτή δημιουργεί ένα φυσικό κίνητρο για τον συνδυασμό σημασιολογικής και γραφικής αναζήτησης.

Στη γραφική προσέγγιση, η ανάκτηση αξιοποιεί τη ρητά αποθηκευμένη δομή. Η πληροφορία μπορεί να εντοπιστεί μέσω συγκεκριμένων τύπων σχέσεων, γειτονικών κόμβων ή μονοπατιών, επιτρέποντας την αξιοποίηση του πλαισίου που προκύπτει από τη σύνδεση διαφορετικών στοιχείων. Η αναζήτηση αυτή είναι κατάλληλη όταν η συνάφεια μιας αποθηκευμένης εμπειρίας εξαρτάται όχι μόνο από το περιεχόμενό της, αλλά και από τον ρόλο ή τη θέση της μέσα στη συνολική δομή γνώσης.

Οι δύο προσεγγίσεις μπορούν να συνδυαστούν σε μία υβριδική αναπαράσταση και ανάκτηση (hybrid retrieval). Σε ένα τέτοιο σύστημα, η διανυσματική αναζήτηση μπορεί αρχικά να εντοπίσει σημασιολογικά συναφείς κόμβους ή μνήμες, οι οποίοι στη συνέχεια χρησιμοποιούνται ως σημεία εκκίνησης για την εξερεύνηση των μεταξύ τους σχέσεων στον γράφο. Αντίστροφα, η δομή του γράφου μπορεί να περιορίσει τον χώρο αναζήτησης και η διανυσματική ομοιότητα να χρησιμοποιηθεί για την κατάταξη των υποψήφιων αποτελεσμάτων. Με τον τρόπο αυτό συνδυάζεται η ευελιξία της σημασιολογικής αναζήτησης με την πιο ρητή σχεσιακή πληροφορία που παρέχει ένας γράφος γνώσης [23, 12].

Συναφής προσέγγιση ακολουθείται στο HippoRAG 2, το οποίο συνδυάζει γραφική οργάνωση της γνώσης με αναπαραστάσεις κειμένων και μηχανισμούς συσχετιστικής ανάκτησης, υποστηρίζοντας τη διαρκή ενσωμάτωση νέας γνώσης χωρίς μεταβολή των παραμέτρων του γλωσσικού μοντέλου [29].

Η επιλογή μεταξύ διανυσματικής, γραφικής ή υβριδικής προσέγγισης εξαρτάται τελικά από το είδος της μνήμης και τις απαιτήσεις της εκάστοτε εφαρμογής. Για απλή ανάκληση σημασιολογικά παρόμοιων κειμένων, μία διανυσματική δομή μπορεί να είναι επαρκής. Όταν, αντίθετα, ενδιαφέρει η διατήρηση της προέλευσης, της χρονικής εξέλιξης ή των σχέσεων μεταξύ πολλαπλών στοιχείων μιας εμπειρίας, η γραφική αναπαράσταση παρέχει πιο ρητή δομή. Οι υβριδικές αρχιτεκτονικές επιχειρούν να αξιοποιήσουν τα πλεονεκτήματα και των δύο προσεγγίσεων και εμφανίζονται σε σύγχρονα συστήματα μακροπρόθεσμης μνήμης, τα οποία παρουσιάζονται αναλυτικότερα σε επόμενες ενότητες [23, 26].

2.7 Μεγάλα Γλωσσικά Μοντέλα και Γράφοι Γνώσης

2.7.1 Αλληλεπίδραση Μεγάλων Γλωσσικών Μοντέλων και Γράφων Γνώσης

Τα Μεγάλα Γλωσσικά Μοντέλα και οι γράφοι γνώσης παρουσιάζουν συμπληρωματικά χαρακτηριστικά ως προς την αναπαράσταση και την αξιοποίηση της γνώσης. Τα γλωσσικά μοντέλα παρέχουν ισχυρές δυνατότητες κατανόησης και παραγωγής φυσικής γλώσσας, ενώ οι γράφοι γνώσης οργανώνουν την πληροφορία σε μία ρητή δομή οντοτήτων, ιδιοτήτων και μεταξύ τους σχέσεων. Ο συνδυασμός των δύο τεχνολογιών επιτρέπει σε συστήματα που βασίζονται σε Μεγάλα Γλωσσικά Μοντέλα να αξιοποιούν εξωτερικά αποθηκευμένη και δομημένη γνώση κατά την εκτέλεση εργασιών συλλογισμού και ερωταποκρίσεων [5, 8, 30].

Σε ένα τέτοιο πλαίσιο, ο γράφος γνώσης μπορεί να λειτουργήσει ως εξωτερική πηγή πληροφορίας για το γλωσσικό μοντέλο. Αντί η απόκριση να βασίζεται αποκλειστικά στην πληροφορία που έχει ενσωματωθεί στις παραμέτρους του μοντέλου ή βρίσκεται ήδη στο πλαίσιο συμφραζομένων, σχετική γνώση μπορεί να ανακτηθεί από τον γράφο και να χρησιμοποιηθεί κατά την παραγωγή της απάντησης. Η γραφική δομή επιτρέπει επιπλέον την αξιοποίηση των ρητών σχέσεων μεταξύ των αποθηκευμένων οντοτήτων, παρέχοντας πληροφορία που δεν περιορίζεται στη σημασιολογική ομοιότητα μεμονωμένων τμημάτων κειμένου [5].

Παράλληλα, τα Μεγάλα Γλωσσικά Μοντέλα μπορούν να λειτουργήσουν ως διεπαφή μεταξύ του χρήστη και της δομημένης πληροφορίας ενός γράφου γνώσης. Μία ερώτηση διατυπωμένη σε φυσική γλώσσα μπορεί να μετασχηματιστεί σε δομημένο ερώτημα προς τη βάση, τα αποτελέσματα του οποίου χρησιμοποιούνται στη συνέχεια για την παραγωγή της τελικής απόκρισης. Με τον τρόπο αυτό, ο χρήστης μπορεί να προσπελάσει πληροφορία από έναν γράφο χωρίς να απαιτείται να γνωρίζει ο ίδιος τη γλώσσα ερωτημάτων της βάσης [7, 8].

Η αποτελεσματικότητα αυτής της αλληλεπίδρασης εξαρτάται σε μεγάλο βαθμό από την ικανότητα του γλωσσικού μοντέλου να αντιστοιχίζει σωστά τις έννοιες της φυσικής γλώσσας στη δομή του υποκείμενου γράφου. Η πληροφορία σχετικά με το σχήμα (schema), τις διαθέσιμες οντότητες, τις σχέσεις και τις ιδιότητές τους αποκτά επομένως ιδιαίτερη σημασία κατά την παραγωγή δομημένων ερωτημάτων [7, 8]. Για βάσεις γράφων όπως το Neo4j, η διαδικασία αυτή οδηγεί στο πρόβλημα της αυτόματης παραγωγής ερωτημάτων στη γλώσσα Cypher, το οποίο εξετάζεται στην επόμενη υποενότητα.

2.7.2 Text-to-Cypher και η Αρχιτεκτονική CyVerACT

Η παραγωγή δομημένων ερωτημάτων από φυσική γλώσσα αποσκοπεί στη μετατροπή μιας ερώτησης του χρήστη σε ένα ερώτημα συμβατό με το σύστημα διαχείρισης της

υποκείμενης βάσης. Στην περίπτωση του Neo4j, η εργασία αυτή είναι γνωστή ως Text-to-Cypher και έχει ως στόχο την παραγωγή ενός ερωτήματος στη δηλωτική γλώσσα Cypher. Η Cypher έχει σχεδιαστεί για την αναζήτηση δεδομένων που οργανώνονται σύμφωνα με το μοντέλο γράφου ιδιοτήτων και επιτρέπει την περιγραφή μοτίβων κόμβων και σχέσεων, την εφαρμογή συνθηκών και την ανάκτηση των αντίστοιχων δεδομένων από τον γράφο [25].

Η παραγωγή ενός ορθού ερωτήματος Cypher απαιτεί από το γλωσσικό μοντέλο να αντιμετωπίσει ταυτόχρονα τη γλωσσική και τη δομική διάσταση του προβλήματος. Αρχικά πρέπει να κατανοήσει τη σημασιολογία της ερώτησης και να αναγνωρίσει τις οντότητες, τις σχέσεις και τους περιορισμούς που αυτή περιγράφει. Στη συνέχεια, οι έννοιες αυτές πρέπει να αντιστοιχιστούν σωστά στα στοιχεία του σχήματος του γράφου και να εκφραστούν μέσω ενός συντακτικά ορθού ερωτήματος Cypher [7, 8].

Η διαδικασία αυτή παρουσιάζει σημαντικές δυσκολίες. Ένα γλωσσικό μοντέλο μπορεί να παράγει ένα συντακτικά ορθό ερώτημα το οποίο, ωστόσο, αναφέρεται σε ετικέτες, τύπους σχέσεων ή ιδιότητες που δεν υπάρχουν στο σχήμα της βάσης. Είναι επίσης δυνατό να παρερμηνεύσει τις σχέσεις μεταξύ των οντοτήτων, να χρησιμοποιήσει λανθασμένη κατεύθυνση σε μία σχέση ή να αποτύχει στην ορθή διατύπωση πιο σύνθετων ερωτημάτων. Τα προβλήματα αυτά καθιστούν την απλή παραγωγή ενός ερωτήματος από το γλωσσικό μοντέλο ανεπαρκή για την εγγύηση τόσο της εκτελεσιμότητας όσο και της ορθότητάς του [7, 8].

Για την παραγωγή ερωτημάτων Cypher έχουν αξιοποιηθεί Μεγάλα Γλωσσικά Μοντέλα μέσω τεχνικών μάθησης εντός συμφραζομένων (in-context learning), στις οποίες το μοντέλο λαμβάνει ως είσοδο την ερώτηση του χρήστη μαζί με μία αναπαράσταση του σχήματος της βάσης. Ανάλογα με τη διαμόρφωση του prompt, μπορούν επίσης να παρέχονται παραδείγματα αντίστοιχων ζευγών φυσικής γλώσσας και Cypher. Η ποιότητα της παρεχόμενης πληροφορίας του σχήματος αποτελεί σημαντικό παράγοντα για την ορθή αντιστοίχιση της ερώτησης στις οντότητες και στις σχέσεις του γράφου [7, 8].

Πέρα από την εφάπαξ παραγωγή ενός ερωτήματος (single-shot generation), ενδιαφέρον παρουσιάζουν επαναληπτικές προσεγγίσεις στις οποίες το αρχικό ερώτημα αξιολογείται και, όταν εντοπιστούν προβλήματα, παρέχεται κατάλληλη πληροφορία στο γλωσσικό μοντέλο ώστε να πραγματοποιηθεί νέα διορθωτική προσπάθεια. Με τον τρόπο αυτό, η παραγωγή του ερωτήματος μετατρέπεται από μία μεμονωμένη ενέργεια σε μία διαδικασία διαδοχικής παραγωγής, αξιολόγησης και διόρθωσης [8].

Ανάλογες σχεδιαστικές αρχές έχουν διερευνηθεί και στο συγγενές πρόβλημα Text-to-SQL. Ενδεικτικά, το LitE-SQL συνδυάζει διανυσματική αντιστοίχιση του σχήματος (vector-based schema linking) με μηχανισμό αυτοδιόρθωσης που καθοδηγείται από την εκτέλεση (execution-guided self-correction). Η προσέγγιση αυτή αναδεικνύει τη χρησιμότητα τόσο της επιλογής σχετικών στοιχείων του σχήματος όσο και της αξιοποίησης ανατροφοδότησης από την εκτέλεση σε εργασίες μετατροπής φυσικής γλώσσας σε δο-

μημένα ερωτήματα [31].

Για το πρόβλημα Text-to-Cypher ωστόσο, χαρακτηριστική προσέγγιση αποτελεί το CyVerACT, μία πρακτορική ροή εργασίας (agentic workflow) για την παραγωγή ερωτημάτων Cypher πάνω σε γράφους γνώσης. Το CyVerACT αρχικά περιορίζει το διαθέσιμο σχήμα του γράφου στα στοιχεία που θεωρούνται σχετικά με την ερώτηση και χρησιμοποιεί ένα Μεγάλο Γλωσσικό Μοντέλο για την παραγωγή του αρχικού ερωτήματος. Το παραγόμενο ερώτημα αξιολογείται μέσω του CyVer ως προς τη συντακτική εγκυρότητα και τη συμμόρφωσή του με το σχήμα και τις ιδιότητες της βάσης. Όταν εντοπίζονται σφάλματα, τα αντίστοιχα μεταδεδομένα παρέχονται σε έναν μηχανισμό διόρθωσης και η διαδικασία επαναλαμβάνεται μέχρι να παραχθεί έγκυρο ερώτημα ή να εξαντληθεί ο προκαθορισμένος αριθμός προσπαθειών [8].

Η επαναληπτική λειτουργία του CyVerACT δημιουργεί ταυτόχρονα ένα κατάλληλο υπόβαθρο για την αξιοποίηση προηγούμενης εμπειρίας. Κατά την εκτέλεση του συστήματος παράγεται πληροφορία σχετικά με τις ερωτήσεις που έχουν αντιμετωπιστεί, τα ερωτήματα Cypher που δημιουργήθηκαν, τα σφάλματα που εντοπίστηκαν, τις διορθωτικές προσπάθειες που ακολούθησαν και τις τελικές επιτυχείς λύσεις. Η διατήρηση και επιλεκτική ανάκτηση τέτοιας πληροφορίας σε μεταγενέστερα συναφή προβλήματα μπορεί να επιτρέψει στον πράκτορα να αξιοποιεί γνώση που αποκτήθηκε από προηγούμενες εκτελέσεις, αντί κάθε νέο ερώτημα να αντιμετωπίζεται ανεξάρτητα. Η παρατήρηση αυτή συνδέει άμεσα την επαναληπτική διαδικασία παραγωγής Cypher με τις αρχιτεκτονικές μακροπρόθεσμης μνήμης που παρουσιάζονται στη συνέχεια.

2.8 Σχετικές Αρχιτεκτονικές Μακροπρόθεσμης Μνήμης

Η ανάγκη διατήρησης και επαναχρησιμοποίησης πληροφορίας πέρα από το ενεργό πλαίσιο συμφραζομένων έχει οδηγήσει στην ανάπτυξη διαφορετικών αρχιτεκτονικών μακροπρόθεσμης μνήμης για πράκτορες που βασίζονται σε Μεγάλα Γλωσσικά Μοντέλα. Οι προσεγγίσεις αυτές διαφέρουν τόσο ως προς το είδος της πληροφορίας που αποθηκεύουν όσο και ως προς τον τρόπο με τον οποίο οργανώνουν, ενημερώνουν και ανακτούν προηγούμενες εμπειρίες. Στην παρούσα ενότητα παρουσιάζονται αντιπροσωπευτικές αρχιτεκτονικές που αναδεικνύουν διαφορετικές κατευθύνσεις σχεδιασμού, από την ιεραρχική διαχείριση του διαθέσιμου συμφραζομένου έως τη γραφική οργάνωση επεισοδιακής γνώσης και την εξαγωγή επαναχρησιμοποιήσιμων διαδικασιών από προηγούμενες τροχιές εκτέλεσης.

2.8.1 MemGPT

Το MemGPT προσεγγίζει το πρόβλημα της μακροπρόθεσμης μνήμης αντλώντας έμπνευση από τα ιεραρχικά συστήματα μνήμης των λειτουργικών συστημάτων. Κεντρική του

ιδέα είναι η εικονική διαχείριση συμφραζομένου (virtual context management), μέσω της οποίας η περιορισμένη χωρητικότητα του context window αντιμετωπίζεται ως ένας γρήγορος αλλά πεπερασμένος χώρος μνήμης, ενώ επιπλέον πληροφορία διατηρείται σε εξωτερικό αποθηκευτικό χώρο και μεταφέρεται επιλεκτικά στο ενεργό συμφραζόμενο όταν απαιτείται [1]. Με τον τρόπο αυτό, το γλωσσικό μοντέλο δεν χρειάζεται να έχει ταυτόχρονα διαθέσιμο ολόκληρο το ιστορικό μιας μακροχρόνιας αλληλεπίδρασης.

Η αρχιτεκτονική οργανώνει τη διαθέσιμη πληροφορία σε διαφορετικά επίπεδα μνήμης. Η βασική μνήμη (core memory) περιλαμβάνει πληροφορίες που πρέπει να παραμένουν άμεσα προσβάσιμες στο μοντέλο, όπως στοιχεία που χαρακτηρίζουν τον πράκτορα ή τον χρήστη. Η μνήμη ανάκλησης (recall memory) διατηρεί το ιστορικό προηγούμενων μηνυμάτων, ενώ η αρχειακή μνήμη (archival memory) παρέχει εξωτερικό χώρο για πληροφορίες που δεν είναι απαραίτητο να παραμένουν συνεχώς μέσα στο ενεργό πλαίσιο συμφραζομένων [1]. Η πρόσβαση στα εξωτερικά επίπεδα πραγματοποιείται μέσω κατάλληλων λειτουργιών, επιτρέποντας στο ίδιο το σύστημα να αναζητά και να μεταφέρει πληροφορία μεταξύ των διαφορετικών επιπέδων.

Η συνεισφορά του MemGPT βρίσκεται κυρίως στην αντιμετώπιση της μνήμης ως διαχειριζόμενου πόρου και όχι ως ενός στατικού ιστορικού που προστίθεται αυτούσιο σε κάθε prompt. Η αρχιτεκτονική αυτή καθιστά σαφή τη διάκριση μεταξύ της πληροφορίας που πρέπει να είναι άμεσα διαθέσιμη και εκείνης που μπορεί να παραμένει σε εξωτερική μακροπρόθεσμη αποθήκευση μέχρι να καταστεί σχετική. Ωστόσο, το κύριο βάρος δίνεται στη διαχείριση και μετακίνηση της πληροφορίας μεταξύ επιπέδων και όχι στη ρητή αναπαράσταση των σχέσεων μεταξύ διαφορετικών αναμνήσεων. Η ανάγκη για πιο δομημένη οργάνωση της εμπειρίας αποτελεί κεντρικό στοιχείο των γραφικών προσεγγίσεων που ακολουθούν.

2.8.2 AriGraph και A-MEM

Το AriGraph αποτελεί μία προσέγγιση μακροπρόθεσμης μνήμης στην οποία η προηγούμενη εμπειρία του πράκτορα οργανώνεται σε μορφή γράφου. Η αρχιτεκτονική συνδυάζει σημασιολογική και επεισοδιακή μνήμη σε ένα ενιαίο μοντέλο, με στόχο ο πράκτορας να συσσωρεύει και να ενημερώνει γνώση καθώς αλληλεπιδρά με το περιβάλλον του [6]. Η σημασιολογική μνήμη αναπαρίσταται μέσω οντοτήτων και σχέσεων που εξάγονται από τις παρατηρήσεις του πράκτορα, ενώ η επεισοδιακή μνήμη διατηρεί τις ίδιες τις προηγούμενες παρατηρήσεις και τη σύνδεσή τους με τη γνώση που προέκυψε από αυτές.

Ειδικότερα, από κάθε νέα παρατήρηση εξάγονται σημασιολογικές τριάδες της μορφής οντότητα-σχέση-οντότητα, οι οποίες ενσωματώνονται στον γράφο και μπορούν να ενημερώνουν ήδη αποθηκευμένη γνώση. Παράλληλα, δημιουργείται μία επεισοδιακή εγγραφή που αντιστοιχεί στη συγκεκριμένη παρατήρηση και συνδέεται με τις σημα-

σιολογικές σχέσεις που εξήχθησαν από αυτή. Με τον τρόπο αυτό διατηρείται τόσο η πιο αφαιρετική γνώση για το περιβάλλον όσο και η προέλευσή της από συγκεκριμένες προηγούμενες εμπειρίες [6].

Αντίστοιχα, η ανάκτηση στο AriGraph αξιοποιεί και τις δύο μορφές μνήμης. Αρχικά εντοπίζονται σημασιολογικά σχετικές πληροφορίες στον γράφο και η συνδεδεμένη δομή χρησιμοποιείται για την επέκταση της αναζήτησης. Στη συνέχεια, οι ανακτημένες σημασιολογικές σχέσεις μπορούν να οδηγήσουν στις επεισοδιακές παρατηρήσεις με τις οποίες συνδέονται. Η προσέγγιση αυτή αναδεικνύει ένα βασικό πλεονέκτημα της γραφικής αναπαράστασης: η ανάκτηση μιας μνήμης δεν βασίζεται μόνο στην ομοιότητα του κειμενικού της περιεχομένου, αλλά μπορεί να αξιοποιήσει και τις ρητές σχέσεις που έχουν σχηματιστεί μεταξύ διαφορετικών στοιχείων της προηγούμενης εμπειρίας [6].

Μία διαφορετική μορφή δυναμικής οργάνωσης της μνήμης προτείνεται από το A-MEM, το οποίο βασίζεται στις αρχές της μεθόδου Zettelkasten. Αντί η μνήμη να αποθηκεύεται ως μία στατική συλλογή ανεξάρτητων εγγραφών, κάθε νέα πληροφορία μετατρέπεται σε μία δομημένη σημείωση μνήμης (memory note), η οποία εμπλουτίζεται με στοιχεία όπως το πλαίσιο, λέξεις-κλειδιά και ετικέτες. Η σημείωση αποκτά επίσης διανυσματική αναπαράσταση, ώστε να είναι δυνατός ο εντοπισμός σημασιολογικά συναφών προηγούμενων αναμνήσεων [4].

Η προσθήκη μιας νέας μνήμης ενεργοποιεί στη συνέχεια μηχανισμό δημιουργίας συνδέσεων (link generation), μέσω του οποίου εντοπίζονται σχετικές υπάρχουσες σημειώσεις και δημιουργούνται μεταξύ τους κατάλληλες συνδέσεις. Επιπλέον, το A-MEM εισάγει τη διαδικασία εξέλιξης της μνήμης (memory evolution), κατά την οποία η εισαγωγή νέας πληροφορίας μπορεί να οδηγήσει στην ενημέρωση του πλαισίου, των λέξεων-κλειδιών ή των ετικετών προηγούμενων αναμνήσεων [4]. Η μνήμη αντιμετωπίζεται επομένως ως μία δυναμικά εξελισσόμενη δομή, στην οποία η οργάνωση της προηγούμενης πληροφορίας μπορεί να μεταβάλλεται καθώς προστίθενται νέες εμπειρίες.

Παρότι το AriGraph και το A-MEM διαφέρουν ως προς τη συγκεκριμένη μοντελοποίηση της μνήμης, μοιράζονται την αρχή ότι οι προηγούμενες εμπειρίες δεν είναι αναγκαίο να αντιμετωπίζονται ως ανεξάρτητες εγγραφές. Η σύνδεση των αναμνήσεων με σημασιολογικές οντότητες ή με άλλες συναφείς αναμνήσεις δημιουργεί μία πιο οργανωμένη δομή, η οποία μπορεί να εξελίσσεται μαζί με τη συσσωρευμένη εμπειρία του πράκτορα.

2.8.3 MIRIX και Memp

Το MIRIX προσεγγίζει τη μακροπρόθεσμη μνήμη ως ένα σύστημα πολλαπλών εξειδικευμένων συστατικών. Η αρχιτεκτονική διακρίνει έξι τύπους μνήμης: βασική (core memory), επεισοδιακή (episodic memory), σημασιολογική (semantic memory), διαδικαστική (procedural memory), μνήμη πόρων (resource memory) και ένα αποθετήριο για πληροφορίες που πρέπει να διατηρούνται με ακρίβεια (Knowledge Vault) [15]. Η διάκριση αυτή επι-

τρέπει σε διαφορετικές μορφές πληροφορίας να οργανώνονται σύμφωνα με τον ρόλο που επιτελούν αντί να αποθηκεύονται όλες σε ένα ενιαίο αποθετήριο.

Η διαχείριση των επιμέρους μορφών μνήμης πραγματοποιείται μέσω εξειδικευμένων πρακτόρων, οι οποίοι συντονίζουν την εγγραφή, την ενημέρωση και την ανάκτηση πληροφορίας. Ιδιαίτερο χαρακτηριστικό του MIRIX αποτελεί επίσης η υποστήριξη πολυτροπικής πληροφορίας, καθώς η μνήμη πόρων μπορεί να διατηρεί υλικό όπως έγγραφα, αρχεία και οπτικό περιεχόμενο. Η αρχιτεκτονική αναδεικνύει έτσι μία διαφορετική κατεύθυνση σχεδιασμού, στην οποία η μνήμη αντιμετωπίζεται ως σύνολο συμπληρωματικών υποσυστημάτων με διακριτές αρμοδιότητες [15].

Ο διαχωρισμός της μνήμης σε επιμέρους συστατικά συναντάται και στην πιο πρόσφατη αρχιτεκτονική Hindsight, με έμφαση στον χαρακτήρα της αποθηκευμένης πληροφορίας. Η αρχιτεκτονική διακρίνει γεγονότα για τον κόσμο, εμπειρίες του πράκτορα, περιλήψεις οντοτήτων και εξελισσόμενες πεποιθήσεις. Έτσι, οι καταγεγραμμένες πληροφορίες διαχωρίζονται από τις εκτιμήσεις που διαμορφώνει ο πράκτορας, υποστηρίζοντας τη διατήρηση της προέλευσης της γνώσης και την ενημέρωσή της [32].

Σε πιο εξειδικευμένη κατεύθυνση κινείται το Memp, το οποίο εστιάζει στη διαδικαστική μνήμη και ειδικότερα στη δυνατότητα ενός πράκτορα να μαθαίνει τρόπους εκτέλεσης εργασιών από προηγούμενες εμπειρίες. Η προσέγγιση ξεκινά από καταγεγραμμένες τροχιές εκτέλεσης (agent trajectories) και αποστάζει από αυτές διαδικαστική γνώση σε δύο επίπεδα: λεπτομερείς οδηγίες βήμα προς βήμα και πιο αφαιρετικά πρότυπα ή σενάρια εκτέλεσης (scripts) [14].

Η διαδικαστική μνήμη του Memp δεν θεωρείται στατική. Η αρχιτεκτονική εξετάζει διακριτούς μηχανισμούς κατασκευής (Build), ανάκτησης (Retrieval) και ενημέρωσης (Update), ώστε οι αποθηκευμένες διαδικασίες να μπορούν να εμπλουτίζονται, να διορθώνονται ή να παύουν να χρησιμοποιούνται καθώς συσσωρεύονται νέες εμπειρίες [14]. Σε αντίθεση επομένως με μία αμιγώς επεισοδιακή προσέγγιση, όπου ανακτάται μία συγκεκριμένη προηγούμενη περίπτωση, το Memp επιδιώκει να μετασχηματίσει τις επιμέρους τροχιές σε πιο γενικευμένη γνώση σχετικά με τον τρόπο επίλυσης μιας εργασίας.

Οι παραπάνω αρχιτεκτονικές αποτυπώνουν διαφορετικές επιλογές ως προς το τι θεωρείται μνήμη και πώς αυτή πρέπει να οργανώνεται. Το MemGPT επικεντρώνεται στη διαχείριση διαφορετικών επιπέδων αποθήκευσης, τα AriGraph και A-MEM δίνουν έμφαση στις σχέσεις και στη δυναμική οργάνωση προηγούμενων εμπειριών, ενώ τα MIRIX και Memp αναδεικνύουν αντίστοιχα τη συνύπαρξη διαφορετικών τύπων μνήμης και την εξαγωγή διαδικαστικής γνώσης από την ιστορική συμπεριφορά του πράκτορα. Οι αρχές αυτές έχουν υιοθετηθεί σε διαφορετικό βαθμό από σύγχρονα πλαίσια και εργαλεία διαχείρισης μνήμης, τα οποία παρουσιάζονται στην επόμενη ενότητα.

Μία συγκριτική σύνοψη επιλεγμένων αρχιτεκτονικών παρουσιάζεται στον Πίνακα 2.1. Στη στήλη της αναπαράστασης περιγράφεται η μορφή των αποθηκευμένων μονάδων πληροφορίας, όπως κείμενο, τριάδες γνώσης ή δομημένες σημειώσεις. Στη

στήλη της δομής περιγράφεται ο τρόπος με τον οποίο οι μονάδες αυτές οργανώνονται και συνδέονται, όπως σε αποθετήρια, γράφους ή εξειδικευμένα συστατικά μνήμης.

Πίνακας 2.1: Συγκριτική σύνοψη αντιπροσωπευτικών αρχιτεκτονικών μακροπρόθεσμης μνήμης.

Αρχιτεκτονική	**Κατηγορία μνήμης**	**Αναπαράσταση**	**Δομή**	**Βασικές λειτουργίες**
MemGPT [1]	Βραχυπρόθεσμη και μακροπρόθεσμη	Κείμενο, ιστορικό μηνυμάτων και αρχειακές εγγραφές	Ενεργό πλαίσιο και εξωτερικά αποθετήρια ανάκλησης και αρχειακής μνήμης	**Εγγραφή:** Καταγραφή μηνυμάτων. **Διαχείριση:** Μεταφορά πληροφορίας μεταξύ επιπέδων. **Ανάκτηση:** Εισαγωγή σχετικής πληροφορίας στο ενεργό πλαίσιο.
AriGraph [6]	Επεισοδιακή και σημασιολογική	Τριάδες γνώσης και επεισοδιακές παρατηρήσεις	Γράφος που συνδέει σημασιολογική γνώση με επεισόδια	**Εγγραφή/ενημέρωση:** Προσθήκη νέων παρατηρήσεων και αναθεώρηση σχέσεων στον γράφο. **Ανάκτηση:** Εντοπισμός συναφών τριάδων και συνδεδεμένων επεισοδίων.
A-MEM [4]	Μακροπρόθεσμη	Δομημένες σημειώσεις με μεταδεδομένα και διανύσματα	Δυναμικό δίκτυο συνδεδεμένων σημειώσεων	**Εγγραφή:** Δημιουργία σημείωσης και συνδέσεων. **Ενημέρωση:** Αναθεώρηση συναφών σημειώσεων με νέα εμπειρία. **Ανάκτηση:** Επιλογή σχετικών σημειώσεων.
MIRIX [15]	Πολλαπλοί τύποι μνήμης	Δομημένες εγγραφές και αναφορές σε πολυτροπικό υλικό	Έξι εξειδικευμένα συστατικά μνήμης	**Εγγραφή/ενημέρωση:** Δρομολόγηση νέας πληροφορίας στα κατάλληλα συστατικά. **Ανάκτηση:** Αναζήτηση με βάση το θέμα και τα διαθέσιμα εργαλεία του πράκτορα.
Memp [14]	Διαδικαστική	Οδηγίες βημάτων και αφαιρετικά σενάρια εκτέλεσης	Αποθετήριο διαδικασιών	**Κατασκευή:** Εξαγωγή διαδικασιών από τροχιές. **Ανάκτηση:** Επιλογή διαδικασιών για νέα εργασία. **Ενημέρωση:** Προσθήκη ή διόρθωση με βάση νέες εκτελέσεις.

2.9 Πλαίσια και Δομές Διαχείρισης Μνήμης

Πέρα από τις ερευνητικές αρχιτεκτονικές μακροπρόθεσμης μνήμης, έχουν αναπτυχθεί πλαίσια και δομές που παρέχουν μηχανισμούς αποθήκευσης, οργάνωσης και ανάκτησης της εμπειρίας πρακτόρων σε πραγματικές εφαρμογές. Οι λύσεις αυτές διαφέρουν ως προς το επίπεδο αφαίρεσης που προσφέρουν. Ορισμένες λειτουργούν ως ολοκληρωμένα επίπεδα μνήμης, αναλαμβάνοντας αυτόματα την εξαγωγή και ανάκτηση σχετικής πληροφορίας, ενώ άλλες παρέχουν τη βασική υποδομή πάνω στην οποία μπορεί να σχεδιαστεί ένα εξειδικευμένο μοντέλο μνήμης. Στην ενότητα αυτή παρουσιάζονται πλαίσια και τεχνολογίες που σχετίζονται άμεσα με τη γραφική αναπαράσταση και τη διαχείριση μακροπρόθεσμης μνήμης σε πράκτορες.

2.9.1 Neo4j ως Υποδομή Γραφικής Μνήμης

Το Neo4j αποτελεί σύστημα διαχείρισης βάσεων γράφων που βασίζεται στο μοντέλο γράφου ιδιοτήτων και επιτρέπει την αναπαράσταση της πληροφορίας μέσω κόμβων, κατευθυνόμενων σχέσεων και ιδιοτήτων. Αν και δεν αποτελεί από μόνο του ένα εξειδικευμένο πλαίσιο μνήμης για πράκτορες, παρέχει την υποδομή για την κατασκευή προσαρμοσμένων συστημάτων μνήμης, στα οποία διαφορετικά είδη προηγούμενης πληροφορίας μπορούν να αποθηκεύονται ως οντότητες και να συνδέονται μέσω ρητά καθορισμένων σχέσεων. Η γραφική αυτή αναπαράσταση επιτρέπει την ανάκτηση πληροφορίας όχι μόνο με βάση το περιεχόμενο μιας μεμονωμένης εγγραφής, αλλά και μέσω των συνδέσεών της με άλλες αναμνήσεις [12].

Η χρήση του Neo4j για τη μοντελοποίηση μνήμης έχει εξεταστεί για διαφορετικές κατηγορίες πληροφορίας. Ενδεικτικά, έχουν προταθεί γραφικά μοντέλα για σημασιολογική, επεισοδιακή, διαδικαστική και χρονική μνήμη. Στην περίπτωση της επεισοδιακής μνήμης ενός πράκτορα παραγωγής Cypher, προηγούμενες ερωτήσεις μπορούν να αποθηκεύονται μαζί με τα ερωτήματα Cypher που χρησιμοποιήθηκαν επιτυχώς για την αντιμετώπισή τους. Η διανυσματική αναπαράσταση των ερωτήσεων επιτρέπει τον εντοπισμό σημασιολογικά συναφών προηγούμενων περιπτώσεων, ενώ η γραφική σύνδεση οδηγεί από την ανακτημένη ερώτηση στην αντίστοιχη λύση [12]. Το παράδειγμα αυτό αναδεικνύει τη δυνατότητα συνδυασμού διανυσματικής αναζήτησης και διάσχισης γράφου μέσα στην ίδια υποδομή.

Μία πιο ολοκληρωμένη εφαρμογή αυτής της λογικής παρουσιάζεται στο Lenny's Memory, όπου η μνήμη ενός πράκτορα οργανώνεται ως γράφος συμφραζομένων (context graph). Η προσέγγιση διακρίνει πληροφορία που σχετίζεται με το ιστορικό της τρέχουσας αλληλεπίδρασης, μακροπρόθεσμη γνώση για οντότητες και μεταξύ τους σχέσεις, καθώς και ίχνη προηγούμενων ενεργειών και αποφάσεων του πράκτορα (reasoning traces). Η συνύπαρξη των διαφορετικών αυτών πληροφοριών στον ίδιο γράφο επιτρέπει τη διατήρηση της προέλευσης και των συσχετίσεων μεταξύ ερωτήσεων, ενεργειών και

αποτελεσμάτων [33]. Το Neo4j μπορεί επομένως να λειτουργήσει ως χαμηλότερου επιπέδου υποδομή πάνω στην οποία το ίδιο το σύστημα καθορίζει το σχήμα της μνήμης, τις συνθήκες εγγραφής και τις στρατηγικές ανάκτησης.

2.9.2 Zep και Graphiti

Το Zep αποτελεί ένα εξειδικευμένο επίπεδο μακροπρόθεσμης μνήμης για πράκτορες, το οποίο οργανώνει πληροφορίες από προηγούμενες αλληλεπιδράσεις και άλλες πηγές σε έναν χρονικά ενήμερο γράφο γνώσης (temporal knowledge graph). Κεντρικό στοιχείο της αρχιτεκτονικής του αποτελεί το Graphiti, ένας μηχανισμός κατασκευής και ενημέρωσης δυναμικών γράφων γνώσης, ο οποίος έχει σχεδιαστεί ώστε να ενσωματώνει συνεχώς νέα επεισόδια χωρίς να απαιτείται ανακατασκευή ολόκληρου του γράφου [26, 34].

Η πληροφορία οργανώνεται γύρω από επεισόδια (episodes), τα οποία μπορούν να προέρχονται από συνομιλίες, μη δομημένο κείμενο ή δομημένα δεδομένα. Από τα επεισόδια εξάγονται οντότητες και γεγονότα, τα οποία συνδέονται στον γράφο μέσω σχέσεων. Ιδιαίτερο χαρακτηριστικό της αρχιτεκτονικής είναι η διχρονική (bi-temporal) αναπαράσταση, η οποία διαχωρίζει τον χρόνο κατά τον οποίο ένα γεγονός θεωρείται έγκυρο από τον χρόνο κατά τον οποίο η πληροφορία εισήχθη στο σύστημα. Με τον τρόπο αυτό, μία νέα πληροφορία μπορεί να καταστήσει παλαιότερη γνώση μη έγκυρη χωρίς να απαιτεί τη διαγραφή της, επιτρέποντας τη διατήρηση του ιστορικού των μεταβολών [26, 34].

Η ανάκτηση πληροφορίας στο Graphiti συνδυάζει σημασιολογική αναζήτηση μέσω διανυσματικών αναπαραστάσεων, λεξική αναζήτηση και αξιοποίηση της δομής του γράφου. Ο συνδυασμός αυτός επιτρέπει την επιλογή μνημών με βάση τόσο τη σημασιολογική συνάφεια όσο και τις σχέσεις μεταξύ των αποθηκευμένων οντοτήτων [26]. Το Zep παρέχει ένα υψηλότερου επιπέδου περιβάλλον κλειστού κώδικα για την αξιοποίηση της συγκεκριμένης αρχιτεκτονικής, ενώ το Graphiti προσφέρει πιο άμεση πρόσβαση στον μηχανισμό κατασκευής και διαχείρισης του χρονικού γράφου μνήμης.

2.9.3 Cognee

Το Cognee προσεγγίζει τη μακροπρόθεσμη μνήμη ως μία διαδικασία μετατροπής ετερογενών δεδομένων σε δομημένη και ανακτήσιμη γνώση. Το πλαίσιο συνδυάζει γραφικές και διανυσματικές αναπαραστάσεις, επιτρέποντας την αξιοποίηση τόσο της σημασιολογικής ομοιότητας όσο και των σχέσεων μεταξύ των αποθηκευμένων εννοιών. Έχει χρησιμοποιηθεί ως αρθρωτό πλαίσιο για την κατασκευή και ανάκτηση γράφων γνώσης και επιτρέπει την παραμετροποίηση διαφορετικών σταδίων, όπως ο τεμαχισμός των δεδομένων, η κατασκευή του γράφου, η στρατηγική ανάκτησης και η διαμόρφωση των προτροπών προς το γλωσσικό μοντέλο [5].

Στη ροή που εξετάστηκε στο πλαίσιο της παρούσας εργασίας, η εισαγωγή των δεδομένων πραγματοποιείται αρχικά μέσω μηχανισμού προσθήκης περιεχομένου, ενώ

στη συνέχεια η διαδικασία cognify αναλαμβάνει την επεξεργασία του υλικού και την κατασκευή της γραφικής αναπαράστασης. Το περιεχόμενο διαχωρίζεται σε επιμέρους τμήματα και ένα Μεγάλο Γλωσσικό Μοντέλο χρησιμοποιείται για την εξαγωγή οντοτήτων και σχέσεων. Οι παραγόμενες οντότητες οργανώνονται σε γράφο, ενώ παράλληλα δημιουργούνται διανυσματικές αναπαραστάσεις για την υποστήριξη σημασιολογικής ανάκτησης. Η αναζήτηση μπορεί επομένως να συνδυάσει διανυσματικά και γραφικά στοιχεία, ανάλογα με την επιλεγμένη στρατηγική [35, 36].

Ένα ακόμη χαρακτηριστικό του Cognee είναι η δυνατότητα προσαρμογής του μοντέλου γνώσης και η ενσωμάτωσή του σε πρακτορικά συστήματα. Η μνήμη μπορεί να χρησιμοποιηθεί από διαφορετικά συστατικά ενός πράκτορα μέσω λειτουργιών εγγραφής και αναζήτησης, ενώ έχουν προταθεί συνδέσεις με πλαίσια όπως το LangGraph για την παροχή επίμονης (persistent) και αναζητήσιμης μνήμης μεταξύ διαφορετικών εκτελέσεων [37]. Η υψηλού επιπέδου αυτοματοποίηση της κατασκευής του γράφου διαφοροποιεί το Cognee από προσεγγίσεις στις οποίες το σχήμα της μνήμης και οι μηχανισμοί εγγραφής ορίζονται εξ ολοκλήρου από την εφαρμογή.

2.9.4 Mem0

Το Mem0 αποτελεί μία αρχιτεκτονική μακροπρόθεσμης μνήμης που σχεδιάστηκε με έμφαση στη διατήρηση σημαντικής πληροφορίας από μακροχρόνιες και πολλαπλές συνεδρίες αλληλεπίδρασης. Αντί να διατηρεί ολόκληρο το ιστορικό της συνομιλίας, το σύστημα εξάγει επιλεκτικά πληροφορίες που θεωρούνται χρήσιμες και τις ενσωματώνει στη μακροπρόθεσμη μνήμη. Η βασική διαδικασία οργανώνεται γύρω από την εξαγωγή νέων μνημών και την ενημέρωση των ήδη αποθηκευμένων εγγραφών, ώστε να περιορίζονται οι επαναλήψεις και οι αντιφατικές πληροφορίες [23].

Για κάθε νέα υποψήφια μνήμη ανακτώνται αρχικά σημασιολογικά συναφείς υπάρχουσες εγγραφές. Με βάση τη σχέση μεταξύ της νέας και της ήδη αποθηκευμένης πληροφορίας, το σύστημα μπορεί να δημιουργήσει νέα μνήμη, να ενημερώσει μία υπάρχουσα ή να αφαιρέσει πληροφορία που έχει καταστεί ασυνεπής. Πέρα από τη βασική αυτή μορφή, το Mem0 προτείνει και γραφική επέκταση (graph memory), στην οποία οι μνήμες αναπαρίστανται ως κατευθυνόμενος γράφος με οντότητες ως κόμβους και τις μεταξύ τους σχέσεις ως ακμές [23].

Η γραφική επέκταση επιτρέπει τη διατήρηση συνδέσεων που δεν αποτυπώνονται επαρκώς μέσω ανεξάρτητων διανυσματικών εγγραφών και διευκολύνει την ανάκτηση πληροφορίας όταν η απάντηση απαιτεί τη σύνδεση περισσότερων από ενός αποθηκευμένων γεγονότων. Το Mem0 αποτελεί έτσι χαρακτηριστικό παράδειγμα υβριδικής προσέγγισης, όπου η σημασιολογική ανάκτηση συμπληρώνεται από μία ρητή αναπαράσταση των σχέσεων μεταξύ των μνημών [23].

2.9.5 LangChain και LangGraph

Τα LangChain και LangGraph δεν αποτελούν εξειδικευμένα συστήματα αναπαράστασης μνήμης αντίστοιχα με τα προηγούμενα πλαίσια, αλλά παρέχουν αφαιρέσεις για την ενσωμάτωση μνήμης σε εφαρμογές και πρακτορικές ροές εργασίας. Ειδικότερα, το LangGraph μοντελοποιεί μία εφαρμογή ως γράφο κατάστασης και παρέχει μηχανισμούς διατήρησης της κατάστασης κατά τη διάρκεια της εκτέλεσης. Μέσω σημείων ελέγχου (checkpoints), η κατάσταση μιας συνομιλίας ή μιας ροής μπορεί να διατηρηθεί μεταξύ διαδοχικών βημάτων και να επαναχρησιμοποιηθεί σε μεταγενέστερες αλληλεπιδράσεις [11].

Για πληροφορίες που πρέπει να παραμένουν διαθέσιμες πέρα από τα όρια μίας συγκεκριμένης συνομιλίας, το LangGraph διακρίνει τη μακροπρόθεσμη μνήμη από την κατάσταση ενός μεμονωμένου νήματος εκτέλεσης. Οι μακροπρόθεσμες μνήμες μπορούν να οργανώνονται σε ξεχωριστούς χώρους ονομάτων και να ανακτώνται από διαφορετικές συνεδρίες, ενώ είναι δυνατή και η εφαρμογή σημασιολογικής αναζήτησης πάνω στο αποθηκευμένο περιεχόμενο [11].

Η συγκεκριμένη σχεδιαστική λογική διαχωρίζει την ορχήστρωση του πράκτορα από την τεχνολογία στην οποία αποθηκεύεται η μακροπρόθεσμη πληροφορία. Ένα σύστημα που βασίζεται στο LangGraph μπορεί επομένως να χρησιμοποιήσει τον δικό του μηχανισμό αποθήκευσης ή να συνδεθεί με εξωτερικά συστήματα μνήμης, όπως το Cognee [37]. Η δυνατότητα αυτή είναι ιδιαίτερα χρήσιμη σε σύνθετες πρακτορικές ροές, στις οποίες η κατάσταση εκτέλεσης και η μακροπρόθεσμη εμπειρία επιτελούν διαφορετικούς ρόλους.

Οι παραπάνω τεχνολογίες καλύπτουν διαφορετικά επίπεδα αφαίρεσης και διαφορετικές επιλογές ως προς την οργάνωση της μνήμης. Το Neo4j παρέχει μία χαμηλότερου επιπέδου υποδομή για τον άμεσο σχεδιασμό γραφικών μοντέλων μνήμης, ενώ πλαίσια όπως τα Zep, Graphiti, Cognee και Mem0 αυτοματοποιούν σε διαφορετικό βαθμό την εξαγωγή, οργάνωση και ανάκτηση της πληροφορίας. Τα LangChain και LangGraph, από την άλλη πλευρά, παρέχουν μηχανισμούς ορχήστρωσης και διατήρησης κατάστασης μέσω των οποίων διαφορετικές υλοποιήσεις μνήμης μπορούν να ενσωματωθούν σε μία πρακτορική ροή. Η βιβλιογραφική αυτή επισκόπηση ολοκληρώνει το θεωρητικό και τεχνολογικό υπόβαθρο που απαιτείται για την παρουσίαση του μηχανισμού μνήμης που αναπτύχθηκε στο πλαίσιο της παρούσας εργασίας, ο οποίος αναλύεται στο επόμενο κεφάλαιο.

Κεφάλαιο 3

Μεθοδολογία

Στο παρόν κεφάλαιο παρουσιάζεται η μεθοδολογία που ακολουθήθηκε για την ενσωμάτωση μηχανισμών μνήμης σε μια πρακτορική ροή παραγωγής και επαναληπτικής διόρθωσης ερωτημάτων Cypher. Ως βάση της υλοποίησης χρησιμοποιήθηκε η αρχιτεκτονική CyVerACT, η οποία συνδυάζει την παραγωγή ερωτημάτων από μεγάλα γλωσσικά μοντέλα με μηχανισμούς φιλτραρίσματος του σχήματος του γράφου, ντετερμινιστικής αξιολόγησης και επαναληπτικής διόρθωσης [8]. Πάνω σε αυτή τη ροή σχεδιάστηκε ένας πρόσθετος μηχανισμός μνήμης, με στόχο την αποθήκευση της εμπειρίας που παράγεται κατά την επίλυση προηγούμενων ερωτημάτων και την επιλεκτική αξιοποίησή της σε μεταγενέστερες εκτελέσεις.

Αρχικά παρουσιάζεται συνοπτικά η βασική πρακτορική ροή και ο τρόπος με τον οποίο πραγματοποιείται η παραγωγή, αξιολόγηση και διόρθωση των ερωτημάτων Cypher. Στη συνέχεια περιγράφεται η διερεύνηση υφιστάμενων πλαισίων μνήμης και η μετάβαση σε μια προσαρμοσμένη υλοποίηση βασισμένη σε γράφο. Ιδιαίτερη έμφαση δίνεται στο μοντέλο δεδομένων της μνήμης, στα σημεία στα οποία καταγράφεται η εμπειρία κατά την εκτέλεση της ροής, καθώς και στους διαφορετικούς μηχανισμούς ανάκτησης προηγούμενων επιτυχημένων λύσεων και τροχιών διόρθωσης. Τέλος, παρουσιάζεται ο τρόπος με τον οποίο η ανακτημένη πληροφορία ενσωματώνεται στις προτροπές του Cypher Generator και του Cypher Corrector, ενώ οι πειραματικές διαμορφώσεις που προκύπτουν από τους επιμέρους μηχανισμούς εξετάζονται στο επόμενο κεφάλαιο.

3.1 Επισκόπηση της Προτεινόμενης Μεθόδου

Η προτεινόμενη μέθοδος εισάγει έναν μηχανισμό μακροπρόθεσμης μνήμης στην πρακτορική ροή CyVerACT, με στόχο την αξιοποίηση εμπειρίας από προηγούμενες εκτελέσεις κατά την παραγωγή και τη διόρθωση νέων ερωτημάτων Cypher. Στη βασική μορφή της ροής, τα μεταδεδομένα σφαλμάτων που προκύπτουν κατά την τρέχουσα εκτέλε-

ση αξιοποιούνται για την καθοδήγηση των επόμενων προσπαθειών διόρθωσης [8]. Η πληροφορία αυτή, ωστόσο, αποτελεί μέρος της ενεργής κατάστασης της συγκεκριμένης εκτέλεσης και δεν οργανώνεται ως επίμονη εμπειρία η οποία μπορεί να ανακτηθεί και να επαναχρησιμοποιηθεί κατά την επεξεργασία μεταγενέστερων ερωτημάτων.

Η βασική ιδέα της παρούσας εργασίας είναι η επέκταση της παραπάνω διαδικασίας με έναν μηχανισμό μακροπρόθεσμης μνήμης, ο οποίος διατηρεί πληροφορία τόσο για επιτυχημένες λύσεις όσο και για τις ενδιάμεσες αποτυχημένες προσπάθειες που οδήγησαν σε αυτές. Με τον τρόπο αυτό, μια αλληλεπίδραση δεν αντιμετωπίζεται αποκλειστικά ως ένα ανεξάρτητο ερώτημα προς επίλυση, αλλά και ως πιθανή πηγή εμπειρίας για μελλοντικές εκτελέσεις. Η μνήμη λειτουργεί επικουρικά προς τον υπάρχοντα μηχανισμό αξιολόγησης και δεν αντικαθιστά ούτε το σχήμα του γράφου γνώσης ούτε τους ελέγχους εγκυρότητας του CyVer. Αντίθετα, παρέχει πρόσθετο πλαίσιο στα στάδια παραγωγής και διόρθωσης, όταν εντοπίζεται προηγούμενη εμπειρία με επαρκή συνάφεια. Η αξιοποίηση προηγούμενων αλληλεπιδράσεων ως ανακτήσιμης εμπειρίας αποτελεί βασικό χαρακτηριστικό σύγχρονων αρχιτεκτονικών μνήμης για πράκτορες τεχνητής νοημοσύνης [2].

Για τον σκοπό αυτό, η πληροφορία που αποθηκεύεται στη μνήμη οργανώνεται σε διακριτό γράφο Neo4j, στον οποίο αναπαρίστανται οι ερωτήσεις φυσικής γλώσσας, οι διαδοχικές προσπάθειες παραγωγής Cypher, τα σφάλματα που εντοπίστηκαν και οι τελικές επιτυχημένες λύσεις. Η αναπαράσταση αυτή επιτρέπει τη διατήρηση ολόκληρης της διαδρομής που ακολούθησε ο πράκτορας από το αρχικό ερώτημα έως μια έγκυρη λύση και, επομένως, την ανάκτηση όχι μόνο μεμονωμένων προηγούμενων απαντήσεων αλλά και χρήσιμων τροχιών διόρθωσης. Η γραφοκεντρική αναπαράσταση της μνήμης επιτρέπει επιπλέον τη ρητή σύνδεση των διαφορετικών στοιχείων της εμπειρίας και την αξιοποίηση των μεταξύ τους σχέσεων κατά την ανάκτηση [12].

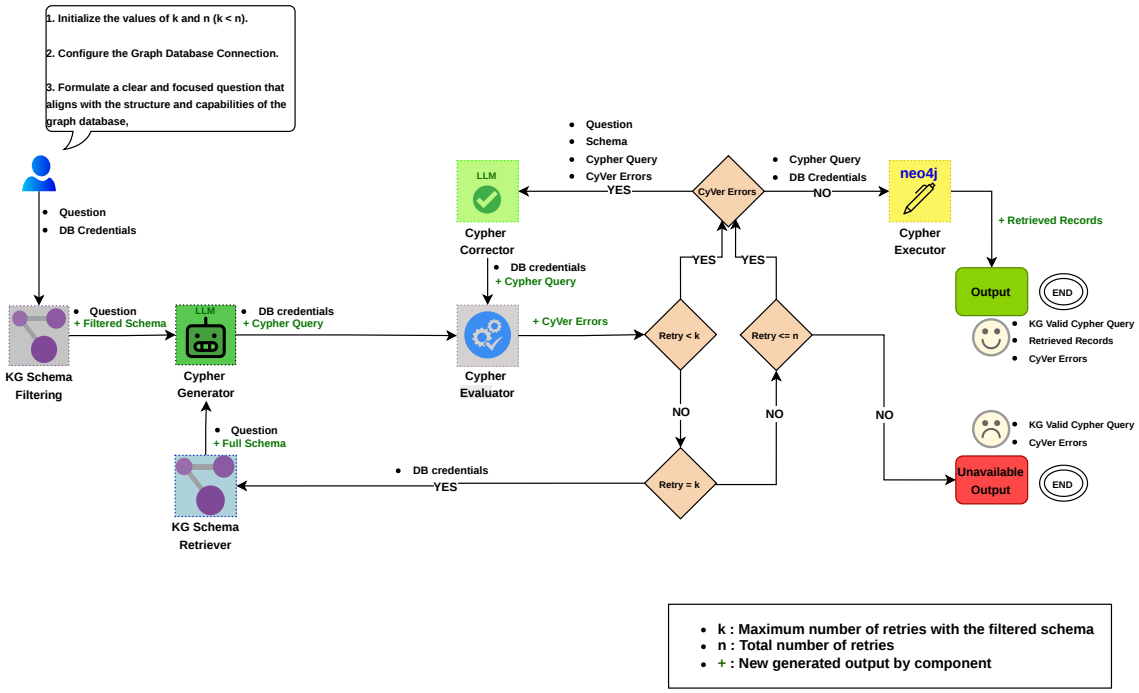
Η ενσωμάτωση της μνήμης πραγματοποιείται σε διακριτά σημεία της πρακτορικής ροής. Κατά την αξιολόγηση καταγράφεται η εμπειρία που προκύπτει από τις διαδοχικές προσπάθειες και τα αντίστοιχα σφάλματα, ενώ μετά από μια επιτυχημένη εκτέλεση αποθηκεύεται η τελική λύση. Η πληροφορία αυτή μπορεί στη συνέχεια να ανακτηθεί είτε πριν από την αρχική παραγωγή είτε κατά το στάδιο της διόρθωσης, ενώ παράλληλα μπορεί να αξιοποιηθεί το βραχυπρόθεσμο ιστορικό της τρέχουσας εκτέλεσης. Οι λειτουργίες αποθήκευσης και ανάκτησης της μνήμης, όπως εφαρμόζονται στην προτεινόμενη αρχιτεκτονική, παρουσιάζονται αναλυτικά στις Ενότητες 3.5 και 3.6.

3.2 Βασική Πρακτορική Ροή CyVerACT

Για την κατανόηση των σημείων στα οποία ενσωματώνεται ο μηχανισμός μνήμης, παρουσιάζεται αρχικά η βασική πρακτορική ροή CyVerACT. Η αρχιτεκτονική έχει σχεδιαστεί για την παραγωγή ερωτημάτων Cypher που είναι συντακτικά έγκυρα και συμβατά με

το σχήμα ενός γράφου γνώσης [8]. Η ροή υλοποιείται με το πλαίσιο LangGraph, το οποίο υποστηρίζει την οργάνωση ροών εκτέλεσης που συνδυάζουν προκαθορισμένες διαδικασίες και ενέργειες πρακτόρων [38]. Στο CyVerACT, τα επιμέρους στάδια αναπαρίστανται ως κόμβοι ενός κατευθυνόμενου γράφου και η μετάβαση μεταξύ τους καθορίζεται από την κατάσταση της εκτέλεσης και τα αποτελέσματα της αξιολόγησης [8].

Η συνολική πρακτορική ροή του CyVerACT παρουσιάζεται στο Σχήμα 3.1.



Σχήμα 3.1: Βασική πρακτορική ροή παραγωγής, αξιολόγησης και επαναληπτικής διόρθωσης ερωτημάτων Cypher στο CyVerACT, προσαρμοσμένη από [8].

Η διαδικασία ξεκινά από την ερώτηση του χρήστη σε φυσική γλώσσα. Αντί να παρέχεται εξαρχής ολόκληρο το σχήμα του γράφου στο γλωσσικό μοντέλο, εφαρμόζεται αρχικά ο μηχανισμός KG Schema Filtering, ο οποίος επιλέγει τα στοιχεία του σχήματος που θεωρούνται σχετικά με το συγκεκριμένο ερώτημα. Το φιλτραρισμένο σχήμα, μαζί με την ερώτηση, μεταβιβάζεται στη συνέχεια στον Cypher Generator, ο οποίος χρησιμοποιεί το επιλεγμένο μεγάλο γλωσσικό μοντέλο για την παραγωγή ενός υποψήφιου ερωτήματος Cypher.

Το παραγόμενο ερώτημα δεν εκτελείται άμεσα στον γράφο γνώσης. Προηγείται η αξιολόγησή του από τον CyVer Evaluator, ο οποίος χρησιμοποιεί τους επιμέρους ελεγκτές του CyVer για τον εντοπισμό συντακτικών σφαλμάτων, ασυμβατοτήτων με το σχήμα και μη έγκυρης πρόσβασης σε ιδιότητες [8]. Τα μεταδεδομένα που επιστρέφονται από τους ελεγκτές περιλαμβάνουν τόσο σφάλματα και προειδοποιήσεις του Neo4j όσο και πρόσθετες κατηγορίες που αφορούν τη δομή του γράφου, όπως περιπτώσεις λανθασμένης κατεύθυνσης μιας σχέσης. Τα μεταδεδομένα αυτά αποτελούν το σήμα ανατροφοδότησης

για το επόμενο στάδιο της ροής.

Εφόσον εντοπιστούν σφάλματα, το ερώτημα μαζί με την ερώτηση, το διαθέσιμο σχήμα και την περιγραφή των σφαλμάτων μεταβιβάζονται στον Cypher Corrector. Ο συγκεκριμένος κόμβος καλεί εκ νέου το γλωσσικό μοντέλο με στόχο την παραγωγή ενός διορθωμένου ερωτήματος, το οποίο επιστρέφει στον CyVer Evaluator και υποβάλλεται στους ίδιους ελέγχους. Δημιουργείται έτσι ένας επαναληπτικός βρόχος αξιολόγησης και διόρθωσης, ο οποίος συνεχίζεται έως ότου παραχθεί ερώτημα χωρίς καταγεγραμμένα σφάλματα ή εξαντληθεί ο διαθέσιμος αριθμός επαναλήψεων.

3.2.1 Φιλτράρισμα του Σχήματος και Παραγωγή Cypher

Το πρώτο στάδιο της ροής αποσκοπεί στον περιορισμό της πληροφορίας του σχήματος που εισάγεται στην προτροπή (prompt) του γλωσσικού μοντέλου. Ο μηχανισμός φιλτραρίσματος επεξεργάζεται τους όρους της ερώτησης και τους αντιστοιχίζει με ετικέτες κόμβων και τύπους σχέσεων του γράφου. Η αντιστοίχιση πραγματοποιείται διαδοχικά με άμεση σύγκριση όρων, μορφολογική κανονικοποίηση μέσω ρίζωσης όρων (stem-matching) και σημασιολογική ομοιότητα. Όταν εντοπιστεί ένα σχετικό στοιχείο, στο φιλτραρισμένο σχήμα προστίθενται επίσης οι αντίστοιχες ιδιότητες και οι σχετικές διαδρομές ενός βήματος, ώστε η πληροφορία που παρέχεται στο μοντέλο να διατηρεί το απαραίτητο τοπολογικό πλαίσιο [8].

Το αποτέλεσμα του φιλτραρίσματος εισάγεται στον Cypher Generator. Η προτροπή περιορίζει ρητά το μοντέλο στη χρήση των ετικετών, των τύπων σχέσεων και των ιδιοτήτων που περιλαμβάνονται στο παρεχόμενο σχήμα και ζητά ως έξοδο αποκλειστικά το παραγόμενο ερώτημα Cypher. Με αυτή τη σχεδίαση διατηρείται σαφής διαχωρισμός μεταξύ του σταδίου κατανόησης της ερώτησης και της μεταγενέστερης ντετερμινιστικής αξιολόγησης του παραγόμενου ερωτήματος.

3.2.2 Αξιολόγηση και Επαναληπτική Διόρθωση

Ο CyVer Evaluator αποτελεί το κεντρικό σημείο ελέγχου της ροής. Για κάθε υποψήφιο ερώτημα ενεργοποιούνται οι ελεγκτές Syntax Validator, Schema Validator και Properties Validator. Ο πρώτος εξετάζει τη συντακτική εγκυρότητα του ερωτήματος, ενώ οι δύο επόμενοι ελέγχουν κατά πόσο οι χρησιμοποιούμενες ετικέτες, σχέσεις, διαδρομές και ιδιότητες συμφωνούν με το πραγματικό σχήμα του γράφου. Σε αντίθεση με προσεγγίσεις στις οποίες η αξιολόγηση πραγματοποιείται από ένα δεύτερο γλωσσικό μοντέλο, το CyVer παρέχει ντετερμινιστικά και δομημένα μεταδεδομένα σφαλμάτων, τα οποία μπορούν να μεταφερθούν απευθείας στον μηχανισμό διόρθωσης [8].

Σε περίπτωση αποτυχίας, ο Cypher Corrector λαμβάνει την αρχική ερώτηση, το προβληματικό ερώτημα, το διαθέσιμο σχήμα και τα σφάλματα που έχουν εντοπιστεί. Το

διορθωμένο ερώτημα αξιολογείται εκ νέου και ο κύκλος επαναλαμβάνεται. Η διαδικασία αυτή αποτελεί τη βάση της παρούσας εργασίας, καθώς κάθε επανάληψη παράγει μια σαφώς ορισμένη μονάδα εμπειρίας: μια συγκεκριμένη προσπάθεια παραγωγής, τα σφάλματα που αυτή προκάλεσε και, εφόσον η διαδικασία συγκλίνει, την τελική επιτυχημένη λύση. Η πληροφορία αυτή αποτελεί τη βάση για τον μηχανισμό μνήμης που παρουσιάζεται στις επόμενες ενότητες.

3.2.3 Ανάκτηση Πλήρους Σχήματος και Ολοκλήρωση της Ροής

Η χρήση φιλτραρισμένου σχήματος μειώνει την ποσότητα άσχετης πληροφορίας που παρέχεται στο γλωσσικό μοντέλο, είναι όμως δυνατό σε ορισμένες ερωτήσεις να παραλειφθεί κάποιο απαραίτητο στοιχείο. Για τον λόγο αυτό, το CyVerACT περιλαμβάνει έναν δεύτερο μηχανισμό ανάκτησης του πλήρους σχήματος [8]. Ο αριθμός των επαναλήψεων ελέγχεται μέσω δύο παραμέτρων της ροής: της `attempts_w_filtered`, η οποία καθορίζει το όριο επαναλήψεων με το φιλτραρισμένο σχήμα, και της `total_attempts`, η οποία καθορίζει το συνολικό όριο επαναλήψεων του μηχανισμού διόρθωσης. Στην πειραματική παραμετροποίηση της παρούσας εργασίας οι δύο τιμές ορίστηκαν σε 5 και 10 αντίστοιχα.

Όταν συμπληρωθεί το πρώτο όριο χωρίς να έχει παραχθεί έγκυρο ερώτημα, ενεργοποιείται ο KG Schema Retriever, ο οποίος αντικαθιστά το φιλτραρισμένο σχήμα με την πλήρη αναπαράσταση του σχήματος της βάσης. Η διαδικασία αξιολόγησης και διόρθωσης συνεχίζεται στη συνέχεια με το πλήρες σχήμα μέχρι την παραγωγή έγκυρου ερωτήματος ή την εξάντληση του συνολικού ορίου.

Όταν ο CyVer Evaluator δεν εντοπίσει κάποιο σφάλμα, το ερώτημα προωθείται στον Cypher Executor και εκτελείται στον γράφο γνώσης. Τα αποτελέσματα της εκτέλεσης επιστρέφονται ως η τελική έξοδος της ροής. Αντίθετα, αν εξαντληθεί το συνολικό όριο χωρίς να παραχθεί έγκυρο ερώτημα, η ροή καταλήγει στην κατάσταση Unavailable Output. Η βασική αυτή διαδικασία διατηρείται στην προτεινόμενη επέκταση, ενώ ο μηχανισμός μνήμης προστίθεται γύρω από συγκεκριμένα σημεία της χωρίς να μεταβάλλει τους ντετερμινιστικούς ελέγχους του CyVer.

3.3 Διερεύνηση Πλαισίων Μνήμης και Επιλογή Προσαρμοσμένης Λύσης

Πριν από την ανάπτυξη του τελικού μηχανισμού μνήμης, πραγματοποιήθηκε διερεύνηση υφιστάμενων πλαισίων, με στόχο να εξεταστεί κατά πόσο μπορούσαν να ενσωματωθούν αποτελεσματικά στην επαναληπτική ροή του CyVerACT. Η διερεύνηση αυτή δεν αποτέλεσε την τελική λύση της εργασίας, αλλά λειτούργησε ως ενδιάμεσο στάδιο σχεδιασμού,

από το οποίο προέκυψαν χρήσιμα συμπεράσματα σχετικά με την αναπαράσταση, την αποθήκευση και την ανάκτηση της εμπειρίας ενός πράκτορα Text-to-Cypher.

Η πρακτική διερεύνηση επικεντρώθηκε κυρίως στα Cognee, Zep και Graphiti. Από αυτά, το Cognee αποτέλεσε το βασικότερο αντικείμενο πειραματισμού, καθώς χρησιμοποιήθηκε εκτενέστερα και προσαρμόστηκε στις ανάγκες της εργασίας. Αντίθετα, τα Zep και Graphiti εξετάστηκαν κυρίως ως εναλλακτικές λύσεις για τη διαχείριση επεισοδιακής μνήμης και συνομιλιακού ιστορικού. Οι βασικές αρχές των αντίστοιχων προσεγγίσεων μνήμης παρουσιάστηκαν στο προηγούμενο κεφάλαιο.

3.3.1 Πειραματισμός με το Cognee

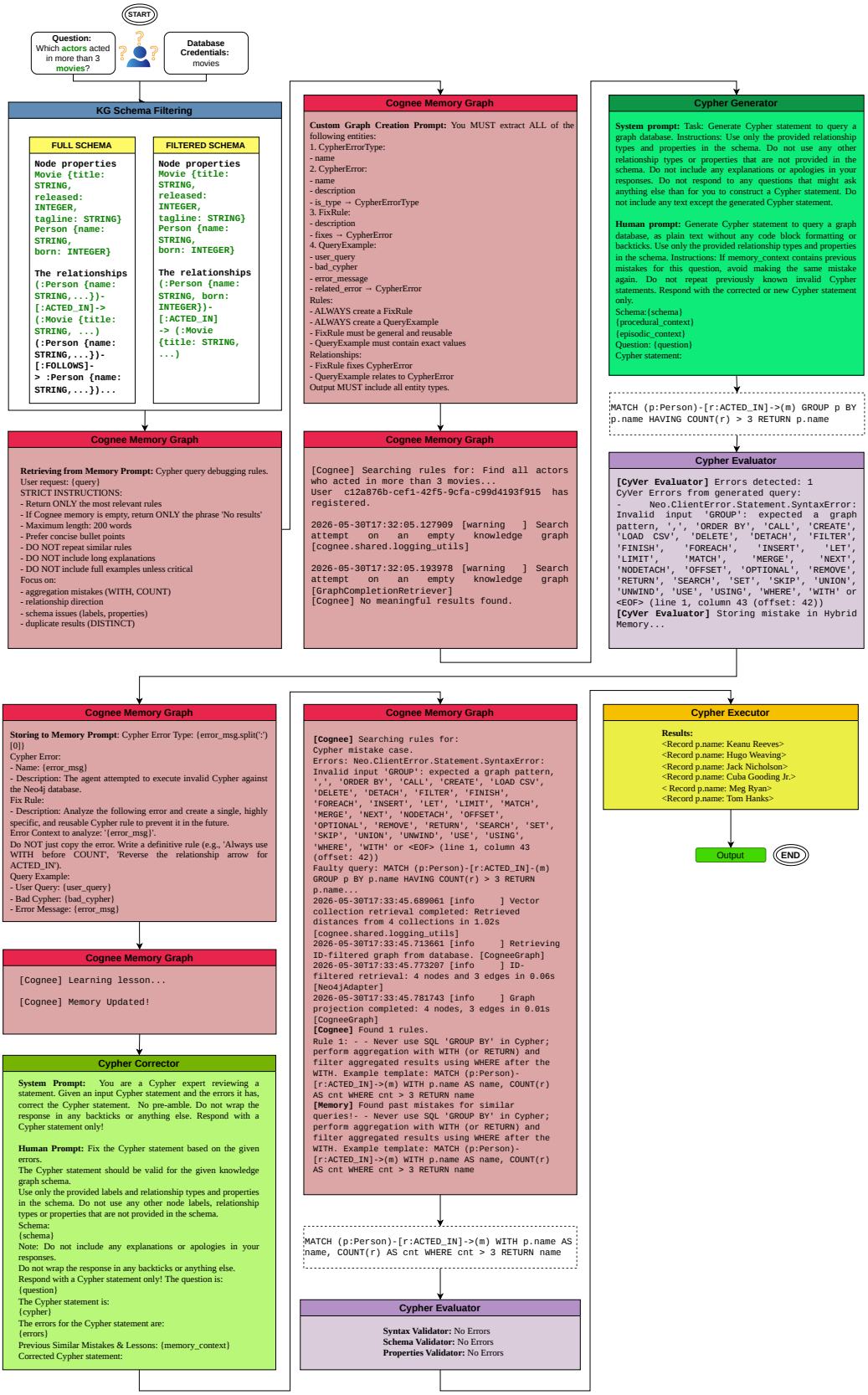
Η πρώτη ουσιαστική προσπάθεια ενσωμάτωσης μνήμης βασίστηκε στο Cognee [37]. Στο πλαίσιο της παρούσας εργασίας, το Cognee χρησιμοποιήθηκε με σκοπό να μετατραπούν προηγούμενες αποτυχημένες προσπάθειες παραγωγής Cypher σε αξιοποιήσιμη γνώση για μεταγενέστερες εκτελέσεις.

Πιο συγκεκριμένα, κάθε περίπτωση αποτυχίας περιγραφόταν από τρία βασικά στοιχεία: την ερώτηση φυσικής γλώσσας, το παραγόμενο ερώτημα Cypher και το αντίστοιχο μήνυμα σφάλματος. Η πληροφορία αυτή εισαγόταν στο σύστημα και ακολουθούσε τη διαδοχή των λειτουργιών `add()`, `cognify()` και `search()`. Μέσω της διαδικασίας `cognify()`, πραγματοποιούνταν κατάτμηση της εισόδου, εξαγωγή δομημένης πληροφορίας και κατασκευή γράφου γνώσης, ο οποίος στη συνέχεια μπορούσε να χρησιμοποιηθεί για την αναζήτηση σχετικής εμπειρίας [35, 36].

Για την καλύτερη προσαρμογή του πλαισίου στις ανάγκες της εργασίας, σχεδιάστηκε ένα προσαρμοσμένο μοντέλο γράφου, στο οποίο περιλαμβάνονταν οντότητες όπως `CypherErrorType`, `CypherError`, `FixRule` και `QueryExample`. Στόχος του μοντέλου αυτού ήταν να αναπαραστήσει όχι απλώς ακατέργαστο κείμενο, αλλά δομικά στοιχεία που σχετίζονται με τη διαδικασία διόρθωσης Cypher. Με τον τρόπο αυτό επιχειρήθηκε η μετατροπή συγκεκριμένων αποτυχημένων περιπτώσεων σε γενικότερη διαδικαστική γνώση, η οποία θα μπορούσε να ανακτηθεί κατά τη διόρθωση επόμενων ερωτημάτων.

Η ανάκτηση της μνήμης πραγματοποιούνταν πριν από την παραγωγή ή τη διόρθωση ενός νέου ερωτήματος. Τα αποτελέσματα της αναζήτησης εισάγονταν ως πρόσθετο πλαίσιο στην προτροπή του γλωσσικού μοντέλου, με στόχο να λειτουργήσουν ως κανόνες ή παραδείγματα καθοδήγησης. Οι προκαταρκτικές δοκιμές έδειξαν ότι η προσέγγιση αυτή μπορούσε, σε ορισμένες περιπτώσεις, να ανακτήσει χρήσιμη καθοδήγηση από προηγούμενα σφάλματα και να επηρεάσει την επόμενη παραγωγή ερωτήματος Cypher.

Η πειραματική ενσωμάτωση του Cognee στη ροή παραγωγής και διόρθωσης παρουσιάζεται στο Σχήμα 3.2. Οι πλήρεις προτροπές που χρησιμοποιήθηκαν για την πειραματική κατασκευή, αποθήκευση και ανάκτηση της μνήμης με το Cognee παρατίθενται στο Παράρτημα Α.4.



Σχήμα 3.2: Πειραματική ενσωμάτωση του Cognee με προσαρμοσμένο σχήμα στη ροή παραγωγής και διόρθωσης ερωτημάτων Cypher. Η βασική πρακτορική ροή προσαρμόστηκε από [8].

Ωστόσο, η πειραματική διερεύνηση ανέδειξε έναν σημαντικό περιορισμό για τους στόχους της παρούσας εργασίας. Στη συγκεκριμένη διαμόρφωση, το Cognee αξιοποιούσε γλωσσικό μοντέλο τόσο κατά την εξαγωγή οντοτήτων και σχέσεων στη διαδικασία `cognify()` όσο και κατά τη σύνθεση των αποτελεσμάτων αναζήτησης. Κατά συνέπεια, δεν ήταν πάντοτε εύκολο να διαπιστωθεί αν η καθοδήγηση που επέστρεφε ο μηχανισμός προερχόταν αποκλειστικά από αποθηκευμένη εμπειρία ή αν είχε εν μέρει παραχθεί εκ νέου από το γλωσσικό μοντέλο. Η συμπεριφορά αυτή δυσχέραινε την απομόνωση της επίδρασης της ίδιας της μνήμης και, επομένως, την ελεγχόμενη πειραματική αξιολόγηση διαφορετικών στρατηγικών ανάκτησης.

Επιπλέον, οι διαδικασίες κατασκευής του γράφου και σύνθεσης της ανακτημένης πληροφορίας εισήγαγαν πρόσθετες κλήσεις σε γλωσσικό μοντέλο και υπολογιστική επιβάρυνση σε μια ήδη επαναληπτική ροή. Οι παρατηρήσεις αυτές δεν αναιρούν τη χρησιμότητα του Cognee ως μηχανισμού γραφοκεντρικής μνήμης, αλλά ανέδειξαν την ανάγκη για μεγαλύτερο έλεγχο της αποθήκευσης, της ανάκτησης και της πληροφορίας που τελικά παρέχεται στους πράκτορες. Η συνολική αποτίμηση της διερεύνησης παρουσιάζεται στον Πίνακα 3.1.

Πίνακας 3.1: Αποτίμηση της διερεύνησης του Cognee και επίδραση στον σχεδιασμό της τελικής λύσης.

Κριτήριο	Θετική παρατήρηση	Περιορισμός
Αναπαράσταση μνήμης	Δυνατότητα οργάνωσης της εμπειρίας σε γράφο και σύνδεσης σφαλμάτων, κανόνων και παραδειγμάτων.	Στη συγκεκριμένη ενσωμάτωση, μέρος της τελικής δομής της γνώσης εξαρτάται από εσωτερικές διαδικασίες του πλαισίου.
Παραμετροποίηση	Υποστηρίζει προσαρμοσμένο μοντέλο γράφου και προσαρμοσμένες προτροπές ανάκτησης.	Δεν παρέχεται άμεσος έλεγχος όλων των ενδιάμεσων σταδίων επεξεργασίας της πληροφορίας.
Ανάκτηση γνώσης	Μπορεί να συνδυάσει γραφική και σημασιολογική αναζήτηση και να ανακτήσει σχετικές προηγούμενες περιπτώσεις για την καθοδήγηση της διόρθωσης.	Στη συγκεκριμένη διαμόρφωση, μέρος της επεξεργασίας των αποτελεσμάτων πραγματοποιείται από γλωσσικό μοντέλο, με αποτέλεσμα να δυσχεραίνεται ο αυστηρός διαχωρισμός μεταξύ πραγματικής ανάκτησης και νέας παραγωγής.
Γενίκευση εμπειρίας	Επιτρέπει τη μετατροπή επιμέρους σφαλμάτων σε επαναχρησιμοποιήσιμη διαδικαστική γνώση.	Η παραγόμενη καθοδήγηση μπορεί να είναι υπερβολικά γενική ή να μη συνδέεται επαρκώς με τη δομή του τρέχοντος ερωτήματος.
Υπολογιστική επιβάρυνση	Παρέχει έτοιμη υποδομή και περιορίζει την ανάγκη ανάπτυξης όλων των μηχανισμών εξ αρχής.	Οι διαδικασίες `cognify()` και αναζήτησης περιλαμβάνουν πρόσθετη επεξεργασία και κλήσεις σε γλωσσικό μοντέλο, αυξάνοντας την καθυστέρηση μιας ήδη επαναληπτικής ροής.
Πειραματικός έλεγχος	Διευκολύνει την αρχική διερεύνηση γραφοκεντρικών μηχανισμών μνήμης.	Η εξάρτηση από εσωτερικούς μηχανισμούς του πλαισίου δυσχεραίνει την απομόνωση και την ελεγχόμενη αξιολόγηση επιμέρους στρατηγικών μνήμης.

3.3.2 Διερεύνηση Zep και Graphiti

Εκτός από το Cognee, εξετάστηκαν εναλλακτικές προσεγγίσεις για τη διαχείριση επεισοδιακής μνήμης. Αρχικά διερευνήθηκε το Zep, το οποίο βασίζεται σε χρονικό γράφο γνώσης για τη διατήρηση και ανάκτηση μακροπρόθεσμης πληροφορίας πρακτόρων [26]. Στην παρούσα υλοποίηση εξετάστηκε κυρίως ως μηχανισμός αποθήκευσης και αξιοποίησης του ιστορικού αλληλεπίδρασης.

Κατά την πρακτική ενσωμάτωση, ωστόσο, προέκυψαν ασυμβατότητες εξαρτήσεων μεταξύ του Python SDK του Zep και των βιβλιοθηκών που χρησιμοποιούνταν από το Cognee. Ως ενδιάμεση λύση, η επικοινωνία με το Zep πραγματοποιήθηκε μέσω του Zep

Cloud API αντί της άμεσης ενσωμάτωσης του αντίστοιχου SDK. Η λύση αυτή επέτρεψε τη συνέχιση της διερεύνησης, εισήγαγε όμως εξάρτηση από διαχειριζόμενη εξωτερική υπηρεσία και περιόριζε τον άμεσο έλεγχο του μηχανισμού αποθήκευσης στο πλαίσιο του πειράματος.

Για τον λόγο αυτό εξετάστηκε στη συνέχεια το Graphiti, το οποίο ακολουθεί γραφοκεντρική προσέγγιση για την οργάνωση επεισοδιακής πληροφορίας και αποτελεί τη βάση της προσέγγισης χρονικών γράφων μνήμης του Zep [34, 26]. Στην πειραματική ενσωμάτωση χρησιμοποιήθηκε απευθείας με Neo4j, επιτρέποντας την αποθήκευση ολοκληρωμένων αλληλεπιδράσεων ως επεισοδίων χωρίς την προηγούμενη εξάρτηση από το Zep Cloud API. Κάθε επεισόδιο μπορούσε να περιλαμβάνει την αρχική ερώτηση, το τελικό ερώτημα Cypher, τα αποτελέσματα της εκτέλεσης, τα σφάλματα και το πλήθος των προσπαθειών που απαιτήθηκαν.

Η συγκεκριμένη λειτουργικότητα ήταν κατάλληλη κυρίως για τη διατήρηση συνομιλιακής συνέχειας και την επίλυση αναφορών σε προηγούμενα αντικείμενα συζήτησης. Ο κύριος στόχος της παρούσας εργασίας, ωστόσο, ήταν διαφορετικός: η μεταφορά εμπειρίας που προκύπτει από την παραγωγή και τη διόρθωση ερωτημάτων Cypher μεταξύ ανεξάρτητων εκτελέσεων. Για τον λόγο αυτό, η γενική επεισοδιακή μνήμη των παραπάνω πλαισίων δεν υιοθετήθηκε ως η τελική λύση, αλλά συνέβαλε στον καθορισμό των απαιτήσεων του προσαρμοσμένου μηχανισμού που αναπτύχθηκε στη συνέχεια.

3.3.3 Επιλογή Προσαρμοσμένου Μηχανισμού Μνήμης

Τα παραπάνω πειράματα κατέδειξαν ότι η αξιοποίηση προηγούμενης εμπειρίας μπορούσε να είναι χρήσιμη σε ένα σύστημα Text-to-Cypher, ανέδειξαν όμως ταυτόχρονα την ανάγκη για μεγαλύτερο έλεγχο στη δομή, στα κριτήρια ανάκτησης και στον τρόπο με τον οποίο η πληροφορία μνήμης αξιοποιείται από τον πράκτορα. Ειδικότερα, κρίθηκε απαραίτητο η τελική λύση να ικανοποιεί τις ακόλουθες απαιτήσεις:

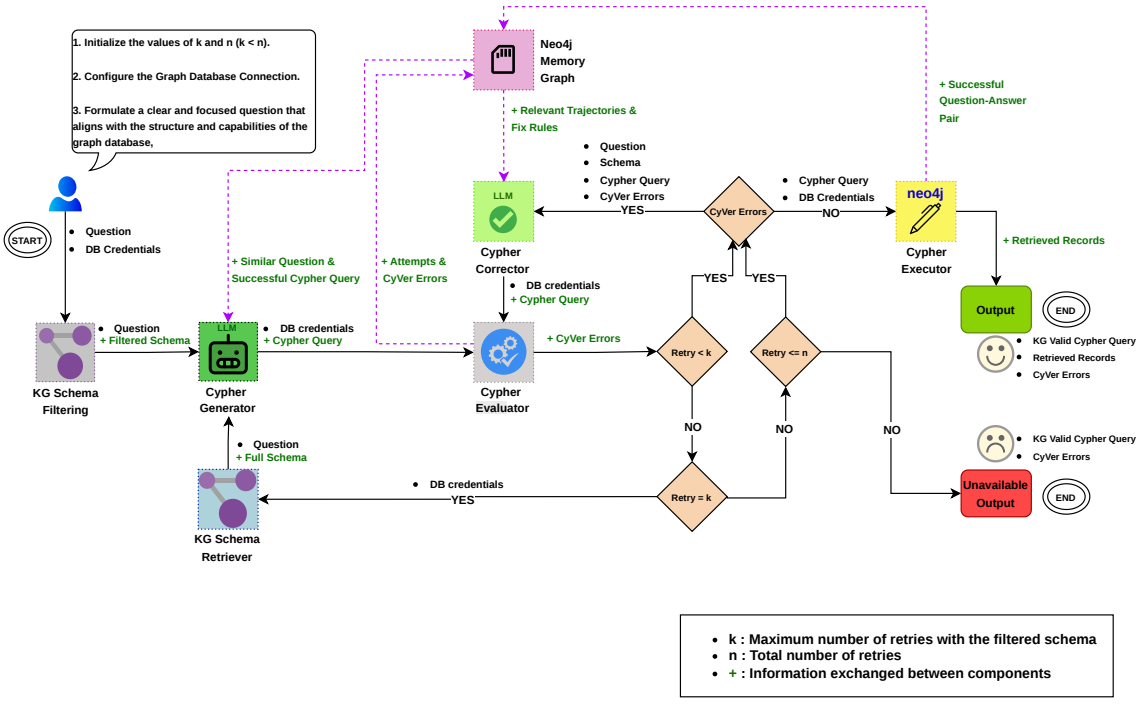
- να επιτρέπει ρητό και πλήρως ελεγχόμενο ορισμό του σχήματος της μνήμης,
- να αναπαριστά ρητά τις σχέσεις μεταξύ ερωτήσεων, προσπαθειών, σφαλμάτων και τελικών λύσεων, ώστε η εμπειρία να διατηρείται ως συνδεδεμένη πορεία επίλυσης και όχι ως σύνολο ανεξάρτητων εγγραφών,
- να αποθηκεύει ντετερμινιστικά τα παραγόμενα δεδομένα χωρίς εξάρτηση από εσωτερική παραγωγή πρόσθετης γνώσης από εξωτερικό πλαίσιο,
- να διαχωρίζει με σαφήνεια τη διαδικασία αποθήκευσης, τη διαδικασία ανάκτησης και τη νέα παραγωγή από το γλωσσικό μοντέλο,
- να επιτρέπει τη διαμόρφωση διαφορετικών συνδυασμών των μηχανισμών μνήμης που εξετάζονται πειραματικά,

- να αξιοποιεί εγγενώς τις δυνατότητες αναζήτησης και διανυσματικής ευρετηρίασης του Neo4j.

Με βάση τα παραπάνω, επιλέχθηκε τελικά η ανάπτυξη ενός προσαρμοσμένου μηχανισμού μνήμης πάνω σε Neo4j. Η επιλογή υβριδικής αναπαράστασης συνδέεται άμεσα με τη φύση της εμπειρίας που παράγει το CyVerACT: μία αρχική ερώτηση μπορεί να οδηγήσει σε διαδοχικές προσπάθειες, κάθε προσπάθεια σε συγκεκριμένα σφάλματα και, τελικά, σε μία επιτυχημένη λύση. Η αναπαράσταση των στοιχείων αυτών ως συνδεδεμένων κόμβων επιτρέπει τη διατήρηση της πορείας επίλυσης και της προέλευσης κάθε αποθηκευμένης πληροφορίας, αντί η εμπειρία να καταγράφεται ως σύνολο ανεξάρτητων εγγραφών. Παράλληλα, το Neo4j επιτρέπει τον συνδυασμό της γραφικής αυτής δομής με διανυσματικά ευρετήρια, ώστε η σημασιολογική αναζήτηση να χρησιμοποιείται μαζί με τη διάσχιση των ρητών σχέσεων του γράφου.

Η επιλογή αυτή επέτρεψε την άμεση και ελεγχόμενη αποθήκευση της εμπειρίας που παράγεται από τη ροή του CyVerACT, καθώς και την υλοποίηση διαφορετικών στρατηγικών ανάκτησης χωρίς εξάρτηση από έναν εξωτερικό μηχανισμό οργάνωσης γνώσης. Επιπλέον, κατέστη δυνατή η υλοποίηση πολλαπλών πειραματικών διαμορφώσεων, στις οποίες οι βασικοί μηχανισμοί μνήμης συνδυάζονται με διαφορετικούς τρόπους, ώστε να αξιολογηθεί η επίδρασή τους.

Η συνολική αρχιτεκτονική της προτεινόμενης επέκτασης, καθώς και τα σημεία εγγραφής και ανάκτησης πληροφορίας από τον γράφο μνήμης, παρουσιάζονται στο Σχήμα 3.3.



Σχήμα 3.3: Προτεινόμενη επέκταση της πρακτορικής ροής CyVerACT με μηχανισμό επίμονης μνήμης. Η βασική ροή βασίζεται στην αρχιτεκτονική του CyVerACT [8].

3.4 Σχεδιασμός του Γράφου Μνήμης

Ο τελικός μηχανισμός μακροπρόθεσμης μνήμης υλοποιήθηκε ως γράφος ιδιοτήτων στο Neo4j, με προκαθορισμένο σχήμα το οποίο ακολουθεί άμεσα τη διαδικασία επίλυσης ενός ερωτήματος από το CyVerACT. Σε αντίθεση με την αρχική διερεύνηση μέσω του Cognee, όπου μέρος της δομής της αποθηκευμένης γνώσης προέκυπτε μέσω επεξεργασίας από γλωσσικό μοντέλο, στην τελική προσέγγιση οι τύποι των κόμβων, οι σχέσεις και τα αποθηκευόμενα πεδία ορίζονται ρητά από την εφαρμογή.

Βασική σχεδιαστική επιλογή αποτέλεσε η διατήρηση όχι μόνο της τελικής επιτυχημένης λύσης, αλλά ολόκληρης της πορείας που ακολουθήθηκε μέχρι την παραγωγή της. Μια ερώτηση μπορεί να οδηγήσει σε πολλαπλές διαδοχικές προσπάθειες παραγωγής ή διόρθωσης Cypher. Κάθε αποτυχημένη προσπάθεια μπορεί με τη σειρά της να συνδεθεί με ένα ή περισσότερα σφάλματα που εντοπίστηκαν από το CyVer, ενώ η τελική επιτυχής εκτέλεση συνδέεται με την αντίστοιχη λύση. Με αυτόν τον τρόπο, η μνήμη διατηρεί τόσο το τελικό αποτέλεσμα όσο και την εμπειρία που αποκτήθηκε κατά τη διαδικασία επίλυσής του.

3.4.1 Κόμβοι και Σχέσεις του Γράφου Μνήμης

Το μοντέλο μνήμης περιλαμβάνει έξι βασικούς τύπους κόμβων: `Question`, `Attempt`, `CypherError`, `ErrorType`, `Answer` και `FixRule`. Η σημασία των κόμβων και τα βασικά αποθηκευόμενα πεδία παρουσιάζονται στον Πίνακα 3.2.

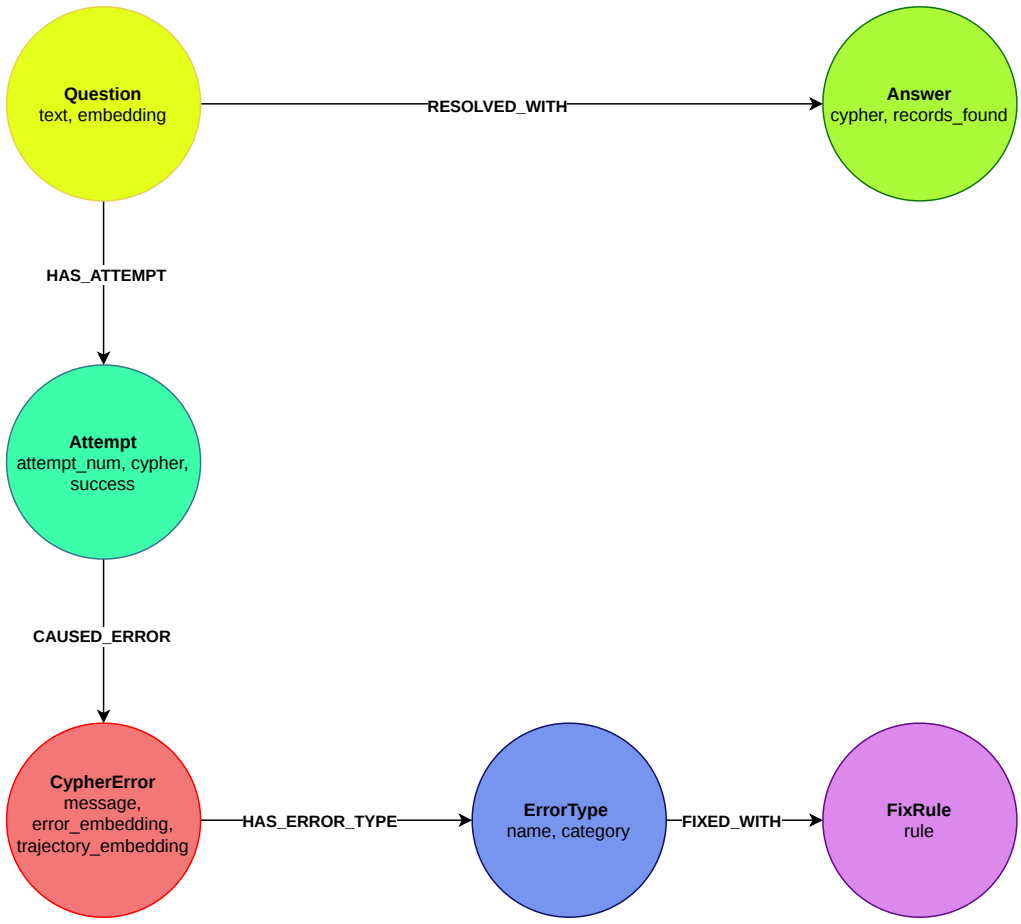
Πίνακας 3.2: Βασικοί τύποι κόμβων του γράφου μνήμης.

Τύπος κόμβου	**Βασικά πεδία**	**Ρόλος**
Question	**text**, **embedding**	Αναπαριστά την αρχική ερώτηση φυσικής γλώσσας. Η διανυσματική της αναπαράσταση χρησιμοποιείται για την ανάκτηση παρόμοιων επιτυχημένων ερωτημάτων.
Attempt	**attempt_num**, **cypher**, **success**	Αναπαριστά μία συγκεκριμένη προσπάθεια παραγωγής ή διόρθωσης ερωτήματος Cypher.
CypherError	**message**, **error_embedding**, **trajectory_embedding**	Αναπαριστά ένα συγκεκριμένο σφάλμα που προέκυψε κατά την αξιολόγηση μιας προσπάθειας.
ErrorType	**name**, **category**	Αναπαριστά έναν συγκεκριμένο τύπο σφάλματος ή προειδοποίησης. Το πεδίο `category` δηλώνει τον ελεγκτή προέλευσης.
Answer	**cypher**, **records_found**	Αναπαριστά το τελικό έγκυρο ερώτημα Cypher που εκτελέστηκε επιτυχώς στη βάση στόχο.
FixRule	**rule**	Περιέχει έναν γενικό, επαναχρησιμοποιήσιμο κανόνα διόρθωσης για συγκεκριμένο τύπο σφάλματος.

Οι κόμβοι συνδέονται μέσω σχέσεων που αναπαριστούν τη διαδρομή επίλυσης. Η σχέση `HAS_ATTEMPT` συνδέει μια `Question` με τις προσπάθειες που πραγματοποιήθηκαν για την επίλυσή της. Κάθε `Attempt` συνδέεται μέσω της `CAUSED_ERROR` με τους αντίστοιχους κόμβους `CypherError` που καταγράφηκαν για αυτήν, ενώ κάθε συγκεκριμένο σφάλμα συνδέεται με τη γενική κατηγορία του μέσω της `HAS_ERROR_TYPE`. Όταν η διαδικασία ολοκληρωθεί επιτυχώς, η αρχική `Question` συνδέεται μέσω της `RESOLVED_WITH` με την τελική `Answer`.

Τέλος, οι κόμβοι `ErrorType` συνδέονται με τους αντίστοιχους γενικούς κανόνες διόρθωσης μέσω της σχέσης `FIXED_WITH`. Η σύνδεση αυτή επιτρέπει την ανάκτηση διαδικαστικής καθοδήγησης με βάση τον ακριβή τύπο ενός σφάλματος, χωρίς να απαιτείται σημασιολογική αναζήτηση.

Η συνολική δομή του προσαρμοσμένου γράφου μνήμης και οι σχέσεις μεταξύ των επιμέρους τύπων κόμβων παρουσιάζονται στο Σχήμα 3.4.



Σχήμα 3.4: Σχήμα του προσαρμοσμένου γράφου μνήμης που χρησιμοποιείται για την αποθήκευση ερωτήσεων, προσπαθειών, σφαλμάτων, κανόνων διόρθωσης και επιτυχημένων λύσεων.

3.4.2 Κατηγορίες Σφαλμάτων και Κανόνες Διόρθωσης

Οι κόμβοι `ErrorType` δημιουργούνται εκ των προτέρων με βάση τις κατηγορίες σφαλμάτων και προειδοποιήσεων που μπορούν να προκύψουν κατά την αξιολόγηση από το CyVer. Περιλαμβάνονται, μεταξύ άλλων, σφάλματα σύνταξης και εκτέλεσης όπως τα `SyntaxError`, `ExecutionFailed` και `ParameterNotProvided`, καθώς και σφάλματα που σχετίζονται με το σχήμα και τις διαδρομές του γράφου, όπως τα `UnknownLabelWarning`, `UnknownRelationshipTypeWarning`, `UnknownPathWarning` καθώς και `WrongDirection PathWarning`. Επιπλέον, περιλαμβάνονται κατηγορίες που αφορούν λανθασμένη πρόσβαση σε ιδιότητες, όπως οι `UnknownPropertyKeyWarning` και `InvalidPropertyAccess ByLabelWarning` [8].

Σε κάθε κατηγορία αντιστοιχίζεται ένας προκαθορισμένος κόμβος `FixRule`, ο οποίος περιέχει μία γενική οδηγία για την αποφυγή ή τη διόρθωση του συγκεκριμένου τύπου σφάλματος. Για παράδειγμα, στην περίπτωση ενός `UnknownRelationshipTypeWarning` ο αντίστοιχος κανόνας υποδεικνύει ότι πρέπει να χρησιμοποιούνται αποκλειστικά τύποι σχέσεων που ορίζονται ρητά στο παρεχόμενο σχήμα. Αντίστοιχα, για ένα `Wrong`

DirectionPathWarning η καθοδήγηση επικεντρώνεται στη σωστή κατεύθυνση της σχέσης, ενώ για ένα InvalidPropertyAccessByLabelWarning απαιτείται η αντιστοίχιση κάθε ιδιότητας με τον σωστό τύπο κόμβου ή σχέσης.

Οι κανόνες αυτοί δεν προκύπτουν δυναμικά κατά τη διάρκεια της εκτέλεσης. Η επιλογή προκαθορισμένων κανόνων FixRule επιτρέπει την ανάκτησή τους με ακριβή αντιστοίχιση της κατηγορίας σφάλματος και διατηρεί τη συγκεκριμένη μορφή διαδικαστικής γνώσης ανεξάρτητη από τη στοχαστική συμπεριφορά του γλωσσικού μοντέλου.

3.4.3 Διανυσματικές Αναπαραστάσεις της Μνήμης

Παράλληλα με τη γραφική δομή, χρησιμοποιούνται διανυσματικές αναπαραστάσεις για την ανάκτηση προηγούμενων περιπτώσεων με παρόμοιο σημασιολογικό ή δομικό περιεχόμενο. Ο συνδυασμός γραφικής αναπαράστασης και διανυσματικής αναζήτησης επιτρέπει τη ρητή διατήρηση των σχέσεων μεταξύ των στοιχείων της εμπειρίας και, ταυτόχρονα, την ανάκτηση σημασιολογικά συναφών περιπτώσεων [12]. Για την παραγωγή των διανυσμάτων χρησιμοποιείται το μοντέλο all-MiniLM-L6-v2, το οποίο στην παρούσα υλοποίηση παράγει διανύσματα διάστασης 384 [39].

Στους κόμβους Question αποθηκεύεται η διανυσματική αναπαράσταση του κειμένου της ερώτησης. Σκοπός της είναι η αναζήτηση προηγούμενων ερωτήσεων με παρόμοια σημασιολογική πρόθεση και η ανάκτηση των αντίστοιχων επιτυχημένων ερωτημάτων Cypher με τα οποία είχαν επιλυθεί.

Για κάθε CypherError αποθηκεύονται δύο διαφορετικές διανυσματικές αναπαραστάσεις. Το CyVer επιστρέφει για κάθε εντοπισμένο σφάλμα έναν κωδικό (code) και μία περιγραφή (description). Κατά την εγγραφή στη μνήμη, από τον κωδικό διατηρείται η τελική κατηγορία του σφάλματος, όπως SyntaxError ή WrongDirectionPath Warning, ενώ η περιγραφή αποθηκεύεται αυτούσια ως το μήνυμα (message) του κόμβου CypherError. Στη συνέχεια του κεφαλαίου, ο όρος 'μήνυμα σφάλματος' αναφέρεται σε αυτή την περιγραφή.

Η πρώτη αναπαράσταση, error_embedding, παράγεται από τον συνδυασμό της κατηγορίας και του μηνύματος του σφάλματος και αποσκοπεί στην αναγνώριση παρόμοιων αποτυχιών. Η δεύτερη, trajectory_embedding, παράγεται από τον συνδυασμό της αρχικής ερώτησης, του αποτυχημένου ερωτήματος Cypher και του αντίστοιχου μηνύματος σφάλματος. Με αυτόν τον τρόπο, η δεύτερη αναπαράσταση ενσωματώνει και το πλαίσιο της συγκεκριμένης αποτυχημένης προσπάθειας, πέρα από το ίδιο το σφάλμα.

Η διάκριση μεταξύ των δύο αναπαραστάσεων είναι σημαντική, καθώς το ίδιο μήνυμα σφάλματος μπορεί να εμφανιστεί σε διαφορετικά προβλήματα, ενώ δύο δομικά παρόμοιες αποτυχημένες προσπάθειες μπορούν να αφορούν διαφορετικές ετικέτες ή ιδιότητες. Η διατήρηση χωριστών διανυσμάτων επιτρέπει επομένως τον συνδυασμό της ομοιότητας του σφάλματος με την ομοιότητα της συνολικής τροχιάς. Οι τρεις διανυσμα-

τικές αναπαραστάσεις συνοψίζονται στον Πίνακα 3.3.

Πίνακας 3.3: Διανυσματικές αναπαραστάσεις του μηχανισμού μνήμης.

Αναπαράσταση	**Περιεχόμενο**	**Σκοπός**
`Question.embedding`	Κείμενο της ερώτησης φυσικής γλώσσας.	Εντοπισμός παρόμοιων προηγούμενων ερωτήσεων και των αντίστοιχων επιτυχημένων λύσεων.
`error_embedding`	Κατηγορία σφάλματος, προερχόμενη από το code του CyVer, και μήνυμα σφάλματος (`description`).	Εντοπισμός προηγούμενων προσπαθειών με παρόμοιο σφάλμα.
`trajectory_embedding`	Αρχική ερώτηση, αποτυχημένο ερώτημα Cypher και μήνυμα σφάλματος (`description`).	Αποτύπωση του συνολικού πλαισίου και της δομής μιας αποτυχημένης προσπάθειας.

Για τις παραπάνω αναπαραστάσεις δημιουργούνται τρία αντίστοιχα διανυσματικά ευρετήρια στο Neo4j: το `question_embeddings`, το `error_vector_embeddings` και το `trajectory_vector_embeddings`, με ομοιότητα συνημιτόνου. Στον τελικό μηχανισμό ανάκτησης, το πρώτο χρησιμοποιείται για την αναζήτηση παρόμοιων προηγούμενων ερωτήσεων, ενώ το `error_vector_embeddings` χρησιμοποιείται για την αρχική επιλογή υποψήφιων προηγούμενων σφαλμάτων. Η ομοιότητα της τροχιάς υπολογίζεται στη συνέχεια απευθείας πάνω στο `trajectory_embedding` των υποψήφιων περιπτώσεων, όπως περιγράφεται στην Ενότητα 3.6.

3.5 Λειτουργία Αποθήκευσης και Οργάνωσης της Μνήμης

Η εγγραφή στη μνήμη πραγματοποιείται σε συγκεκριμένα σημεία της πρακτορικής ροής και διαχωρίζεται από την ίδια τη διαδικασία παραγωγής Cypher. Για τον σκοπό αυτό χρησιμοποιείται ξεχωριστή σύνδεση προς τη βάση μνήμης, ανεξάρτητη από εκείνη που χρησιμοποιείται για την εκτέλεση του παραγόμενου ερωτήματος.

Η αποθήκευση οργανώνεται σε δύο κύρια στάδια. Το πρώτο πραγματοποιείται κατά την αξιολόγηση κάθε προσπάθειας και αφορά την καταγραφή της τροχιάς επίλυσης. Το δεύτερο πραγματοποιείται όταν ένα έγκυρο ερώτημα εκτελεστεί επιτυχώς και αφορά την αποθήκευση της τελικής λύσης.

3.5.1 Καταγραφή Προσπαθειών και Σφαλμάτων

Μετά από κάθε παραγωγή ή διόρθωση Cypher, ο CyVer Evaluator ενεργοποιεί τους ελεγκτές σύνταξης, σχήματος και ιδιοτήτων. Εκτός από τα μηνύματα που χρησιμοποιούνται

για τη δρομολόγηση της τρέχουσας εκτέλεσης, διατηρούνται και τα αντίστοιχα δομημένα μεταδεδομένα των σφαλμάτων.

Για κάθε προσπάθεια δημιουργείται ένας κόμβος `Attempt`, ο οποίος αποθηκεύει το ερώτημα Cypher, τον αύξοντα αριθμό της προσπάθειας και την ένδειξη επιτυχίας. Ο αριθμός της προσπάθειας προκύπτει από το ιστορικό των ερωτημάτων Cypher που έχει συσσωρευτεί στην κατάσταση της τρέχουσας εκτέλεσης.

Για κάθε σφάλμα που αντιστοιχεί στην προσπάθεια δημιουργείται ξεχωριστός κόμβος `CypherError`. Από τα μεταδεδομένα του CyVer απομονώνονται η ακριβής κατηγορία του σφάλματος και η περιγραφή του. Ταυτόχρονα παράγονται οι δύο διανυσματικές αναπαραστάσεις που παρουσιάστηκαν προηγουμένως: μία για το ίδιο το σφάλμα και μία για τη συνολική τροχιά. Στη συνέχεια, το συγκεκριμένο `CypherError` συνδέεται με τον αντίστοιχο προκαθορισμένο `ErrorType`.

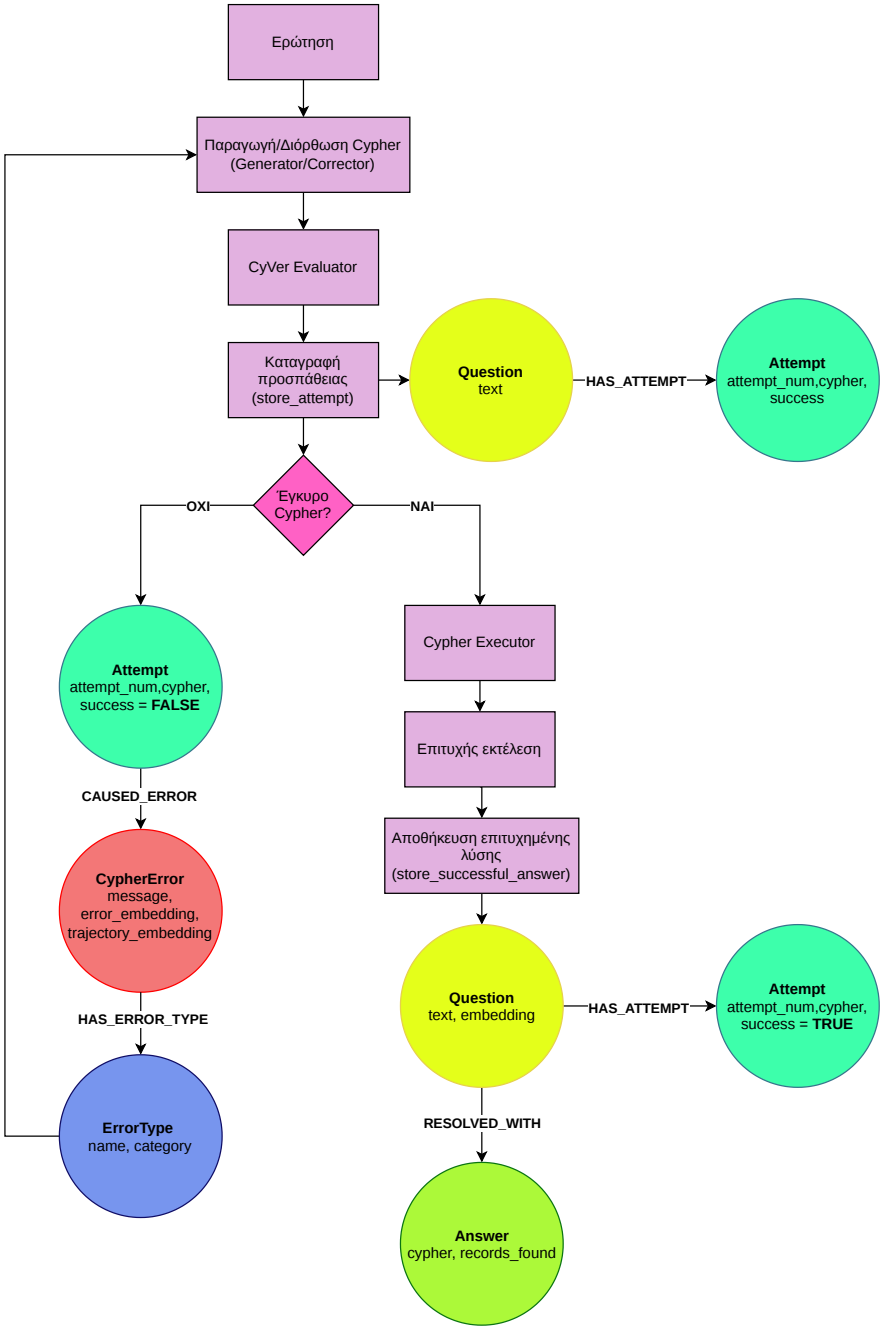
Η επιλογή αποθήκευσης κάθε προσπάθειας ξεχωριστά επιτρέπει στη μνήμη να διατηρεί την πορεία επίλυσης του πράκτορα. Εάν, για παράδειγμα, τρεις διαδοχικές διορθώσεις οδηγήσουν στο ίδιο ή σε διαφορετικά σφάλματα πριν από την τελική επιτυχία, οι τρεις προσπάθειες παραμένουν διαθέσιμες ως διακριτά σημεία της πορείας επίλυσης.

3.5.2 Αποθήκευση Επιτυχημένων Λύσεων

Όταν ένα υποψήφιο ερώτημα δεν περιλαμβάνει σφάλματα σύμφωνα με τον CyVer Evaluator, προωθείται στον Cypher Executor. Μετά την επιτυχή εκτέλεσή του στη βάση στόχο, δημιουργείται ή ενημερώνεται ο αντίστοιχος κόμβος `Question` και αποθηκεύεται η διανυσματική αναπαράσταση της ερώτησης.

Το τελικό ερώτημα αποθηκεύεται σε κόμβο `Answer`, μαζί με το πλήθος των αποτελεσμάτων που καταγράφηκαν κατά την εκτέλεση. Η σχέση `RESOLVED_WITH` συνδέει στη συνέχεια την ερώτηση με την επιτυχημένη λύση. Η σύνδεση αυτή υποστηρίζει άμεσα την ανάκτηση μνήμης, καθώς επιτρέπει σε μια μελλοντική ερώτηση να εντοπίσει όχι μόνο μια παρόμοια προηγούμενη πρόθεση αλλά και το ερώτημα που είχε οδηγήσει σε επιτυχημένη εκτέλεση.

Τα δύο στάδια εγγραφής της εμπειρίας στη μακροπρόθεσμη μνήμη και η σύνδεσή τους με την πρακτορική ροή συνοψίζονται στο Σχήμα 3.5.



Σχήμα 3.5: Ροή εγγραφής στη μακροπρόθεσμη μνήμη κατά την καταγραφή προσπαθειών και σφαλμάτων και την αποθήκευση επιτυχημένων λύσεων. Οι κυκλικοί κόμβοι αντιστοιχούν στις οντότητες που εγγράφονται στον γράφο μνήμης.

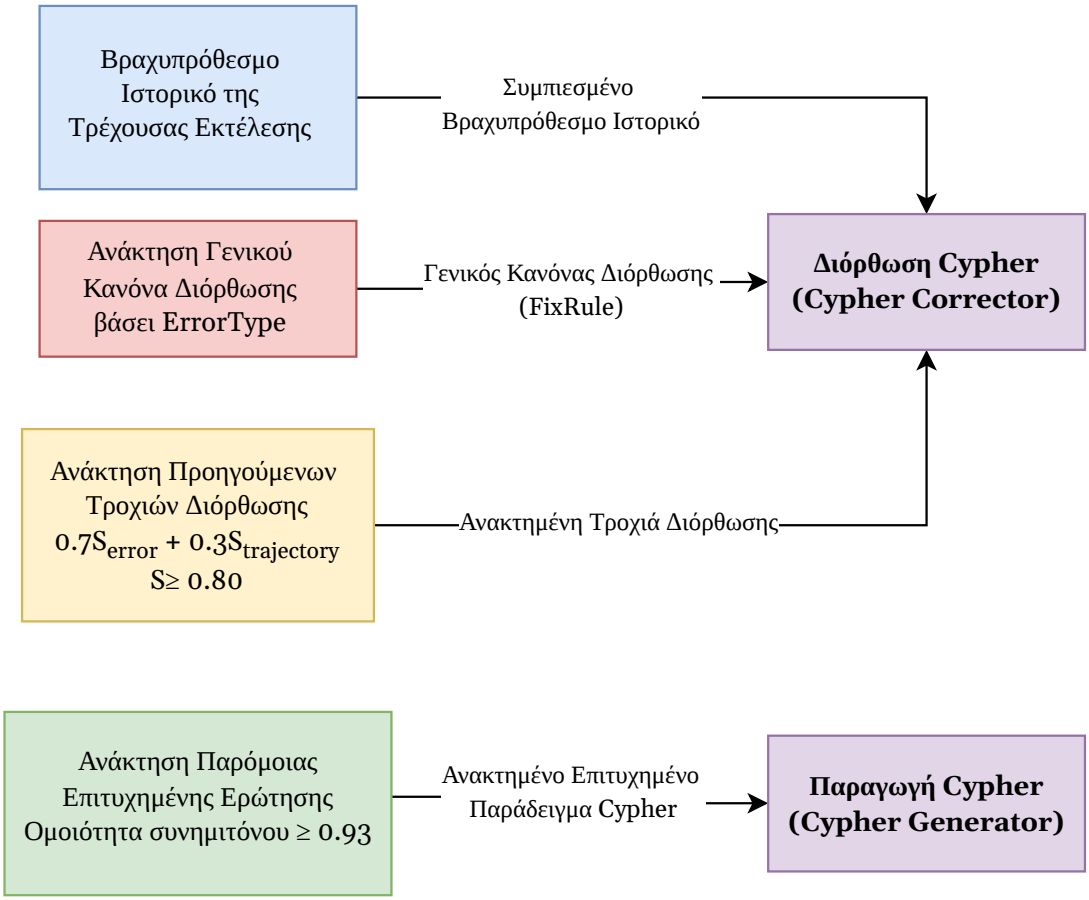
3.6 Λειτουργία Ανάκτησης και Αξιοποίησης της Μνήμης

Η αποθήκευση προηγούμενης εμπειρίας από μόνη της δεν επαρκεί για την αξιοποίησή της από την πρακτορική ροή. Απαιτείται επιπλέον ένας μηχανισμός επιλογής της πληροφορίας που είναι σχετική με το τρέχον πρόβλημα και εισαγωγής της στο κατάλλη-

λο στάδιο της διαδικασίας. Η επιλεκτική ανάκτηση σχετικών προηγούμενων εμπειριών αποτελεί βασική λειτουργία των αρχιτεκτονικών μνήμης πρακτόρων [2].

Στην προτεινόμενη αρχιτεκτονική υλοποιήθηκαν τρεις βασικοί μηχανισμοί αξιοποίησης της μνήμης: η ανάκτηση παρόμοιων προηγούμενων επιτυχημένων λύσεων πριν από την αρχική παραγωγή, η αξιοποίηση του βραχυπρόθεσμου ιστορικού της τρέχουσας εκτέλεσης και η ανάκτηση προηγούμενων τροχιών διόρθωσης κατά το στάδιο της διόρθωσης. Επιπλέον, στο στάδιο της διόρθωσης μπορεί να παρέχεται στον Cypher Corrector προκαθορισμένη καθοδήγηση μέσω των κανόνων `FixRule`, ανάλογα με τον τύπο του εντοπισμένου σφάλματος.

Οι διαφορετικές μορφές ανακτημένης πληροφορίας και τα σημεία της πρακτορικής ροής στα οποία αξιοποιούνται συνοψίζονται στο Σχήμα 3.6.



Σχήμα 3.6: Μορφές ανακτημένης πληροφορίας και προκαθορισμένης διορθωτικής καθοδήγησης που αξιοποιούνται στα στάδια παραγωγής και διόρθωσης ερωτημάτων Cypher.

3.6.1 Ανάκτηση Παρόμοιας Επιτυχημένης Ερώτησης

Η ανάκτηση παρόμοιας επιτυχημένης ερώτησης εφαρμόζεται πριν από την αρχική παραγωγή Cypher. Η τρέχουσα ερώτηση μετατρέπεται σε διάνυσμα και χρησιμοποιείται για αναζήτηση στο ευρετήριο των κόμβων `Question`. Από τη βάση ανακτάται η πλησιέστε-

ρη προηγούμενη ερώτηση που συνδέεται μέσω της RESOLVED_WITH με μια επιτυχημένη Answer.

Η ανάκτηση γίνεται επιλεκτικά. Η προηγούμενη λύση χρησιμοποιείται μόνο όταν η ομοιότητα συνημιτόνου είναι τουλάχιστον 0.93. Το συγκεκριμένο κατώφλι αποτελεί παράμετρο της προτεινόμενης υλοποίησης και χρησιμοποιείται ώστε να περιορίζεται η εισαγωγή παραδειγμάτων τα οποία είναι μόνο θεματικά συγγενή, χωρίς να παρουσιάζουν επαρκώς παρόμοια πρόθεση.

Η ανακτημένη προηγούμενη εμπειρία, δηλαδή η παρόμοια ερώτηση και η επιτυχημένη εντολή Cypher που τη συνόδευε, δεν αντιμετωπίζεται ως λύση που πρέπει να αντιγραφεί αυτούσια. Αντίθετα, ενσωματώνεται στην προτροπή του Cypher Generator ως δομικό παράδειγμα για τον τρόπο οργάνωσης των προτάσεων MATCH, WITH, WHERE και RETURN. Η πλήρης προτροπή του Cypher Generator, καθώς και η μορφή με την οποία το ανακτημένο επιτυχημένο παράδειγμα ενσωματώνεται σε αυτή, παρατίθενται στις Ενότητες Α.1 και Α.3.1 του Παραρτήματος Α. Επισημαίνεται ότι το γλωσσικό μοντέλο εξακολουθεί να δεσμεύεται από το πραγματικό σχήμα και το περιεχόμενο της τρέχουσας ερώτησης.

3.6.2 Βραχυπρόθεσμο Ιστορικό της Τρέχουσας Εκτέλεσης

Το βραχυπρόθεσμο ιστορικό αφορά αποκλειστικά το ενεργό ερώτημα και δεν απαιτεί πρόσβαση στη μακροπρόθεσμη βάση. Η κατάσταση του LangGraph διατηρεί αθροιστικά τα διαδοχικά ερωτήματα Cypher και τα σφάλματα που παρήχθησαν κατά τον τρέχοντα κύκλο παραγωγής και διόρθωσης.

Η πληροφορία αυτή μπορεί να εισαχθεί στην προτροπή του Cypher Corrector, ώστε το γλωσσικό μοντέλο να γνωρίζει ποιες λύσεις έχουν ήδη δοκιμαστεί και ποια προβλήματα παρουσίασαν. Με αυτόν τον τρόπο επιδιώκεται να μειωθεί η πιθανότητα επανάληψης της ίδιας ανεπιτυχούς στρατηγικής σε διαδοχικές προσπάθειες.

Καθώς η απλή παράθεση όλων των προηγούμενων προσπαθειών μπορεί να αυξήσει σημαντικά το μήκος της προτροπής, υλοποιήθηκε επιπλέον μηχανισμός συμπίεσης του ιστορικού. Για τον σχηματισμό του κλειδιού ομαδοποίησης, τα μηνύματα σφάλματος κανονικοποιούνται αφαιρώντας αποκλειστικά πληροφορία θέσης της μορφής αριθμού γραμμής, στήλης και μετατόπισης χαρακτήρα (offset). Στη συνέχεια ταξινομούνται, ώστε προσπάθειες που παρήγαγαν την ίδια συλλογή κανονικοποιημένων σφαλμάτων να ομαδοποιούνται ανεξάρτητα από τη σειρά εμφάνισής τους. Η κανονικοποίηση αυτή χρησιμοποιείται μόνο για τη σύγκριση και την ομαδοποίηση των προσπαθειών, ενώ στο τελικό πλαίσιο που παρέχεται στον Cypher Corrector διατηρούνται τα αρχικά μηνύματα σφάλματος. Με τον τρόπο αυτό μειώνεται ο πλεονασμός χωρίς να απορρίπτεται η αρχική πληροφορία ανατροφοδότησης του CyVer. Η πλήρης προτροπή του Cypher Corrector και η μορφή της συμπιεσμένης αναπαράστασης του βραχυπρόθεσμου ιστορικού

παρατίθενται στις Ενότητες Α.2 και Α.3.2 του Παραρτήματος Α.

3.6.3 Ανάκτηση Προηγούμενων Τροχιών Διόρθωσης

Η πιο σύνθετη στρατηγική μακροπρόθεσμης ανάκτησης αφορά την αναζήτηση προηγούμενων αποτυχημένων προσπαθειών που είναι σχετικές τόσο ως προς το σφάλμα όσο και ως προς το συνολικό πλαίσιο του προβλήματος.

Για το τρέχον σφάλμα δημιουργείται αρχικά ένα διάνυσμα σφάλματος και πραγματοποιείται αναζήτηση στο `error_vector_embeddings`, ώστε να προκύψει ένα αρχικό σύνολο υποψήφιων προηγούμενων περιπτώσεων. Για κάθε υποψήφια περίπτωση υπολογίζεται στη συνέχεια απευθείας η ομοιότητα συνημιτόνου μεταξύ του `trajectory_embedding` της αποθηκευμένης περίπτωσης και της αντίστοιχης αναπαράστασης της τρέχουσας προσπάθειας.

Οι δύο τιμές συνδυάζονται σε μία τελική βαθμολογία:

$$S = 0.7S_{\text{error}} + 0.3S_{\text{trajectory}},$$

όπου S_{error} είναι η ομοιότητα του σφάλματος και $S_{\text{trajectory}}$ η ομοιότητα της συνολικής τροχιάς. Μεγαλύτερη βαρύτητα αποδίδεται στο ίδιο το σφάλμα, καθώς αποτελεί το άμεσο πρόβλημα που καλείται να επιλύσει ο Cypher Corrector, ενώ η πληροφορία της τροχιάς χρησιμοποιείται για τη διαφοροποίηση περιπτώσεων με παρόμοιο μήνυμα σφάλματος αλλά διαφορετική δομή.

Διατηρούνται μόνο υποψήφιες περιπτώσεις με συνολική βαθμολογία τουλάχιστον 0.80. Η αποδιπλοποίηση εφαρμόζεται στη συνέχεια, κατά τη συγκρότηση του τελικού ανακτημένου πλαισίου. Για τη σύγκριση των υποψήφιων περιπτώσεων χρησιμοποιείται μια κανονικοποιημένη υπογραφή του μηνύματος σφάλματος, στην οποία διατηρείται το βασικό τμήμα του μηνύματος και παραλείπονται τυχόν παρενθετικές λεπτομέρειες που το ακολουθούν. Με τον τρόπο αυτό, περιπτώσεις που αντιστοιχούν ουσιαστικά στο ίδιο σφάλμα δεν εισάγονται επανειλημμένα στην προτροπή του Cypher Corrector. Η αποδιπλοποίηση πραγματοποιείται μετά τον υπολογισμό της βαθμολογίας ομοιότητας, χρησιμοποιείται μόνο για τον περιορισμό του πλεονασμού στο τελικό ανακτημένο πλαίσιο και δεν μεταβάλλει τα αποθηκευμένα μηνύματα σφάλματος ή τις διανυσματικές τους αναπαραστάσεις.

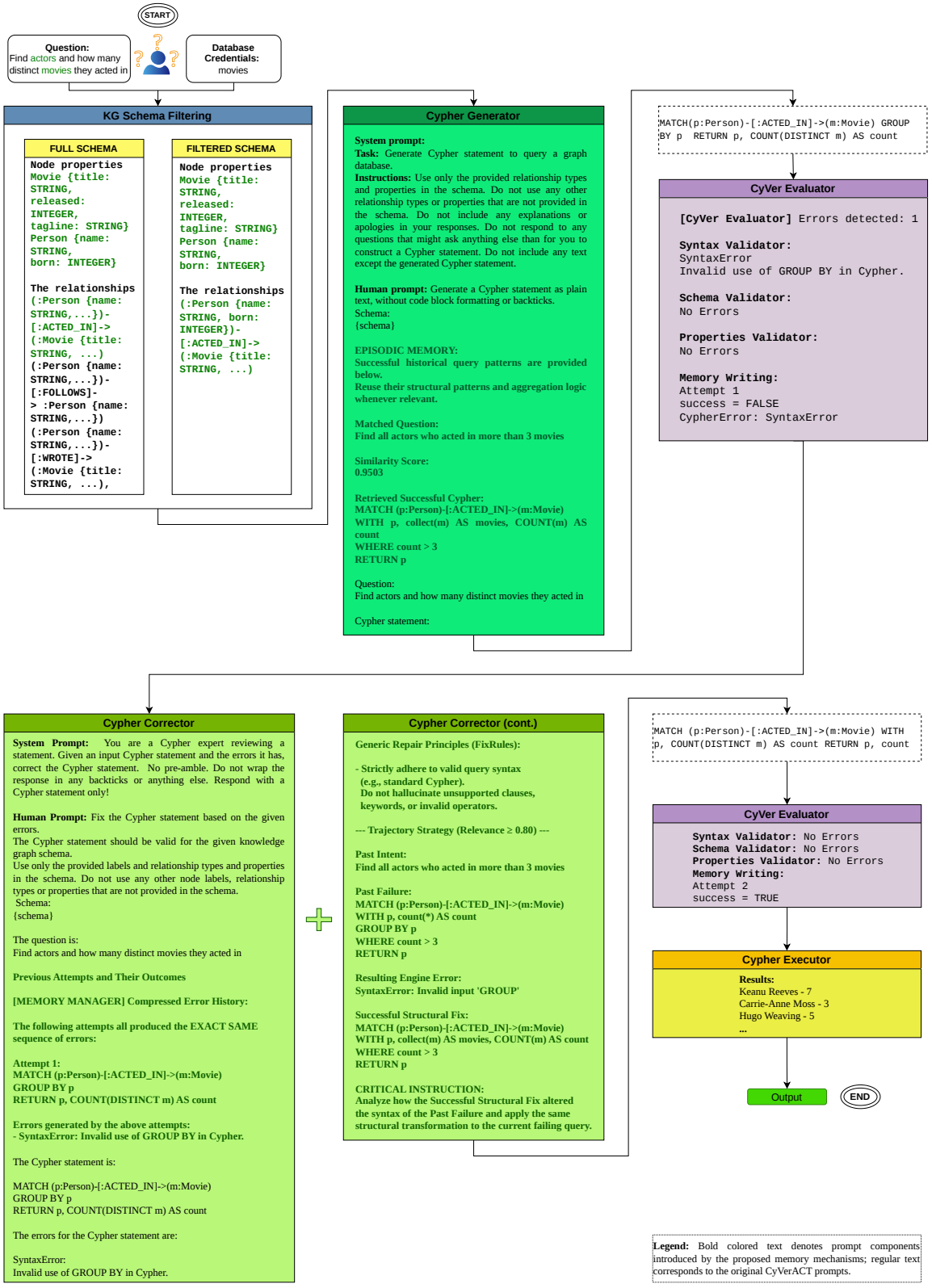
Για κάθε τροχιά που επιλέγεται μπορούν να ανακτηθούν η προηγούμενη ερώτηση, το αποτυχημένο ερώτημα Cypher, το μήνυμα σφάλματος και το τελικό επιτυχημένο ερώτημα με το οποίο είχε επιλυθεί η αντίστοιχη περίπτωση. Το πλαίσιο που εισάγεται στον Cypher Corrector τον καθοδηγεί να εντοπίσει τη δομική μεταβολή που μετέτρεψε το προηγούμενο αποτυχημένο ερώτημα σε έγκυρη λύση και να εξετάσει κατά πόσο η ίδια στρατηγική μπορεί να εφαρμοστεί στο τρέχον πρόβλημα.

Σημαντικό στοιχείο της διαδικασίας είναι ότι λαμβάνονται υπόψη μόνο προηγούμενες ερωτήσεις για τις οποίες υπάρχει αποθηκευμένη τελική `Answer`. Με τον τρόπο αυτό, η μνήμη δεν προτείνει ως παράδειγμα μια τροχιά για την οποία δεν είναι γνωστό ότι η διαδικασία διόρθωσης ολοκληρώθηκε επιτυχώς.

Οι παραπάνω μηχανισμοί ανάκτησης αξιοποιούνται τελικά μέσω του εμπλουτισμού των προτροπών των γλωσσικών σταδίων της πρακτορικής ροής. Στον Cypher Generator, ένα ανακτημένο επιτυχημένο παράδειγμα μπορεί να χρησιμοποιηθεί ως δομικός οδηγός για την παραγωγή της νέας εντολής. Αν η παραγόμενη εντολή αποτύχει στους ελέγχους του CyVer, ο Cypher Corrector μπορεί επιπλέον να αξιοποιήσει το ιστορικό της τρέχουσας εκτέλεσης, τον γενικό κανόνα διόρθωσης που αντιστοιχεί στο εντοπισμένο σφάλμα και μία συναφή προηγούμενη τροχιά διόρθωσης.

Ένα ενδεικτικό παράδειγμα εκτέλεσης στο οποίο αξιοποιούνται ταυτόχρονα οι τρεις παραπάνω μηχανισμοί μνήμης, μαζί με την προκαθορισμένη καθοδήγηση `FixRule`, παρουσιάζεται στο Σχήμα 3.7. Στο σχήμα επισημαίνεται τόσο το περιεχόμενο της πληροφορίας που ανακτάται από τη μνήμη όσο και η θέση στην οποία αυτή ενσωματώνεται στις προτροπές του Cypher Generator και του Cypher Corrector.

Για λόγους αναγνωσιμότητας, το Σχήμα 3.7 παρουσιάζει μόνο τα βασικά τμήματα των προτροπών και ενδεικτικά παραδείγματα της ανακτημένης πληροφορίας. Οι πλήρεις πρότυπες προτροπές του Cypher Generator και του Cypher Corrector, καθώς και η μορφή των δυναμικών συστατικών μνήμης, παρατίθενται στο Παράρτημα Α.



Σχήμα 3.7: Ενδεικτικό παράδειγμα εκτέλεσης της πρακτορικής ροής με ταυτόχρονη αξιοποίηση ανακτημένου επιτυχημένου παραδείγματος, βραχυπρόθεσμου ιστορικού, γενικού κανόνα διόρθωσης και προηγούμενης τροχιάς. Η βασική πρακτορική ροή είναι προσαρμοσμένη από [8], ενώ η ανακτημένη πληροφορία ενσωματώνεται στις προτροπές του Cypher Generator και του Cypher Corrector.

Οι τρεις μηχανισμοί μνήμης που εξετάζονται πειραματικά σχεδιάστηκαν ώστε να μπορούν να ενεργοποιούνται ανεξάρτητα ή σε συνδυασμό. Οι συγκεκριμένες πειραματικές διαμορφώσεις που προκύπτουν από τους συνδυασμούς αυτούς και χρησιμοποιούνται για την αξιολόγηση της προτεινόμενης προσέγγισης παρουσιάζονται στο Κεφάλαιο 4.

3.6.4 Ανάκτηση Γενικών Κανόνων Διόρθωσης

Οι γενικοί κανόνες διόρθωσης (`FixRule`) αποτελούν συμπληρωματική προκαθορισμένη καθοδήγηση προς τον Cypher Corrector. Σε αντίθεση με τις επεισοδιακές τροχιές προηγούμενων προσπαθειών, δεν αναπαριστούν μία συγκεκριμένη προηγούμενη εκτέλεση, αλλά κωδικοποιούν μία επαναχρησιμοποιήσιμη οδηγία για την αντιμετώπιση μιας κατηγορίας σφάλματος.

Στην υλοποίηση χρησιμοποιούνται δεκαπέντε προκαθορισμένες κατηγορίες σφάλματος, οι οποίες αντιστοιχούν στους ελέγχους του CyVer. Επτά από αυτές προέρχονται από τον `SyntaxValidator`, έξι από τον `SchemaValidator` και δύο από τον `Properties Validator` [8]. Για κάθε κατηγορία υπάρχει στον γράφο μνήμης ένας αντίστοιχος κόμβος `FixRule`, ο οποίος συνδέεται με τον `ErrorType` μέσω σχέσης `FIXED_WITH`.

Οι κανόνες αυτοί δεν παράγονται δυναμικά από τις προηγούμενες εκτελέσεις του πράκτορα, αλλά έχουν οριστεί εκ των προτέρων ως γενικές κατευθύνσεις διόρθωσης για τις αντίστοιχες κατηγορίες σφαλμάτων. Κατά τη διαδικασία διόρθωσης, ο κατάλληλος κανόνας μπορεί να ανακτηθεί με βάση τον τύπο του τρέχοντος σφάλματος και να ενσωματωθεί στο πλαίσιο που παρέχεται στον Cypher Corrector. Με τον τρόπο αυτό, η γενικευμένη διορθωτική γνώση λειτουργεί συμπληρωματικά προς την ανάκτηση συγκεκριμένων προηγούμενων αποτυχημένων προσπαθειών.

Η μορφή με την οποία οι ανακτημένοι κανόνες εισάγονται στην προτροπή του Cypher Corrector, καθώς και το πλήρες σύνολο των προκαθορισμένων `FixRules` που χρησιμοποιήθηκαν στην υλοποίηση, παρατίθενται στην Ενότητα Α.3.4 και στον Πίνακα Α.1 του Παραρτήματος Α.

3.7 Αποθετήριο Κώδικα της Εργασίας

Η υλοποίηση της προτεινόμενης αρχιτεκτονικής, καθώς και ο κώδικας που χρησιμοποιήθηκε για την ενσωμάτωση και αξιολόγηση των μηχανισμών μνήμης στο CyVerACT, είναι διαθέσιμα στο δημόσιο αποθετήριο GitLab της εργασίας:

`https://gitlab.com/netmode/agentic-memory`

3.8 Σύνοψη Κεφαλαίου

Στο παρόν κεφάλαιο παρουσιάστηκε η μεθοδολογία που ακολουθήθηκε για την επέκταση της πρακτορικής ροής CyVerACT με μηχανισμούς μνήμης. Αρχικά περιγράφηκε η βασική διαδικασία παραγωγής, αξιολόγησης και επαναληπτικής διόρθωσης ερωτημάτων Cypher και στη συνέχεια παρουσιάστηκε η διερεύνηση υφιστάμενων πλαισίων μνήμης, με ιδιαίτερη έμφαση στο Cognee. Οι περιορισμοί που εντοπίστηκαν κατά τη συγκεκριμένη διερεύνηση οδήγησαν στην επιλογή μιας προσαρμοσμένης λύσης, στην οποία η δομή, η αποθήκευση και η ανάκτηση της εμπειρίας ελέγχονται άμεσα από την εφαρμογή.

Η τελική αρχιτεκτονική οργανώνει την εμπειρία σε γράφο Neo4j, διατηρώντας τις ερωτήσεις, τις διαδοχικές προσπάθειες, τα σφάλματα και τις τελικές επιτυχημένες λύσεις. Πάνω στη δομή αυτή υλοποιήθηκαν διαφορετικοί μηχανισμοί αξιοποίησης της μνήμης: βραχυπρόθεσμο ιστορικό της τρέχουσας εκτέλεσης, ανάκτηση παρόμοιων επιτυχημένων ερωτήσεων και ανάκτηση προηγούμενων τροχιών διόρθωσης. Η δυνατότητα ανεξάρτητης ενεργοποίησης των μηχανισμών αυτών επιτρέπει την πειραματική μελέτη της επίδρασης διαφορετικών μορφών μνήμης, η οποία αποτελεί το αντικείμενο του επόμενου κεφαλαίου.

Κεφάλαιο 4

Πειραματική Αξιολόγηση

Στο παρόν κεφάλαιο παρουσιάζεται η πειραματική αξιολόγηση των μηχανισμών μνήμης που ενσωματώθηκαν στην πρακτορική ροή CyVerACT [8]. Η βασική πρακτορική ροή παρουσιάστηκε στην Ενότητα 3.2, ενώ οι μηχανισμοί αξιοποίησης της μνήμης περιγράφηκαν στην Ενότητα 3.6. Στόχος της αξιολόγησης είναι να διερευνηθεί κατά πόσο η αξιοποίηση προηγούμενης εμπειρίας μπορεί να επηρεάσει την παραγωγή και την επαναληπτική διόρθωση ερωτημάτων Cypher, καθώς και αν η επίδραση αυτή διαφοροποιείται ανάλογα με τον τρόπο ενσωμάτωσης της μνήμης και το χρησιμοποιούμενο γλωσσικό μοντέλο.

Για τον σκοπό αυτό συγκρίνεται η βασική πρακτορική ροή, χωρίς πρόσθετη πληροφορία μνήμης, με έξι παραλλαγές στις οποίες ενεργοποιούνται μεμονωμένα ή συνδυαστικά οι τρεις μηχανισμοί που εξετάζονται πειραματικά: το βραχυπρόθεσμο ιστορικό, η ανάκτηση προηγούμενων τροχιών και η ανάκτηση επιτυχημένου παραδείγματος στον Cypher Generator. Οι μηχανισμοί αυτοί περιγράφηκαν στην Ενότητα 3.6. Η αξιολόγηση πραγματοποιείται με τρία γλωσσικά μοντέλα διαφορετικής κλίμακας και με κοινό σύνολο 100 ερωτημάτων αξιολόγησης, ώστε οι επιμέρους διαμορφώσεις να εξετάζονται υπό τις ίδιες πειραματικές συνθήκες.

Το κεφάλαιο οργανώνεται ως εξής. Αρχικά, παρουσιάζονται η πειραματική διάταξη, το σύνολο δεδομένων, τα χρησιμοποιούμενα μοντέλα και οι εξεταζόμενες διαμορφώσεις. Στη συνέχεια, ορίζονται οι μετρικές αξιολόγησης και παρουσιάζονται τα πειραματικά αποτελέσματα. Ακολουθεί η συγκριτική εξέταση της επίδρασης των διαφορετικών μηχανισμών μνήμης, ενώ στο τέλος διατυπώνονται συνολικές παρατηρήσεις επί των αποτελεσμάτων και παρουσιάζονται οι περιορισμοί της πειραματικής αξιολόγησης.

4.1 Πειραματική Διάταξη

Η πειραματική διάταξη σχεδιάστηκε ώστε η σύγκριση μεταξύ των διαφορετικών παραλλαγών του συστήματος να πραγματοποιείται υπό κοινές συνθήκες. Για κάθε γλωσσικό

μοντέλο χρησιμοποιήθηκε το ίδιο σύνολο ερωτημάτων αξιολόγησης, ενώ οι βασικές παράμετροι της πρακτορικής ροής παρέμειναν σταθερές. Οι επιμέρους εκτελέσεις διαφοροποιούνται ως προς τους μηχανισμούς μνήμης που είναι ενεργοί, επιτρέποντας τη μελέτη της συνεισφοράς τόσο της πληροφορίας που παράγεται κατά την τρέχουσα εκτέλεση όσο και της εμπειρίας που έχει αποθηκευτεί από προηγούμενες εκτελέσεις.

4.1.1 Βάση και Περιβάλλον Εκτέλεσης

Ως γράφος γνώσης χρησιμοποιήθηκε η βάση Movies του Neo4j. Πρόκειται για τη βάση πάνω στην οποία εκτελούνται τόσο τα ερωτήματα αναφοράς Cypher όσο και τα ερωτήματα που παράγονται από την πρακτορική ροή. Η βάση Movies είναι διακριτή από τον γράφο μακροπρόθεσμης μνήμης που παρουσιάστηκε στην Ενότητα 3.4, ο οποίος χρησιμοποιείται για την αποθήκευση και την ανάκτηση προηγούμενης εμπειρίας.

Η σύνδεση με τη βάση Movies πραγματοποιήθηκε μέσω του διαθέσιμου στιγμιότυπου επίδειξης του Neo4j, με τις παραμέτρους που παρουσιάζονται στον Πίνακα 4.1.

Πίνακας 4.1: Παράμετροι σύνδεσης με τη βάση Movies.

Παράμετρος	**Τιμή**
Connection URI	`neo4j+s://demo.neo4jlabs.com`
Όνομα βάσης	`movies`
Όνομα χρήστη	`movies`
Κωδικός πρόσβασης	`movies`

Η χρήση της ίδιας βάσης σε όλες τις εκτελέσεις διασφαλίζει ότι οι συγκρίσεις μεταξύ γλωσσικών μοντέλων και πειραματικών διαμορφώσεων δεν επηρεάζονται από μεταβολές στο υποκείμενο σχήμα ή στα διαθέσιμα δεδομένα.

4.1.2 Σύνολο Δεδομένων και Διαχωρισμός Ερωτημάτων

Για την πειραματική αξιολόγηση χρησιμοποιήθηκε το test split του συνόλου δεδομένων Neo4j Text2Cypher (2024) (text2cypher-2024v1) [40], από το οποίο επιλέχθηκε το τμήμα που αντιστοιχεί στη βάση Movies. Το αρχικό σύνολο δεδομένων περιλαμβάνει ζεύγη ερωτήσεων φυσικής γλώσσας και αντίστοιχων ερωτημάτων Cypher, μαζί με πληροφορία για το σχήμα της βάσης και σχετικά μεταδεδομένα. Για τις ανάγκες της παρούσας αξιολόγησης χρησιμοποιήθηκαν η ερώτηση φυσικής γλώσσας και το αντίστοιχο ερώτημα αναφοράς Cypher. Προκειμένου το τελικό πειραματικό σύνολο να περιλαμβάνει ερωτήματα διαφορετικού βαθμού πολυπλοκότητας, τα ερωτήματα αναφοράς Cypher ταξινομήθηκαν σε τρεις κατηγορίες: εύκολα (Easy), μέτριας δυσκολίας (Medium) και δύσκολα (Hard).

Η ταξινόμηση πραγματοποιήθηκε βάσει των χαρακτηριστικών όρων και λειτουργιών που εμφανίζονται στο αντίστοιχο ερώτημα αναφοράς Cypher και εφαρμόστηκε ιεραρχικά. Εάν εντοπιζόταν τουλάχιστον ένα στοιχείο της κατηγορίας Hard, η εγγραφή κατατασσόταν σε αυτή την κατηγορία. Διαφορετικά, η παρουσία στοιχείου της κατηγορίας Medium οδηγούσε στην κατηγορία μέτριας δυσκολίας, ενώ οι υπόλοιπες περιπτώσεις χαρακτηρίζονταν ως Easy. Τα κριτήρια που χρησιμοποιήθηκαν συνοψίζονται στον Πίνακα 4.2.

Πίνακας 4.2: Κριτήρια ταξινόμησης των ερωτημάτων με βάση την πολυπλοκότητα του ερωτήματος αναφοράς Cypher.

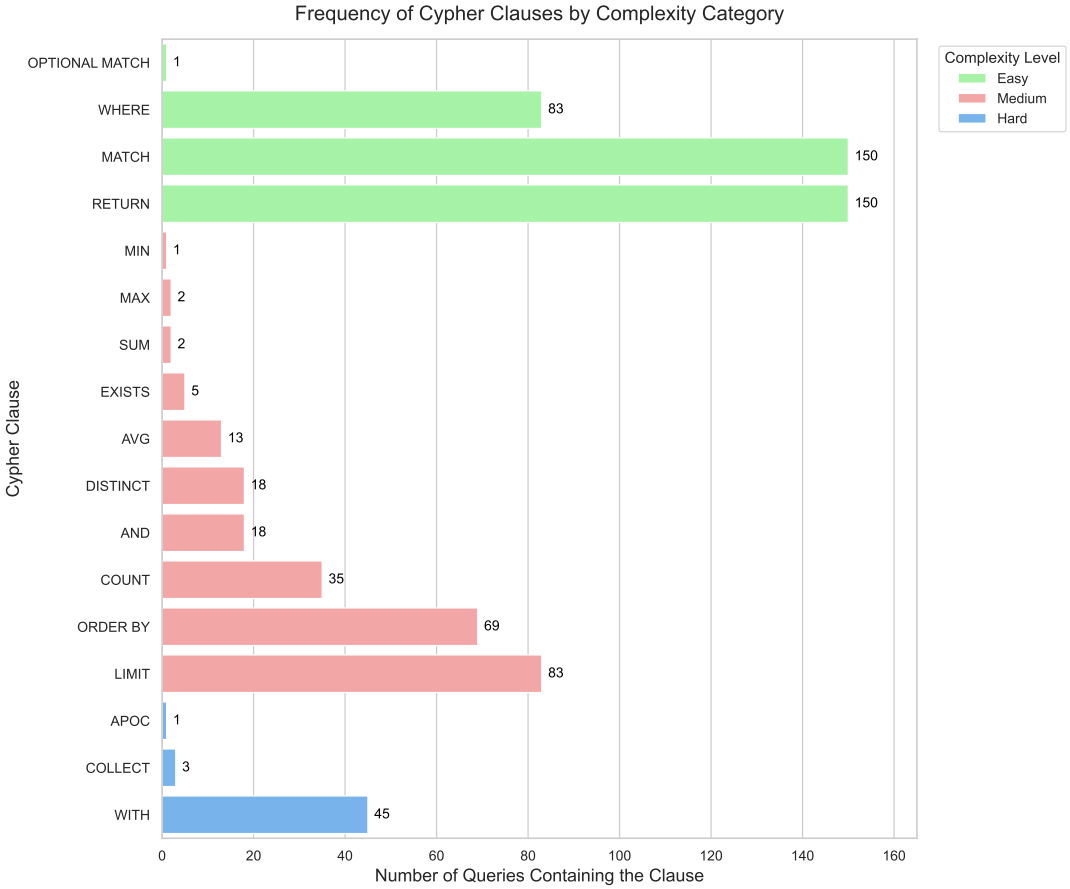
Κατηγορία	**Κριτήριο ταξινόμησης**
Hard	Παρουσία τουλάχιστον μίας από τις λειτουργίες `COLLECT`, `FILTER`, `UNWIND`, `WITH`, `FOREACH`, `REDUCE`, `CALL`, `YIELD` ή `APOC`.
Medium	Εφόσον δεν έχει ενεργοποιηθεί η κατηγορία Hard, παρουσία `ORDER BY`, `LIMIT`, `SKIP`, `COUNT`, `DISTINCT`, `EXISTS`, `IN`, `AND`, `OR`, `MIN`, `MAX`, `AVG` ή `SUM`.
Easy	Ερωτήματα στα οποία δεν εμφανίζεται κάποιο από τα παραπάνω στοιχεία και τα οποία βασίζονται κυρίως σε βασικές λειτουργίες αναζήτησης, φιλτραρίσματος και επιστροφής αποτελεσμάτων, όπως `MATCH`, `OPTIONAL MATCH`, `WHERE` και `RETURN`.

Μετά την κατηγοριοποίηση επιλέχθηκαν συνολικά 150 ερωτήματα, με δειγματοληψία που διατήρησε την αναλογία των τριών κατηγοριών δυσκολίας. Το σύνολο αυτό διαχωρίστηκε στη συνέχεια σε 100 ερωτήματα αξιολόγησης και 50 ερωτήματα για την αρχική τροφοδότηση της μνήμης, διατηρώντας αντίστοιχη αναλογία κατηγοριών και στα δύο υποσύνολα. Και στα δύο στάδια χρησιμοποιήθηκε σταθερός σπόρος τυχαιότητας, ώστε η διαδικασία επιλογής και διαχωρισμού να είναι αναπαραγώγιμη.

Τα 50 ερωτήματα μνήμης χρησιμοποιήθηκαν πριν από την κύρια αξιολόγηση για τη δημιουργία προηγούμενης εμπειρίας στις διαμορφώσεις που αξιοποιούν μακροπρόθεσμη μνήμη, ενώ τα υπόλοιπα 100 αποτέλεσαν το κοινό σύνολο αξιολόγησης για όλα τα γλωσσικά μοντέλα και όλες τις πειραματικές διαμορφώσεις. Ο διαχωρισμός πραγματοποιήθηκε έτσι ώστε κάθε εγγραφή να ανήκει αποκλειστικά σε ένα από τα δύο σύνολα. Κατά την κύρια αξιολόγηση, η εγγραφή νέας εμπειρίας στη μακροπρόθεσμη μνήμη παρέμενε απενεργοποιημένη, με αποτέλεσμα τα 100 ερωτήματα αξιολόγησης να μην προστίθενται στον γράφο μνήμης και οι αντίστοιχες διαμορφώσεις να αξιοποιούν μόνο την εμπειρία που είχε δημιουργηθεί από τα 50 ερωτήματα μνήμης. Με τον τρόπο αυτό αποφεύχθηκε η μεταφορά πληροφορίας μεταξύ ερωτημάτων του ίδιου συνόλου αξιολόγησης.

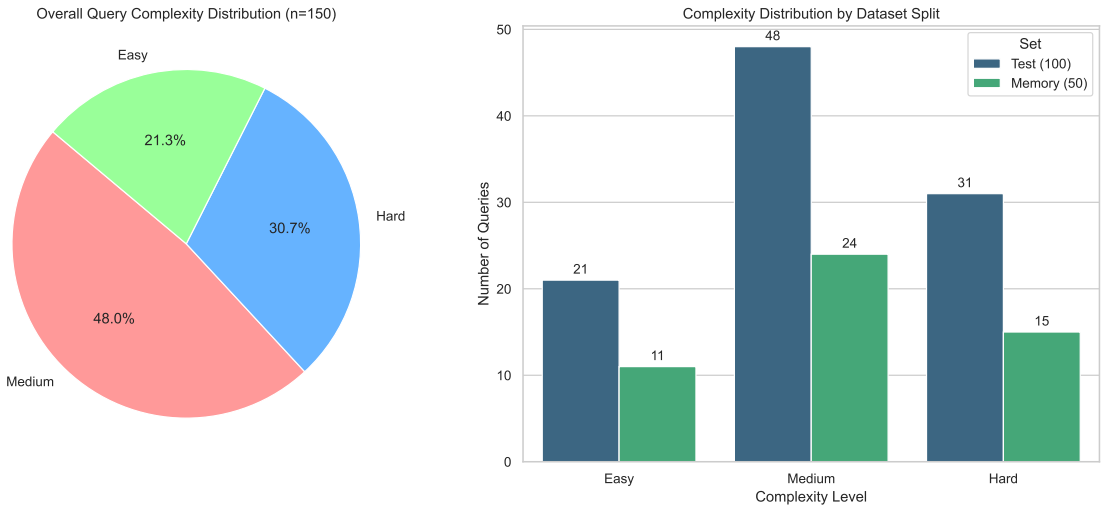
Η συχνότητα εμφάνισης των βασικών χαρακτηριστικών όρων και λειτουργιών

Cypher στα 150 επιλεγμένα ερωτήματα παρουσιάζεται στο Σχήμα 4.1. Η απεικόνιση λειτουργεί συμπληρωματικά προς τον Πίνακα 4.2, καθώς αποτυπώνει τη συχνότητα των χαρακτηριστικών που χρησιμοποιήθηκαν για την κατηγοριοποίηση των ερωτημάτων στο τελικό πειραματικό σύνολο.



Σχήμα 4.1: Συχνότητα εμφάνισης βασικών χαρακτηριστικών όρων και λειτουργιών Cypher στα 150 επιλεγμένα ερωτήματα, ομαδοποιημένων σύμφωνα με τα κριτήρια πολυπλοκότητας.

Η τελική κατανομή παρουσιάζεται στο Σχήμα 4.2. Από τα 150 επιλεγμένα ερωτήματα, 32 χαρακτηρίστηκαν ως Easy, 72 ως Medium και 46 ως Hard, αντιστοιχώντας σε ποσοστά 21.3%, 48.0% και 30.7%, αντίστοιχα. Στο σύνολο αξιολόγησης περιλαμβάνονται 21 εύκολα, 48 μέτρια και 31 δύσκολα ερωτήματα, ενώ το σύνολο μνήμης περιλαμβάνει αντίστοιχα 11, 24 και 15 ερωτήματα. Με τον τρόπο αυτό διατηρείται ουσιαστικά η ίδια αναλογία επιπέδων δυσκολίας στα δύο υποσύνολα.



Σχήμα 4.2: Κατανομή των 150 επιλεγμένων ερωτημάτων ανά επίπεδο δυσκολίας και διαχωρισμός τους σε σύνολο αξιολόγησης και σύνολο μνήμης.

4.1.3 Γλωσσικά Μοντέλα και Παράμετροι Εκτέλεσης

Για τη διερεύνηση της επίδρασης της μνήμης σε μοντέλα διαφορετικής κλίμακας επιλέχθηκαν τα Llama-3.1-8B-Instruct, Qwen-2.5-7B-Instruct και Llama-3.2-3B-Instruct. Η χρήση των τριών μοντέλων επιτρέπει να εξεταστεί αν η επίδραση των μηχανισμών μνήμης παραμένει ίδια ή μεταβάλλεται καθώς διαφοροποιείται η βασική ικανότητα του γλωσσικού μοντέλου.

Η πρόσβαση στα μοντέλα πραγματοποιήθηκε μέσω της υπηρεσίας OpenRouter. Οι βασικές παράμετροι της παραγωγής και της πρακτορικής ροής διατηρήθηκαν ίδιες σε όλες τις πειραματικές εκτελέσεις και συνοψίζονται στον Πίνακα 4.3.

Πίνακας 4.3: Κοινές παράμετροι των πειραματικών εκτελέσεων.

Παράμετρος	**Τιμή**
Θερμοκρασία παραγωγής (`temperature`)	0.0
Μέγιστο μήκος απόκρισης (`max_tokens`)	512 tokens
Μέγιστος αριθμός προσπαθειών με φιλτραρισμένο σχήμα (`attempts_w_filtered`)	5
Μέγιστος συνολικός αριθμός προσπαθειών (`total_attempts`)	10
Όριο αναδρομής της πρακτορικής ροής (`recursion_limit`)	50

Η μηδενική θερμοκρασία χρησιμοποιήθηκε για τον περιορισμό της στοχαστικότητας κατά την παραγωγή και για τη διατήρηση όσο το δυνατόν πιο σταθερών συνθηκών

μεταξύ των συγκρινόμενων διαμορφώσεων. Όπως περιγράφηκε στην Ενότητα 3.2, η διαδικασία ξεκινά με το φιλτραρισμένο σχήμα και επιτρέπει έως πέντε προσπάθειες σε αυτή τη φάση. Εάν δεν παραχθεί έγκυρο ερώτημα Cypher, ενεργοποιείται ο KG Schema Retriever και η διαδικασία συνεχίζεται με το πλήρες σχήμα, μέχρι το συνολικό όριο των δέκα προσπαθειών. Η ίδια παραμετροποίηση εφαρμόστηκε σε όλα τα μοντέλα και σε όλες τις πειραματικές διαμορφώσεις.

4.1.4 Πειραματικές Διαμορφώσεις Μνήμης

Για καθένα από τα τρία γλωσσικά μοντέλα αξιολογήθηκαν επτά διαμορφώσεις της πρακτορικής ροής. Ως σημείο αναφοράς χρησιμοποιείται η Baseline, δηλαδή η κλασική πρακτορική ροή CyVerACT [8], στην οποία δεν προστίθεται πληροφορία μνήμης στις προτροπές του Cypher Generator ή του Cypher Corrector. Οι υπόλοιπες διαμορφώσεις ενεργοποιούν μεμονωμένα ή συνδυαστικά τους μηχανισμούς που παρουσιάστηκαν στην Ενότητα 3.6.

Η αντιστοίχιση των διαμορφώσεων με τους ενεργούς μηχανισμούς παρουσιάζεται στον Πίνακα 4.4. Οι τρεις μηχανισμοί αντιστοιχούν στις στρατηγικές αξιοποίησης της μνήμης που παρουσιάστηκαν στο Κεφάλαιο 3. Ειδικότερα, το ιστορικό (History) αντιστοιχεί στο βραχυπρόθεσμο ιστορικό της τρέχουσας εκτέλεσης (Ενότητα 3.6.2), η τροχιά (Trajectory) στην ανάκτηση προηγούμενων τροχιών διόρθωσης από τη μακροπρόθεσμη μνήμη (Ενότητα 3.6.3), ενώ το Gen Helper αντιστοιχεί στην ανάκτηση παρόμοιας επιτυχημένης ερώτησης και του αντίστοιχου ερωτήματος Cypher πριν από την αρχική παραγωγή (Ενότητα 3.6.1).

Οι προκαθορισμένοι κανόνες `FixRule` (Ενότητα 3.6.4) αποτελούν σταθερή διορθωτική καθοδήγηση που μπορεί να ενσωματωθεί στην προτροπή του Cypher Corrector με βάση τον εντοπισμένο τύπο σφάλματος. Παρόλο που είναι ενεργοί σε όλες τις παραλλαγές μνήμης, δεν αντιμετωπίζονται ως ανεξάρτητος μηχανισμός μνήμης ούτε απομονώνονται ως ξεχωριστή πειραματική διάσταση. Για τον λόγο αυτό, οι διαμορφώσεις του Πίνακα 4.4 διαφοροποιούνται ως προς τους τρεις μηχανισμούς που εξετάζονται ανεξάρτητα: το ιστορικό, την ανάκτηση προηγούμενης τροχιάς και το παράδειγμα στον Cypher Generator.

Πίνακας 4.4: Πειραματικές διαμορφώσεις και ενεργοί μηχανισμοί αξιοποίησης της μνήμης.

Διαμόρφωση	**Ιστορικό**	**Τροχιά**	**Παράδειγμα στον Generator**
Baseline	–	–	–
History Only	✓	–	–
Trajectory Only	–	✓	–
Trajectory + History	✓	✓	–
Gen Helper + History	✓	–	✓
Gen Helper + Trajectory	–	✓	✓
CyVerACT All-In	✓	✓	✓

Οι διαμορφώσεις History Only και Trajectory Only απομονώνουν την επίδραση των δύο αντίστοιχων μορφών πληροφορίας στο στάδιο της διόρθωσης. Η Trajectory + History τις συνδυάζει, ενώ οι Gen Helper + History και Gen Helper + Trajectory ενεργοποιούν επιπλέον την ανάκτηση επιτυχημένου παραδείγματος πριν από την πρώτη παραγωγή. Στη διαμόρφωση CyVerACT All-In είναι ενεργοί και οι τρεις εξεταζόμενοι μηχανισμοί.

Οι παράμετροι ανάκτησης διατηρήθηκαν σταθερές σε όλες τις σχετικές πειραματικές διαμορφώσεις. Όπως περιγράφηκε στην Ενότητα 3.6, για την ανάκτηση παρόμοιας επιτυχημένης ερώτησης χρησιμοποιήθηκε κατώφλι ομοιότητας συνημιτόνου 0.93, ενώ για την ανάκτηση προηγούμενης τροχιάς εφαρμόστηκε κατώφλι 0.80 στη συνδυαστική βαθμολογία συνάφειας. Με τον τρόπο αυτό οι πειραματικές διαμορφώσεις διαφοροποιούνται ως προς τους ενεργούς μηχανισμούς μνήμης και όχι ως προς τις παραμέτρους λειτουργίας τους.

Η παραπάνω διάταξη καθορίζει το κοινό πειραματικό πλαίσιο πάνω στο οποίο πραγματοποιούνται όλες οι μετρήσεις. Στην επόμενη ενότητα παρουσιάζονται οι μετρικές που χρησιμοποιούνται για την αξιολόγηση της εγκυρότητας των παραγόμενων ερωτημάτων Cypher, της συμφωνίας των αποτελεσμάτων εκτέλεσης και της ομοιότητάς τους με τα ερωτήματα αναφοράς Cypher.

Κατά την εκτέλεση των πειραμάτων καταγράφηκε επιπλέον ο χρόνος που απαιτήθηκε για την επεξεργασία κάθε ερωτήματος από την πρακτορική ροή. Η καταγραφή αυτή χρησιμοποιείται στη συνέχεια για τη σύγκριση του χρόνου εκτέλεσης μεταξύ των διαφορετικών πειραματικών διαμορφώσεων.

4.2 Μετρικές Αξιολόγησης

Η αξιολόγηση των παραγόμενων ερωτημάτων Cypher πραγματοποιήθηκε σε διαφορετικά επίπεδα, ακολουθώντας το πλαίσιο αξιολόγησης του CyVerACT [8]. Η χρήση

περισσότερων του ενός κριτηρίων είναι απαραίτητη, καθώς η τεχνική εγκυρότητα ενός ερωτήματος Cypher δεν συνεπάγεται απαραίτητα ότι αυτό εκφράζει σωστά την πρόθεση της ερώτησης φυσικής γλώσσας και επιστρέφει το αναμενόμενο αποτέλεσμα. Αντίστοιχα, δύο ερωτήματα Cypher μπορούν να διαφέρουν ως προς τη συντακτική τους μορφή χωρίς η διαφορά αυτή να συνεπάγεται διαφορετική σημασιολογία.

Για τον λόγο αυτό εξετάζονται τρεις κύριες πτυχές: η εγκυρότητα του παραγόμενου ερωτήματος ως προς τον γράφο γνώσης, η συμφωνία του αποτελέσματος εκτέλεσης με εκείνο του ερωτήματος αναφοράς Cypher και η ομοιότητα του παραγόμενου ερωτήματος με το αντίστοιχο ερώτημα αναφοράς Cypher. Επιπλέον, καταγράφονται ο αριθμός των διορθωτικών επαναλήψεων που απαιτούνται μέχρι την ολοκλήρωση της πρακτορικής ροής καθώς και οι χρόνοι εκτέλεσης.

4.2.1 CyVer Μετρικές

Η πρώτη κατηγορία αξιολόγησης αφορά την εγκυρότητα του τελικού παραγόμενου ερωτήματος Cypher ως προς τη βάση Movies. Για κάθε ερώτημα ενεργοποιούνται οι τρεις ελεγκτές του CyVer που παρουσιάστηκαν στην Ενότητα 3.2: ο Syntax Validator, ο Schema Validator και ο Properties Validator. Οι ίδιοι έλεγχοι χρησιμοποιούνται και στο πλαίσιο αξιολόγησης του CyVerACT [8].

Ο Syntax Validator εξετάζει τη συντακτική εγκυρότητα του ερωτήματος, ο Schema Validator τη συμβατότητά του με τις ετικέτες, τους τύπους σχέσεων και τις διαδρομές του γράφου, ενώ ο Properties Validator ελέγχει τη χρήση των ιδιοτήτων. Με βάση τα αποτελέσματα των επιμέρους ελέγχων προκύπτει η ένδειξη KG Valid. Ένα ερώτημα Cypher χαρακτηρίζεται ως έγκυρο όταν είναι συντακτικά ορθό και δεν παρουσιάζει ασυμβατότητα με το σχήμα ή τις ιδιότητες του γράφου.

Η συγκεκριμένη μετρική αποτελεί βασικό κριτήριο της αξιολόγησης, καθώς εκφράζει το ποσοστό των ερωτήσεων φυσικής γλώσσας για τις οποίες η πρακτορική ροή καταλήγει σε τεχνικά έγκυρο ερώτημα Cypher. Για τις επιτυχημένες περιπτώσεις καταγράφεται επιπλέον αν η τελική λύση προέκυψε κατά τη φάση χρήσης του φιλτραρισμένου σχήματος ή μετά τη μετάβαση στο πλήρες σχήμα.

4.2.2 Μετρικές Αποτελέσματος Εκτέλεσης

Η εγκυρότητα ως προς τον γράφο γνώσης δεν επαρκεί για να αποδειχθεί ότι το παραγόμενο ερώτημα Cypher απαντά σωστά στην αρχική ερώτηση φυσικής γλώσσας. Ένα ερώτημα μπορεί να είναι συντακτικά και δομικά έγκυρο, αλλά να επιστρέφει διαφορετικά αποτελέσματα από το ερώτημα αναφοράς Cypher. Για τον λόγο αυτό χρησιμοποιούνται επιπλέον μετρικές που βασίζονται στην εκτέλεση των δύο ερωτημάτων στη βάση, όπως και στην αξιολόγηση του CyVerACT [8].

Η βασική αριθμητική μετρική αυτής της κατηγορίας είναι η Jaccard Output. Η μετρική αυτή δεν συγκρίνει τη συντακτική μορφή των δύο ερωτημάτων Cypher, αλλά τα αποτελέσματα που επιστρέφονται μετά την εκτέλεσή τους. Για δύο σύνολα αποτελεσμάτων A και B, η ομοιότητα Jaccard ορίζεται ως

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

και λαμβάνει τιμές στο διάστημα $[0, 1]$. Η τιμή 0 δηλώνει ότι δεν υπάρχει κοινό στοιχείο μεταξύ των αποτελεσμάτων, ενώ η τιμή 1 δηλώνει πλήρη συμφωνία. Ενδιάμεσες τιμές αντιστοιχούν σε μερική επικάλυψη των αποτελεσμάτων.

Στην υλοποίηση της παρούσας εργασίας, όταν το ερώτημα αναφοράς Cypher περιλαμβάνει `ORDER BY`, η σειρά των επιστρεφόμενων εγγραφών διατηρείται κατά τη σύγκριση. Διαφορετικά, οι γραμμές των δύο αποτελεσμάτων αντιστοιχίζονται πριν από τον υπολογισμό της ομοιότητας, ώστε μία διαφορετική σειρά επιστροφής των ίδιων εγγραφών να μην θεωρείται απόκλιση. Όταν επιστρέφονται ολόκληροι κόμβοι ή σχέσεις, η σύγκριση πραγματοποιείται με βάση τις ιδιότητές τους. Στην ειδική περίπτωση όπου και τα δύο αποτελέσματα είναι κενά, η Jaccard Output ορίζεται ίση με 1.

Από την τιμή της Jaccard Output προκύπτουν δύο επιπλέον δυαδικές ενδείξεις. Η Pass@1 Output αποτελεί το αυστηρότερο κριτήριο και λαμβάνει την τιμή True μόνο όταν η ομοιότητα αποτελέσματος είναι ίση με 1, δηλαδή όταν υπάρχει πλήρης συμφωνία μεταξύ του αποτελέσματος του παραγόμενου ερωτήματος και εκείνου του ερωτήματος αναφοράς Cypher. Σε κάθε άλλη περίπτωση λαμβάνει την τιμή False. Η αντίστοιχη λογική χρησιμοποιείται στο CyVerACT, όπου η Pass@1 αναφέρεται επίσης ως Exact Match [8].

Η Positive Jaccard Output αποτελεί λιγότερο αυστηρή δυαδική ένδειξη. Λαμβάνει την τιμή True όταν η Jaccard Output είναι μεγαλύτερη από το μηδέν, ανεξάρτητα από το μέγεθος της επικάλυψης. Επομένως, καταγράφει τόσο τις περιπτώσεις πλήρους συμφωνίας όσο και εκείνες στις οποίες υπάρχει έστω μερική κοινή πληροφορία μεταξύ των δύο αποτελεσμάτων.

Οι τρεις παραπάνω ποσότητες συνδέονται επομένως άμεσα: η Jaccard Output εκφράζει τον βαθμό συμφωνίας στο διάστημα $[0, 1]$, η Positive Jaccard Output ελέγχει αν η τιμή αυτή είναι μεγαλύτερη από 0, ενώ η Pass@1 Output ελέγχει αν είναι ακριβώς ίση με 1.

Οι μετρικές αποτελέσματος υπολογίζονται όταν το παραγόμενο ερώτημα Cypher έχει χαρακτηριστεί έγκυρο ως προς τον γράφο γνώσης και υπάρχει διαθέσιμο αποτέλεσμα για σύγκριση. Σε περιπτώσεις στις οποίες δεν πληρούνται οι προϋποθέσεις αυτές, οι αντίστοιχες τιμές δεν υπολογίζονται.

4.2.3 Μετρικές Ομοιότητας Ερωτημάτων Cypher

Σε αντίθεση με τις μετρικές της προηγούμενης ενότητας, οι οποίες συγκρίνουν τα αποτελέσματα εκτέλεσης, οι μετρικές της παρούσας ενότητας συγκρίνουν απευθείας τη μορφή του παραγόμενου ερωτήματος Cypher με τη μορφή του ερωτήματος αναφοράς Cypher. Συμπληρωματικά προς τις μετρικές εκτέλεσης υπολογίζονται οι BLEU, ROUGE-L, Jaro-Winkler και Jaccard Similarity, οι οποίες χρησιμοποιούνται επίσης στην αξιολόγηση του CyVerACT [8].

Η BLEU βασίζεται στην επικάλυψη ακολουθιών όρων μεταξύ των δύο ερωτημάτων, ενώ η ROUGE-L εξετάζει τη μεγαλύτερη κοινή υποακολουθία και παρέχει τιμές ακρίβειας (precision), ανάκλησης (recall) και F_1. Στην παρούσα αξιολόγηση χρησιμοποιείται κυρίως η τιμή ROUGE-L F1-score.

Η Jaccard Similarity, σε αντίθεση με τη Jaccard Output της προηγούμενης ενότητας, δεν υπολογίζεται πάνω στα αποτελέσματα εκτέλεσης. Υπολογίζεται πάνω στα σύνολα όρων των δύο κανονικοποιημένων ερωτημάτων Cypher και αποτυπώνει την έκταση της λεκτικής τους επικάλυψης. Η Jaro–Winkler συγκρίνει αντίστοιχα τις κανονικοποιημένες συμβολοσειρές σε επίπεδο χαρακτήρων.

Οι συγκεκριμένες μετρικές αντιμετωπίζονται ως συμπληρωματικές ενδείξεις και όχι ως αποκλειστικά κριτήρια σημασιολογικής ορθότητας. Η Cypher επιτρέπει διαφορετικές συντακτικές διατυπώσεις της ίδιας λογικής, με αποτέλεσμα δύο ερωτήματα που επιστρέφουν ισοδύναμα αποτελέσματα να εμφανίζουν χαμηλότερη λεκτική ομοιότητα [8]. Για τον λόγο αυτό, κατά την ανάλυση των αποτελεσμάτων δίνεται μεγαλύτερη βαρύτητα στην εγκυρότητα ως προς τον γράφο και στις μετρικές αποτελέσματος εκτέλεσης.

4.2.4 Διορθωτικές Επαναλήψεις

Εκτός από την τελική ποιότητα του παραγόμενου ερωτήματος Cypher, εξετάστηκε και η πορεία που ακολούθησε η πρακτορική ροή μέχρι την ολοκλήρωσή της. Για κάθε έγκυρη περίπτωση καταγράφεται ο αριθμός των διορθωτικών επαναλήψεων του Cypher Corrector. Η ανάλυση των επαναλήψεων ακολουθεί τη διάκριση μεταξύ των δύο φάσεων της ροής που περιγράφηκαν στην Ενότητα 3.2 και χρησιμοποιείται επίσης στην ανάλυση του βασικού CyVerACT [8].

Στα διαγράμματα ο αριθμός των διορθωτικών επαναλήψεων προκύπτει από το καταγεγραμμένο μονοπάτι εκτέλεσης (`path`), μετρώντας τις εμφανίσεις του `cypher_corrector` στην αντίστοιχη φάση της ροής. Για τα ερωτήματα που ολοκληρώθηκαν με το φιλτραρισμένο σχήμα μετρώνται οι κλήσεις του Cypher Corrector πριν από την ενεργοποίηση του KG Schema Retriever, ενώ για τις περιπτώσεις που απαιτούν το πλήρες σχήμα μετρώνται μόνο οι κλήσεις που ακολουθούν τη μετάβαση σε αυτό. Η τιμή 0 δηλώνει ότι το ερώτημα κρίθηκε έγκυρο χωρίς να απαιτηθεί διορθωτική κλήση στη συγκεκριμένη φάση. Με τον τρόπο αυτό μπορεί να εξεταστεί όχι μόνο αν μία διαμόρφω-

ση καταλήγει σε έγκυρη λύση, αλλά και πόσες διορθωτικές προσπάθειες απαιτούνται μέχρι την επίτευξή της.

Πίνακας 4.5: Σύνοψη των μετρικών και των συμπληρωματικών κριτηρίων που χρησιμοποιούνται στην πειραματική αξιολόγηση.

Μετρική / κριτήριο	**Επίπεδο αξιολόγησης**	**Ερμηνεία**
KG Valid	Εγκυρότητα ερωτήματος	Συνδυαστικός έλεγχος σύνταξης, σχήματος και ιδιοτήτων του παραγόμενου ερωτήματος Cypher.
Pass@1 Output	Αποτέλεσμα εκτέλεσης	Πλήρης συμφωνία με το αποτέλεσμα του ερωτήματος αναφοράς Cypher.
Jaccard Output	Αποτέλεσμα εκτέλεσης	Βαθμός ομοιότητας μεταξύ των αποτελεσμάτων των δύο ερωτημάτων, στο διάστημα $[0, 1]$.
Positive Jaccard Output	Αποτέλεσμα εκτέλεσης	Ένδειξη ύπαρξης θετικής ομοιότητας αποτελεσμάτων (Jaccard Output > 0).
BLEU	Ομοιότητα ερωτημάτων	Ομοιότητα βάσει της επικάλυψης ακολουθιών όρων.
ROUGE-L F1-score	Ομοιότητα ερωτημάτων	Ομοιότητα βάσει της μεγαλύτερης κοινής υποακολουθίας.
Jaro–Winkler	Ομοιότητα ερωτημάτων	Ομοιότητα των κανονικοποιημένων συμβολοσειρών σε επίπεδο χαρακτήρων.
Jaccard Similarity	Ομοιότητα ερωτημάτων	Επικάλυψη των συνόλων όρων των δύο ερωτημάτων.
Διορθωτικές επαναλήψεις	Διαδικασία επίλυσης	Πλήθος διορθώσεων μέχρι την επίτευξη έγκυρης λύσης, διακριτά για τη φάση φιλτραρισμένου και πλήρους σχήματος.

Στα διαγράμματα που ακολουθούν, το ποσοστό των περιπτώσεων που καταλήγουν σε KG Valid ερώτημα Cypher υπολογίζεται επί του συνολικού συνόλου των 100 ερωτημάτων αξιολόγησης. Αντίθετα, τα ποσοστά Pass@1 Output και Positive Jaccard Output παρουσιάζονται επί του υποσυνόλου των περιπτώσεων που έχουν καταλήξει σε KG Valid ερώτημα Cypher, ώστε η συμφωνία των αποτελεσμάτων να εξετάζεται διακριτά από την τεχνική εγκυρότητα του παραγόμενου ερωτήματος.

4.3 Συνολικά Πειραματικά Αποτελέσματα

Στην παρούσα ενότητα παρουσιάζονται τα αποτελέσματα των επτά πειραματικών διαμορφώσεων ξεχωριστά για καθένα από τα τρία γλωσσικά μοντέλα. Η παρουσίαση επι-

κεντρώνεται αρχικά στην εγκυρότητα των παραγόμενων ερωτημάτων Cypher ως προς τον γράφο γνώσης και στη συνέχεια στη συμφωνία των αποτελεσμάτων εκτέλεσης, μέσω των μετρικών Pass@1 Output και Positive Jaccard Output. Η συγκριτική επίδραση των επιμέρους μηχανισμών μνήμης μεταξύ των μοντέλων εξετάζεται αναλυτικότερα στην Ενότητα 4.4.

Για τη διευκόλυνση της συνολικής σύγκρισης, οι Πίνακες 4.6 και 4.7 συγκεντρώνουν τις επιδόσεις και των επτά πειραματικών διαμορφώσεων για τα τρία γλωσσικά μοντέλα. Ο Πίνακας 4.6 παρουσιάζει τις κύριες μετρικές εγκυρότητας και αποτελέσματος εκτέλεσης, ενώ ο Πίνακας 4.7 συγκεντρώνει τις συμπληρωματικές μετρικές ομοιότητας μεταξύ του παραγόμενου ερωτήματος Cypher και του ερωτήματος αναφοράς. Όλες οι τιμές παρουσιάζονται σε κλίμακα 0–100, ενώ οι τιμές σε παρένθεση δηλώνουν τη μεταβολή ως προς την αντίστοιχη διαμόρφωση Baseline. Με έντονη γραφή επισημαίνεται η υψηλότερη επίδοση για κάθε συνδυασμό μετρικής και γλωσσικού μοντέλου.

Σε αντίθεση με τα διαγράμματα που ακολουθούν, όπου οι μετρικές Pass@1 Output και Positive Jaccard Output παρουσιάζονται υπό τη συνθήκη ότι έχει προηγουμένως παραχθεί KG Valid ερώτημα, στον συγκεντρωτικό πίνακα τα αντίστοιχα ποσοστά αναφέρονται στο σύνολο των 100 ερωτημάτων αξιολόγησης. Με τον τρόπο αυτό παρέχεται μία κοινή βάση σύγκρισης της συνολικής απόδοσης των διαφορετικών διαμορφώσεων.

Τα αποτελέσματα που συνοψίζονται στον Πίνακα 4.6 δείχνουν ότι η επίδραση των μηχανισμών μνήμης είναι πιο συνεπής ως προς την KG Valid, ενώ οι μεταβολές στις Pass@1 Output και Positive Jaccard Output εξαρτώνται περισσότερο από το μοντέλο και τη διαμόρφωση. Η επίδραση των επιμέρους μηχανισμών εξετάζεται αναλυτικότερα στην Ενότητα 4.4.

Πίνακας 4.6: Συγκεντρωτικά αποτελέσματα των κύριων μετρικών αξιολόγησης για τις επτά πειραματικές διαμορφώσεις και τα τρία γλωσσικά μοντέλα. Όλες οι τιμές παρουσιάζονται σε κλίμακα 0–100 και οι τιμές σε παρένθεση δηλώνουν τη μεταβολή ως προς την αντίστοιχη διαμόρφωση Baseline. Με έντονη γραφή σημειώνεται η υψηλότερη επίδοση ανά μετρική και μοντέλο.

Μετρική	**Διαμόρφωση**	**Llama-3.1-8B-Instruct**	**Qwen-2.5-7B-Instruct**	**Llama-3.2-3B-Instruct**
KG Valid	Baseline	88.00	73.00	44.00
	Gen Helper + History	87.00 (-1.00)	76.00 (+3.00)	48.00 (+4.00)
	Gen Helper + Trajectory	**91.00 (+3.00)**	82.00 (+9.00)	**65.00 (+21.00)**
	History Only	85.00 (-3.00)	73.00 (+0.00)	51.00 (+7.00)
	Trajectory + History	84.00 (-4.00)	80.00 (+7.00)	54.00 (+10.00)
	Trajectory Only	89.00 (+1.00)	81.00 (+8.00)	60.00 (+16.00)
	CyVerACT All-In	80.00 (-8.00)	**83.00 (+10.00)**	62.00 (+18.00)
Pass@1 Output	Baseline	19.00	20.00	7.00
	Gen Helper + History	**24.00 (+5.00)**	19.00 (-1.00)	3.00 (-4.00)
	Gen Helper + Trajectory	22.00 (+3.00)	16.00 (-4.00)	7.00 (+0.00)
	History Only	21.00 (+2.00)	17.00 (-3.00)	3.00 (-4.00)
	Trajectory + History	23.00 (+4.00)	18.00 (-2.00)	**10.00 (+3.00)**
	Trajectory Only	21.00 (+2.00)	**24.00 (+4.00)**	6.00 (-1.00)
	CyVerACT All-In	**24.00 (+5.00)**	23.00 (+3.00)	5.00 (-2.00)
Positive Jaccard Output	Baseline	52.00	42.00	25.00
	Gen Helper + History	54.00 (+2.00)	**48.00 (+6.00)**	23.00 (-2.00)
	Gen Helper + Trajectory	51.00 (-1.00)	42.00 (+0.00)	**31.00 (+6.00)**
	History Only	45.00 (-7.00)	44.00 (+2.00)	24.00 (-1.00)
	Trajectory + History	49.00 (-3.00)	40.00 (-2.00)	26.00 (+1.00)
	Trajectory Only	48.00 (-4.00)	45.00 (+3.00)	28.00 (+3.00)
	CyVerACT All-In	**55.00 (+3.00)**	**48.00 (+6.00)**	27.00 (+2.00)

Πίνακας 4.7: Συγκεντρωτικά αποτελέσματα των μετρικών ομοιότητας μεταξύ των παραγόμενων ερωτημάτων Cypher και των αντίστοιχων ερωτημάτων αναφοράς για τις επτά πειραματικές διαμορφώσεις και τα τρία γλωσσικά μοντέλα. Όλες οι τιμές παρουσιάζονται σε κλίμακα 0–100 και οι τιμές σε παρένθεση δηλώνουν τη μεταβολή ως προς την αντίστοιχη διαμόρφωση Baseline. Με έντονη γραφή σημειώνεται η υψηλότερη επίδοση ανά μετρική και μοντέλο.

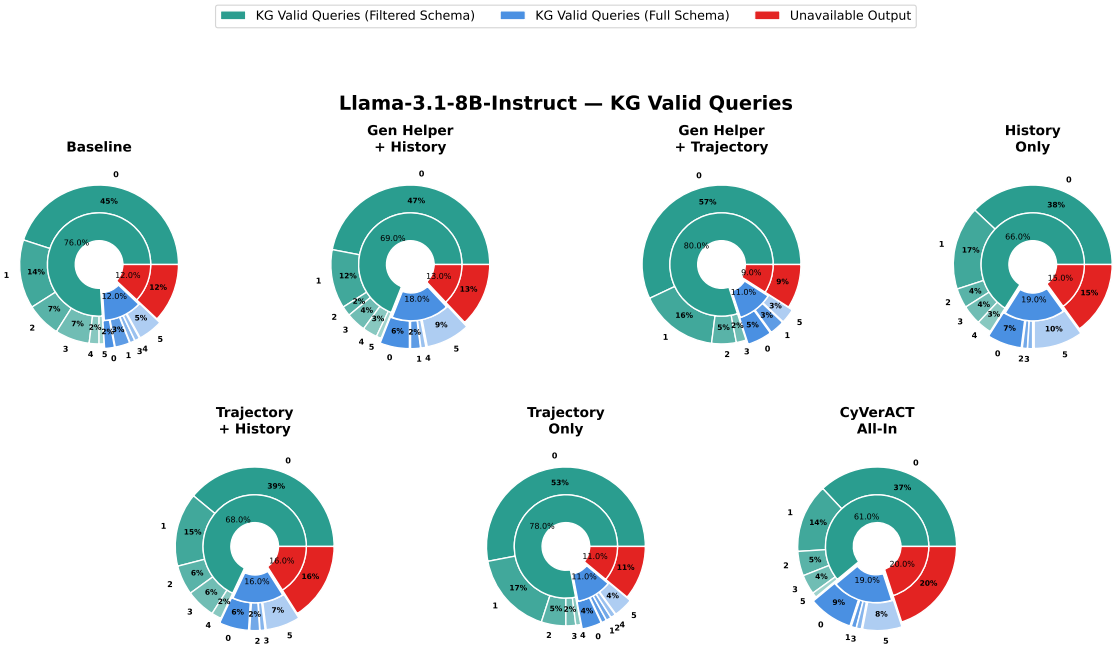
Μετρική	Διαμόρφωση	Llama-3.1-8B-Instruct	Qwen-2.5-7B-Instruct	Llama-3.2-3B-Instruct
BLEU	Baseline	48.97	51.78	41.01
	Gen Helper + History	50.91 (+1.94)	**52.53 (+0.76)**	40.88 (-0.13)
	Gen Helper + Trajectory	51.14 (+2.17)	48.99 (-2.79)	**43.31 (+2.30)**
	History Only	48.74 (-0.24)	50.15 (-1.63)	41.17 (+0.16)
	Trajectory + History	49.94 (+0.97)	52.42 (+0.65)	41.78 (+0.77)
	Trajectory Only	**51.42 (+2.45)**	51.63 (-0.14)	43.04 (+2.03)
	CyVerACT All-In	51.22 (+2.25)	52.28 (+0.50)	42.56 (+1.55)
ROUGE-L F1-score	Baseline	67.78	66.91	59.06
	Gen Helper + History	66.78 (-1.00)	**68.71 (+1.80)**	59.41 (+0.35)
	Gen Helper + Trajectory	**67.86 (+0.08)**	64.43 (-2.48)	60.73 (+1.67)
	History Only	66.49 (-1.29)	66.55 (-0.36)	59.82 (+0.76)
	Trajectory + History	66.31 (-1.47)	68.21 (+1.30)	60.63 (+1.57)
	Trajectory Only	67.57 (-0.21)	67.30 (+0.39)	60.51 (+1.45)
	CyVerACT All-In	67.70 (-0.08)	68.25 (+1.34)	**61.26 (+2.20)**
Jaro–Winkler	Baseline	87.19	86.22	82.91
	Gen Helper + History	87.00 (-0.19)	87.04 (+0.81)	82.82 (-0.09)
	Gen Helper + Trajectory	**87.52 (+0.33)**	86.77 (+0.54)	83.23 (+0.32)
	History Only	86.23 (-0.96)	86.13 (-0.09)	82.91 (+0.01)
	Trajectory + History	85.80 (-1.39)	**87.90 (+1.68)**	82.60 (-0.30)
	Trajectory Only	87.39 (+0.20)	87.17 (+0.94)	**83.61 (+0.70)**
	CyVerACT All-In	86.61 (-0.57)	87.47 (+1.24)	82.28 (-0.63)
Jaccard Similarity	Baseline	67.92	70.13	56.74
	Gen Helper + History	67.25 (-0.68)	69.98 (-0.15)	56.65 (-0.08)
	Gen Helper + Trajectory	68.00 (+0.07)	68.55 (-1.58)	57.91 (+1.18)
	History Only	67.40 (-0.52)	68.68 (-1.45)	56.91 (+0.18)
	Trajectory + History	67.38 (-0.54)	70.00 (-0.12)	58.06 (+1.32)
	Trajectory Only	**69.72 (+1.79)**	69.84 (-0.29)	56.15 (-0.59)
	CyVerACT All-In	68.53 (+0.60)	**70.24 (+0.11)**	**58.78 (+2.04)**

Οι συμπληρωματικές μετρικές ομοιότητας του Πίνακα 4.7 εμφανίζουν συνολικά μικρότερες μεταβολές από εκείνες που παρατηρούνται στην KG Valid. Ειδικότερα, η Jaccard Similarity δεν παρουσιάζει μία συνεπή βελτίωση μεταξύ των μοντέλων: η μεγαλύτερη αύξηση είναι $+1.79$ μονάδες με τη Trajectory Only στο Llama-3.1-8B-Instruct και $+2.04$ μονάδες με τη CyVerACT All-In στο Llama-3.2-3B-Instruct, ενώ στο Qwen-2.5-7B-Instruct οι περισσότερες διαμορφώσεις κινούνται οριακά κάτω από τη Baseline. Η εικόνα αυτή δείχνει ότι η βελτίωση της τεχνικής εγκυρότητας δεν συνοδεύεται απαραίτητα από μεγαλύτερη λεκτική ομοιότητα με το ερώτημα αναφοράς. Το αποτέλεσμα είναι αναμενόμενο, καθώς διαφορετικές συντακτικές διατυπώσεις Cypher μπορούν να εκφράζουν την ίδια λογική και να οδηγούν σε ισοδύναμο αποτέλεσμα.

Οι συγκεντρωτικοί πίνακες λειτουργούν ως συνοπτική αναφορά των αριθμητικών αποτελεσμάτων, ενώ οι επόμενες υποενότητες εξετάζουν αναλυτικότερα τη συμπεριφορά κάθε γλωσσικού μοντέλου. Ιδιαίτερη έμφαση δίνεται στη διάκριση μεταξύ τεχνικής εγκυρότητας του παραγόμενου ερωτήματος και συμφωνίας των αποτελεσμάτων εκτέλεσης, καθώς και στον αριθμό των διορθωτικών επαναλήψεων που απαιτήθηκαν για την επίτευξη έγκυρης λύσης.

4.3.1 Llama-3.1-8B-Instruct

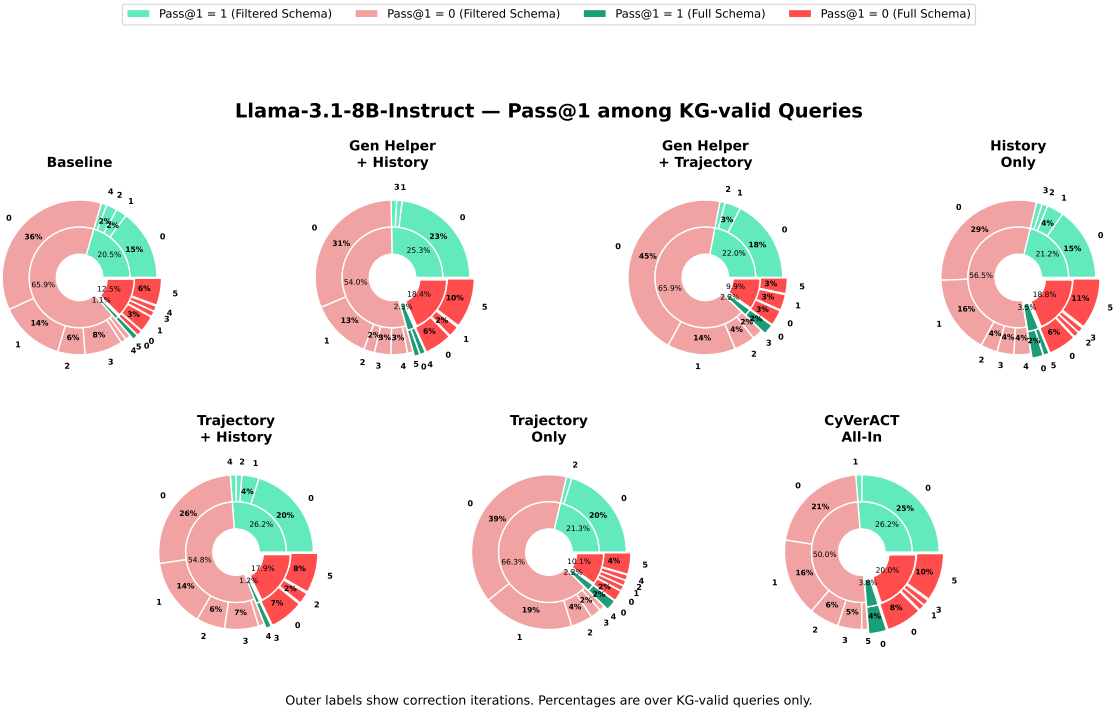
Η κατανομή των έγκυρων ερωτημάτων ανά διαμόρφωση και διορθωτική επανάληψη παρουσιάζεται στο Σχήμα 4.3.



Σχήμα 4.3: Κατανομή των περιπτώσεων που καταλήγουν σε KG Valid ερώτημα Cypher για τις επτά πειραματικές διαμορφώσεις του Llama-3.1-8B-Instruct. Διακρίνονται οι λύσεις που προέκυψαν με φιλτραρισμένο και πλήρες σχήμα, ενώ ο εξωτερικός δακτύλιος αποτυπώνει τις διορθωτικές επαναλήψεις.

Το Llama-3.1-8B-Instruct εμφανίζει υψηλή αρχική επίδοση ως προς την εγκυρότητα των παραγόμενων ερωτημάτων Cypher. Στη διαμόρφωση Baseline, 88% των ερωτήσεων αξιολόγησης καταλήγουν σε KG Valid ερώτημα Cypher. Η υψηλότερη τιμή παρατηρείται στη Gen Helper + Trajectory, όπου το ποσοστό αυξάνεται σε 91%, ενώ η Trajectory Only παρουσιάζει επίσης μικρή βελτίωση, φθάνοντας το 89%. Αντίθετα, οι διαμορφώσεις Gen Helper + History, History Only, Trajectory + History και CyVerACT All-In καταγράφουν χαμηλότερο ποσοστό εγκυρότητας από τη βασική διαμόρφωση, με την τελευταία να περιορίζεται στο 80%. Το αποτέλεσμα αυτό δείχνει ότι η ταυτόχρονη ενεργοποίηση περισσότερων μηχανισμών δεν οδηγεί κατ' ανάγκη σε μεγαλύτερο αριθμό έγκυρων ερωτημάτων.

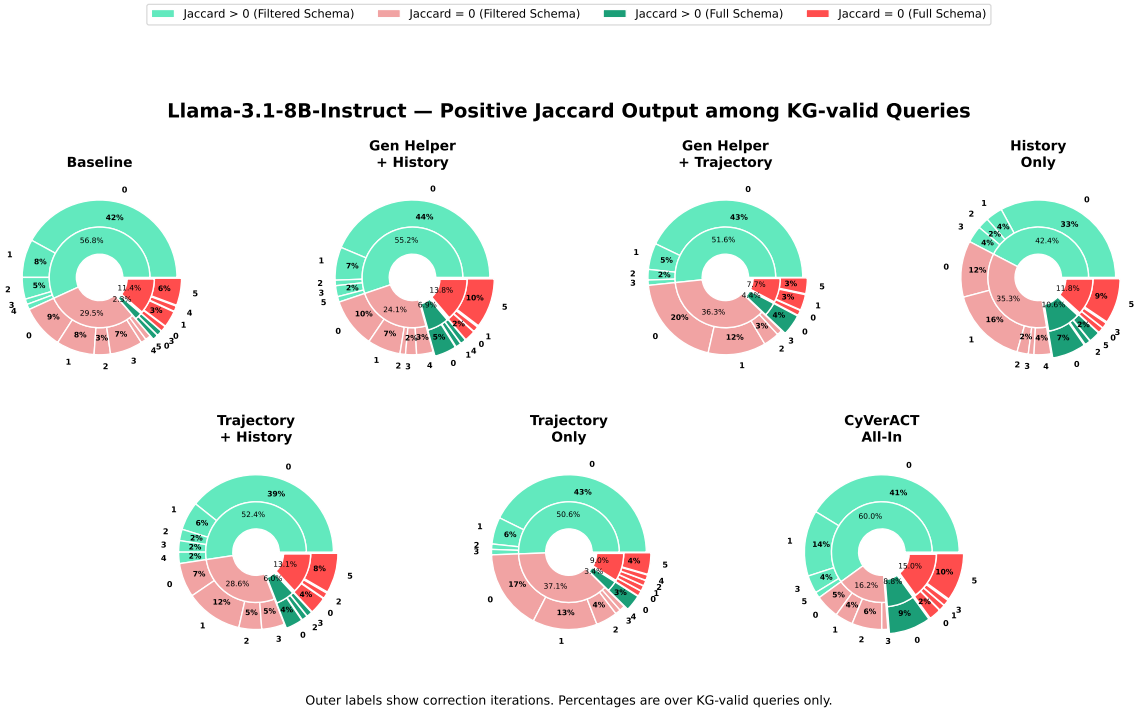
Η συμπεριφορά των ίδιων διαμορφώσεων ως προς την πλήρη συμφωνία των αποτελεσμάτων αποτυπώνεται στο Σχήμα 4.4.



Σχήμα 4.4: Κατανομή της Pass@1 Output μεταξύ των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher για τις επτά διαμορφώσεις του Llama-3.1-8B-Instruct. Ο εξωτερικός δακτύλιος παρουσιάζει τις αντίστοιχες διορθωτικές επαναλήψεις.

Ως προς την πλήρη συμφωνία των αποτελεσμάτων, η εικόνα διαφοροποιείται. Στη βασική διαμόρφωση, 19 από τα 88 έγκυρα ερωτήματα παρουσιάζουν πλήρη συμφωνία με το αποτέλεσμα του ερωτήματος αναφοράς Cypher, ποσοστό 21.6% επί των KG Valid περιπτώσεων. Η Gen Helper + History αυξάνει τον αριθμό αυτό σε 24 από 87 έγκυρα ερωτήματα (27.6%), ενώ η Trajectory + History φθάνει το 27.4%. Η υψηλότερη αναλογία εμφανίζεται στη CyVerACT All-In, με 24 πλήρεις συμφωνίες μεταξύ 80 έγκυρων ερωτημάτων, δηλαδή 30.0%. Η τελευταία περίπτωση είναι ενδεικτική της ανάγκης να εξετάζονται χωριστά η τεχνική εγκυρότητα και η ορθότητα του αποτελέσματος: η συγκεκριμένη διαμόρφωση παράγει λιγότερα έγκυρα ερωτήματα συνολικά, αλλά μεγαλύτερο ποσοστό πλήρων συμφωνιών μέσα στο έγκυρο υποσύνολο.

Συμπληρωματικά, το Σχήμα 4.5 παρουσιάζει την κατανομή των περιπτώσεων με θετική Jaccard Output, ώστε να εξεταστεί και η ύπαρξη μερικής συμφωνίας των αποτελεσμάτων.



Σχήμα 4.5: Κατανομή θετικής Jaccard Output μεταξύ των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher για τις επτά διαμορφώσεις του Llama-3.1-8B-Instruct. Ο εξωτερικός δακτύλιος παρουσιάζει τις αντίστοιχες διορθωτικές επαναλήψεις.

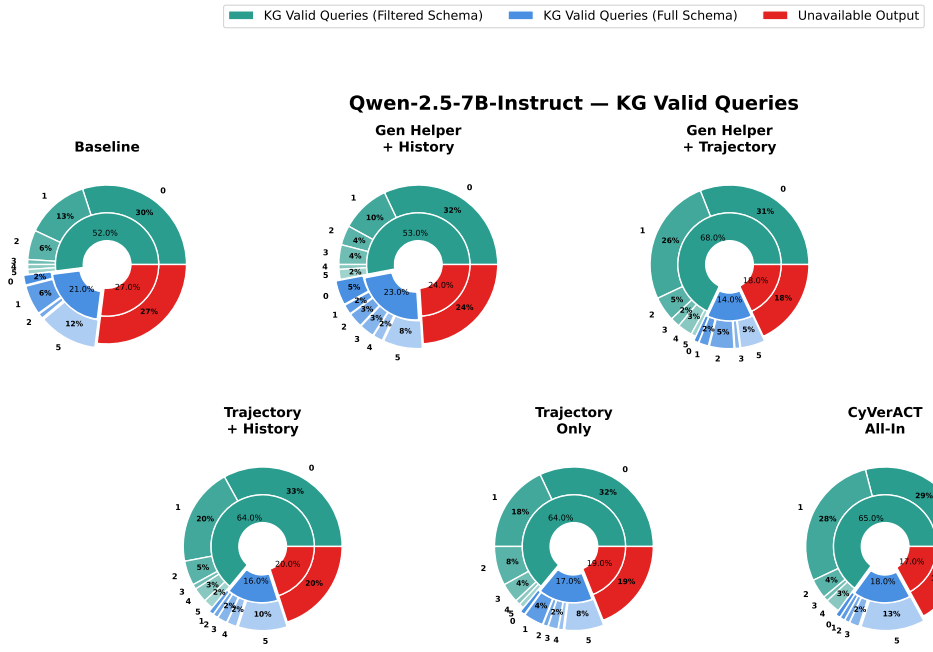
Παρόμοια διαφοροποίηση παρατηρείται και όταν εξετάζεται η ύπαρξη μερικής συμφωνίας των αποτελεσμάτων. Στη Baseline, το 59.1% των έγκυρων ερωτημάτων παρουσιάζει θετική τιμή Jaccard Output. Το ποσοστό αυξάνεται σε 62.1% με τη Gen Helper + History, ενώ η υψηλότερη τιμή καταγράφεται στη CyVerACT All-In, με 68.8%. Αντίθετα, ορισμένες διαμορφώσεις που εμφανίζουν υψηλή εγκυρότητα, όπως η Gen Helper + Trajectory, δεν παρουσιάζουν αντίστοιχη αύξηση στη μερική συμφωνία των αποτελεσμάτων.

Συνολικά, στο Llama-3.1-8B-Instruct οι μηχανισμοί μνήμης επηρεάζουν διαφορετικά την τεχνική εγκυρότητα και τις μετρικές αποτελέσματος. Η Gen Helper + Trajectory επιτυγχάνει την υψηλότερη KG Valid, ενώ οι υψηλότερες αναλογίες Pass@1 Output και θετικής Jaccard Output εμφανίζονται στη CyVerACT All-In. Επομένως, δεν προκύπτει μία διαμόρφωση που να υπερέχει ταυτόχρονα σε όλες τις διαστάσεις της απόδοσης.

Παράλληλα, οι εξωτερικοί δακτύλιοι των τριών διαγραμμάτων δείχνουν ότι οι μεγαλύτερες συγκεντρώσεις επιτυχημένων περιπτώσεων εμφανίζονται στις πρώτες διορθωτικές επαναλήψεις, κυρίως στις τιμές 0, 1 και 2. Οι μεταγενέστερες επαναλήψεις εξακολουθούν να συνεισφέρουν πρόσθετες επιτυχίες, αλλά με μικρότερη συχνότητα. Η εικόνα αυτή είναι συμβατή με την ήδη υψηλή βασική επίδοση του μοντέλου, το οποίο σε μεγάλο μέρος των επιτυχημένων περιπτώσεων καταλήγει σε λύση στα πρώτα στάδια της διορθωτικής διαδικασίας.

4.3.2 Qwen-2.5-7B-Instruct

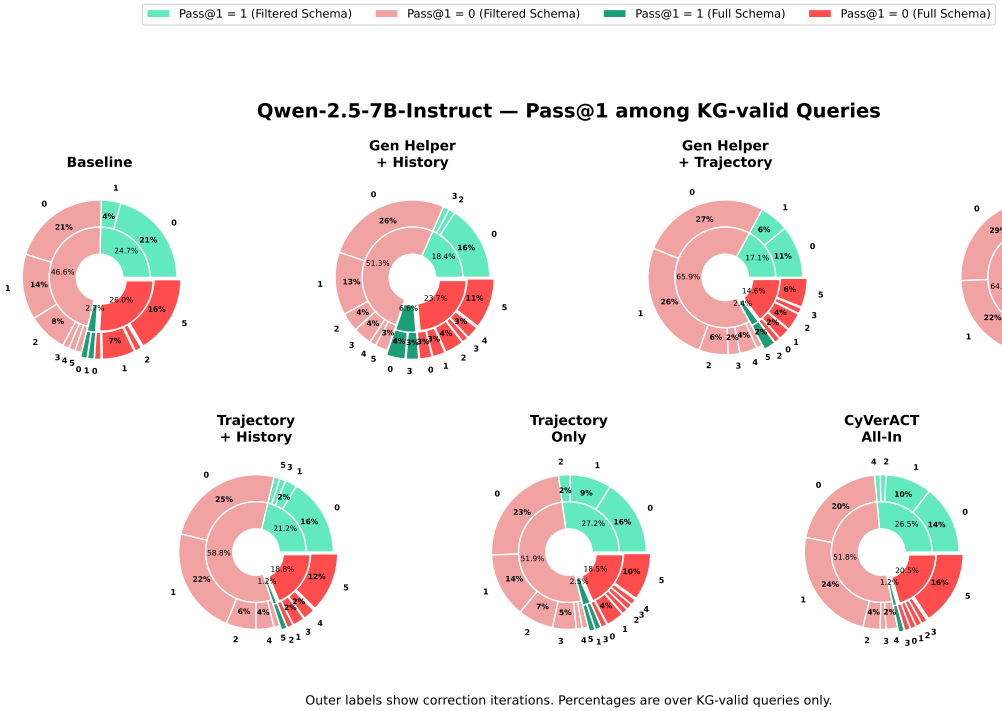
Η κατανομή των έγκυρων ερωτημάτων ανά διαμόρφωση και διορθωτική επανάληψη παρουσιάζεται στο Σχήμα 4.6.



Σχήμα 4.6: Κατανομή των περιπτώσεων που καταλήγουν σε KG Valid ερώτημα Cypher για τις επτά πειραματικές διαμορφώσεις του Qwen-2.5-7B-Instruct, με διάκριση μεταξύ φιλτραρισμένου και πλήρους σχήματος και ανάλυση των διορθωτικών επαναλήψεων.

Στο Qwen-2.5-7B-Instruct η βασική διαμόρφωση καταλήγει σε KG Valid ερώτημα Cypher στο 73% των ερωτήσεων αξιολόγησης. Η ενεργοποίηση της τροχιάς οδηγεί σε σαφέστερη βελτίωση από ό,τι στο προηγούμενο μοντέλο: η Trajectory Only φθάνει το 81%, η Trajectory + History το 80% και η Gen Helper + Trajectory το 82%. Η υψηλότερη εγκυρότητα καταγράφεται στη CyVerACT All-In, με 83%.

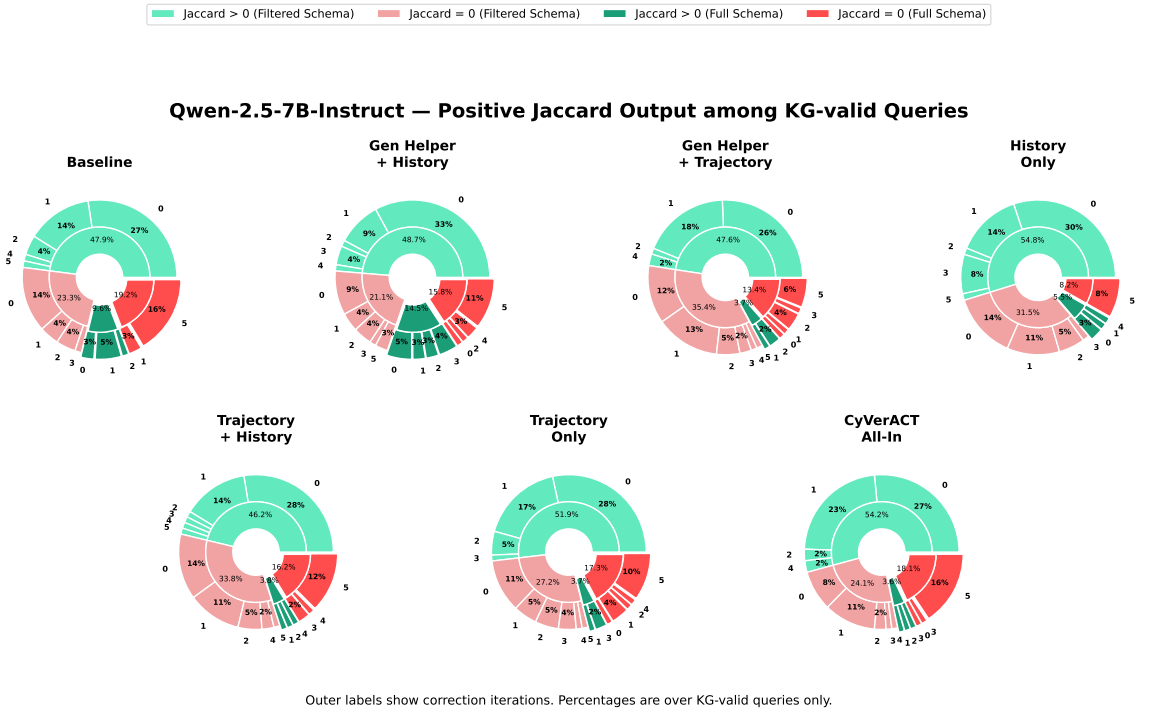
Η συμπεριφορά των διαμορφώσεων ως προς την πλήρη συμφωνία των αποτελεσμάτων παρουσιάζεται στο Σχήμα 4.7.



Σχήμα 4.7: Κατανομή της Pass@1 Output μεταξύ των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher για τις επτά διαμορφώσεις του Qwen-2.5-7B-Instruct. Ο εξωτερικός δακτύλιος παρουσιάζει τις αντίστοιχες διορθωτικές επαναλήψεις.

Στη Pass@1 Output, η Baseline καταγράφει 20 πλήρεις συμφωνίες μεταξύ 73 έγκυρων ερωτημάτων, δηλαδή 27.4%. Η υψηλότερη επίδοση σε απόλυτο αριθμό και αναλογία εμφανίζεται στη Trajectory Only, με 24 πλήρεις συμφωνίες μεταξύ 81 έγκυρων ερωτημάτων (29.6%). Η CyVerACT All-In καταγράφει 27.7%, ενώ η Gen Helper + Trajectory, παρά την υψηλή εγκυρότητα, εμφανίζει χαμηλότερη αναλογία πλήρους συμφωνίας, 19.5%.

Συμπληρωματικά, το Σχήμα 4.8 παρουσιάζει την κατανομή των περιπτώσεων με θετική Jaccard Output μέσα στο σύνολο των έγκυρων ερωτημάτων.



Σχήμα 4.8: Κατανομή θετικής Jaccard Output μεταξύ των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher για τις επτά διαμορφώσεις του Qwen-2.5-7B-Instruct. Ο εξωτερικός δακτύλιος παρουσιάζει τις αντίστοιχες διορθωτικές επαναλήψεις.

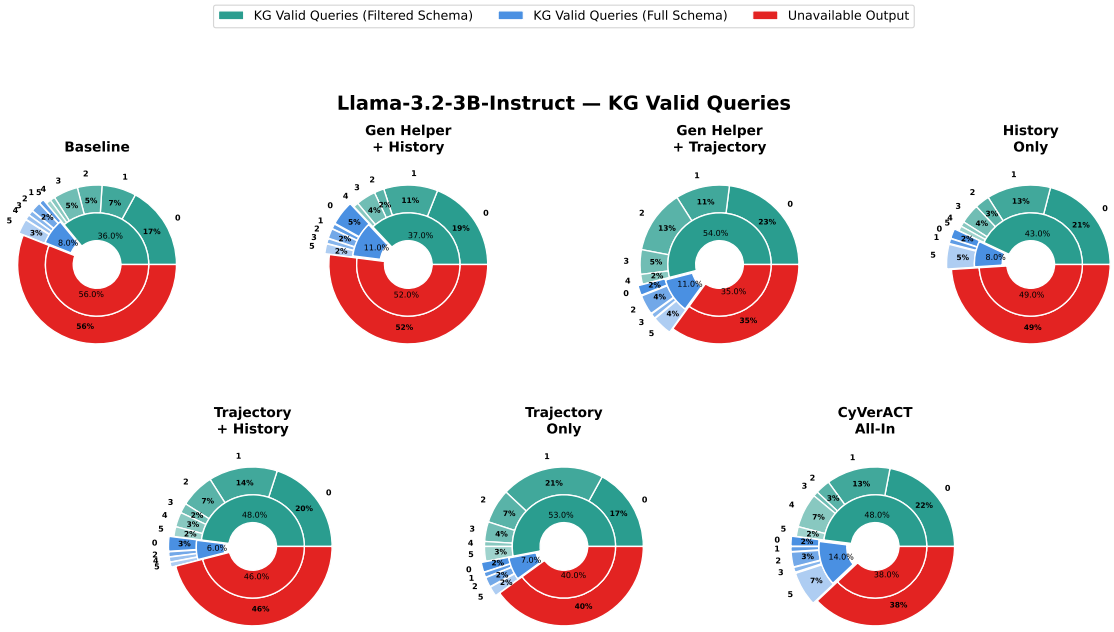
Ως προς τη θετική Jaccard Output, η Baseline καταγράφει ποσοστό 57.5% επί των έγκυρων περιπτώσεων. Η υψηλότερη αναλογία παρατηρείται στη Gen Helper + History, με 63.2%, ενώ η History Only φθάνει το 60.3%. Η CyVerACT All-In παρουσιάζει 57.8%, τιμή πολύ κοντά στη βασική διαμόρφωση, παρότι έχει προηγουμένως αυξήσει σημαντικά τον συνολικό αριθμό των έγκυρων ερωτημάτων.

Συνολικά, στο Qwen-2.5-7B-Instruct η θετική επίδραση της ανάκτησης προηγούμενων τροχιών είναι πιο εμφανής ως προς την τεχνική εγκυρότητα. Οι διαμορφώσεις που αξιοποιούν Trajectory εμφανίζουν υψηλές τιμές KG Valid, ενώ η Trajectory Only παρουσιάζει και την υψηλότερη αναλογία Pass@1 Output μέσα στο έγκυρο υποσύνολο. Στη θετική Jaccard Output, αντίθετα, η υψηλότερη αναλογία καταγράφεται από τη Gen Helper + History. Η διαφοροποίηση αυτή επιβεβαιώνει ότι η βελτίωση της τεχνικής εγκυρότητας δεν συνεπάγεται απαραίτητα αντίστοιχη βελτίωση σε κάθε μετρική αποτελέσματος.

Ως προς τη διαδικασία διόρθωσης, και στα τρία διαγράμματα οι μεγαλύτερες συγκεντρώσεις επιτυχημένων περιπτώσεων εμφανίζονται στις πρώτες επαναλήψεις, ιδιαίτερα στις τιμές 0–2. Οι επόμενες επαναλήψεις εξακολουθούν να συνεισφέρουν πρόσθετες επιτυχίες, αλλά σε μικρότερη έκταση.

4.3.3 Llama-3.2-3B-Instruct

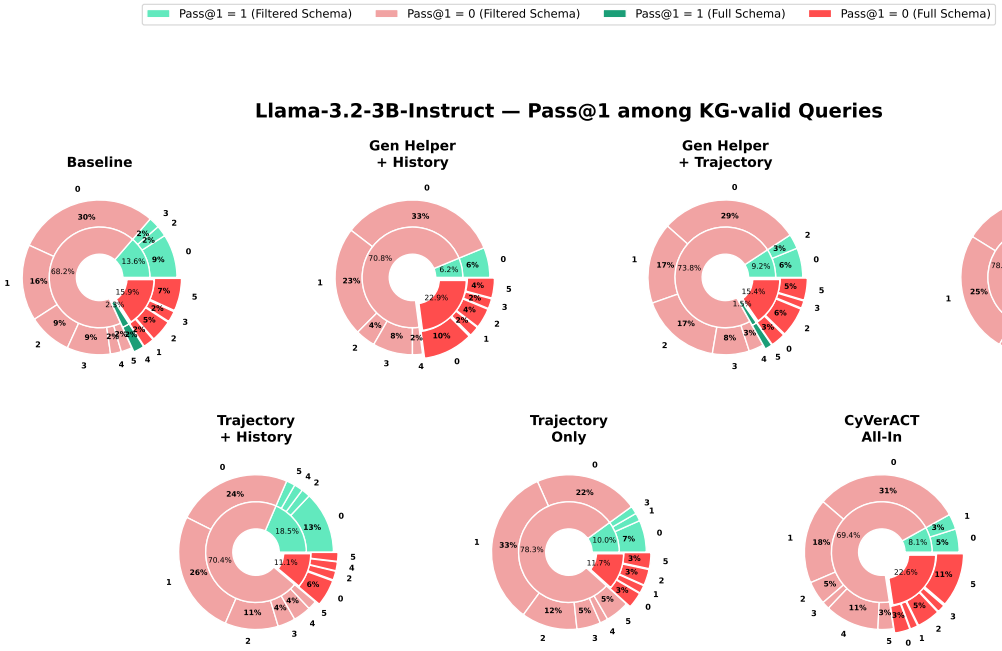
Η κατανομή των έγκυρων ερωτημάτων ανά διαμόρφωση και διορθωτική επανάληψη παρουσιάζεται στο Σχήμα 4.9.



Σχήμα 4.9: Κατανομή των περιπτώσεων που καταλήγουν σε KG Valid ερώτημα Cypher για τις επτά πειραματικές διαμορφώσεις του Llama-3.2-3B-Instruct, με διάκριση μεταξύ φιλτραρισμένου και πλήρους σχήματος και ανάλυση των διορθωτικών επαναλήψεων.

Το Llama-3.2-3B-Instruct παρουσιάζει τη χαμηλότερη αρχική εγκυρότητα μεταξύ των τριών μοντέλων, με τη Baseline να καταλήγει σε KG Valid ερώτημα Cypher στο 44% των ερωτήσεων αξιολόγησης. Η ενεργοποίηση των μηχανισμών μνήμης οδηγεί σε αισθητές μεταβολές: η History Only φθάνει το 51%, η Trajectory + History το 54%, η Trajectory Only το 60% και η CyVerACT All-In το 62%. Η υψηλότερη τιμή καταγράφεται στη Gen Helper + Trajectory, με 65%, δηλαδή αύξηση 21 ποσοστιαίων μονάδων σε σχέση με τη βασική διαμόρφωση.

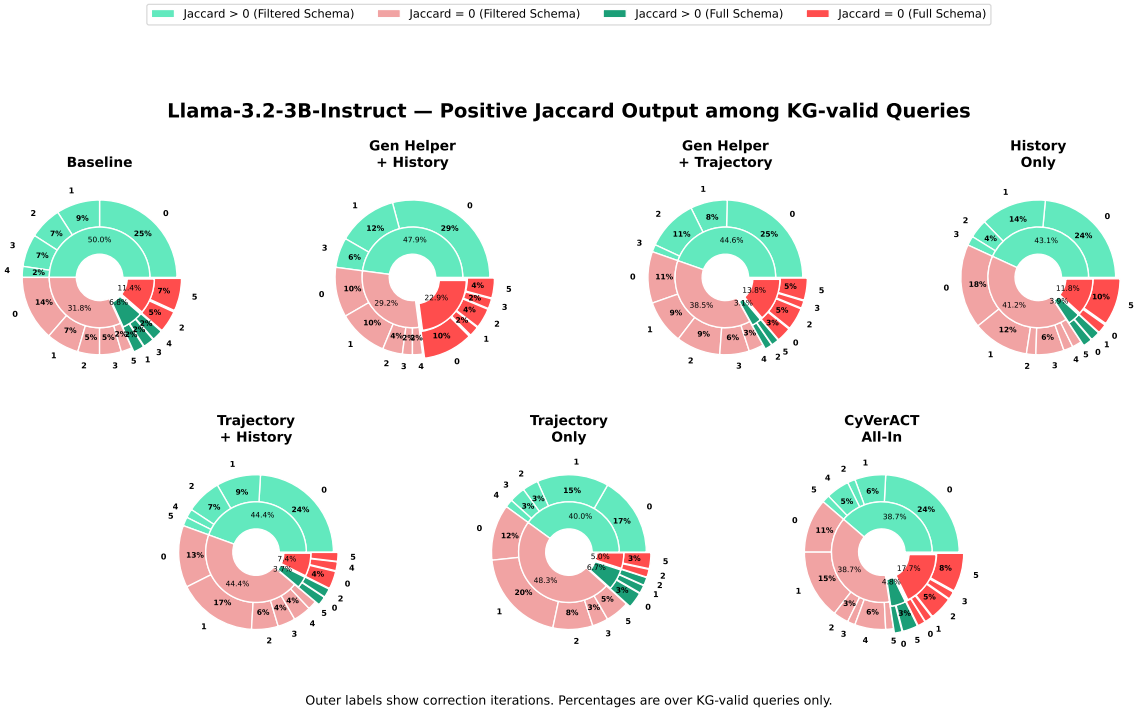
Η αντίστοιχη εικόνα ως προς την πλήρη συμφωνία των αποτελεσμάτων παρουσιάζεται στο Σχήμα 4.10.



Σχήμα 4.10: Κατανομή της Pass@1 Output μεταξύ των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher για τις επτά διαμορφώσεις του Llama-3.2-3B-Instruct. Ο εξωτερικός δακτύλιος παρουσιάζει τις αντίστοιχες διορθωτικές επαναλήψεις.

Η σημαντική αύξηση της εγκυρότητας δεν μεταφέρεται στον ίδιο βαθμό στην πλήρη συμφωνία των αποτελεσμάτων. Η Baseline παρουσιάζει 7 πλήρεις συμφωνίες μεταξύ 44 έγκυρων ερωτημάτων, δηλαδή 15.9%. Η καλύτερη αναλογία εμφανίζεται στη Trajectory + History, με 10 πλήρεις συμφωνίες μεταξύ 54 έγκυρων ερωτημάτων (18.5%). Αντίθετα, η Gen Helper + Trajectory, παρότι παράγει το μεγαλύτερο πλήθος έγκυρων ερωτημάτων, παρουσιάζει πλήρη συμφωνία στο 10.8% του έγκυρου υποσυνόλου.

Συμπληρωματικά, το Σχήμα 4.11 παρουσιάζει τις περιπτώσεις με θετική Jaccard Output, ώστε να εξεταστεί η ύπαρξη τουλάχιστον μερικής επικάλυψης των αποτελεσμάτων.



Σχήμα 4.11: Κατανομή θετικής Jaccard Output μεταξύ των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher για τις επτά διαμορφώσεις του Llama-3.2-3B-Instruct. Ο εξωτερικός δακτύλιος παρουσιάζει τις αντίστοιχες διορθωτικές επαναλήψεις.

Αντίστοιχη εικόνα εμφανίζεται στη Positive Jaccard Output. Η Baseline παρουσιάζει θετική επικάλυψη στο 56.8% των έγκυρων περιπτώσεων, ενώ οι διαμορφώσεις μνήμης κινούνται σε χαμηλότερα ποσοστά. Ωστόσο, επειδή αρκετές από αυτές αυξάνουν σημαντικά το συνολικό πλήθος των KG Valid ερωτημάτων, η χαμηλότερη αναλογία στο έγκυρο υποσύνολο δεν συνεπάγεται απαραίτητα μικρότερο απόλυτο αριθμό περιπτώσεων με θετική επικάλυψη. Ενδεικτικά, η Gen Helper + Trajectory καταγράφει 31 περιπτώσεις με θετική Jaccard Output, έναντι 25 στη Baseline.

Συνολικά, στο Llama-3.2-3B-Instruct παρατηρείται η ισχυρότερη επίδραση των μηχανισμών μνήμης στην τεχνική εγκυρότητα. Η Gen Helper + Trajectory αυξάνει την KG Valid από 44% σε 65%, χωρίς όμως η μεγάλη αυτή βελτίωση να μεταφέρεται στον ίδιο βαθμό στις μετρικές αποτελέσματος. Η υψηλότερη αναλογία Pass@1 Output μέσα στο έγκυρο υποσύνολο εμφανίζεται στη Trajectory + History, ενώ στη θετική Jaccard Output καμία διαμόρφωση μνήμης δεν υπερβαίνει την αναλογία της Baseline. Παράλληλα, η αύξηση του συνολικού πλήθους των έγκυρων ερωτημάτων μπορεί να οδηγεί σε μεγαλύτερο απόλυτο αριθμό θετικών αποτελεσμάτων, όπως παρατηρείται στη Gen Helper + Trajectory.

Οι επιτυχημένες περιπτώσεις εμφανίζονται περισσότερο κατανεμημένες μεταξύ των διορθωτικών επαναλήψεων σε σχέση με το Llama-3.1-8B-Instruct, γεγονός συμβατό με τη χαμηλότερη αρχική επίδοση του μικρότερου μοντέλου. Παρ' όλα αυτά, και εδώ οι μεγαλύτερες συγκεντρώσεις επιτυχιών εντοπίζονται κυρίως στις πρώτες επαναλήψεις,

ιδιαίτερα στις τιμές 0–2, ενώ οι πιο όψιμες επαναλήψεις συνεισφέρουν μικρότερο αριθμό επιτυχημένων περιπτώσεων.

4.4 Επίδραση των Μηχανισμών Μνήμης

Με βάση τα αποτελέσματα της προηγούμενης ενότητας, η ανάλυση μετατοπίζεται στη συνέχεια από την εξέταση κάθε γλωσσικού μοντέλου ξεχωριστά στη συγκριτική αξιολόγηση των επιμέρους μηχανισμών μνήμης. Εξετάζονται η επίδρασή τους στην τεχνική εγκυρότητα και στην ορθότητα των αποτελεσμάτων, η συμπεριφορά της διορθωτικής διαδικασίας, ο πρόσθετος χρόνος εκτέλεσης και η διαφοροποίηση των αποτελεσμάτων ως προς τη δυσκολία των ερωτημάτων.

4.4.1 Επίδραση στην Εγκυρότητα των Παραγόμενων Ερωτημάτων

Η σαφέστερη επίδραση των μηχανισμών μνήμης παρατηρείται στη μετρική KG Valid. Η έκταση της μεταβολής διαφοροποιείται σημαντικά μεταξύ των τριών μοντέλων. Το Llama-3.1-8B-Instruct ξεκινά ήδη από υψηλή επίδοση, με 88% στη Baseline, και η υψηλότερη τιμή που επιτυγχάνεται από τις διαμορφώσεις μνήμης είναι 91% με τη Gen Helper + Trajectory. Αντίθετα, στο Qwen-2.5-7B-Instruct η καλύτερη διαμόρφωση αυξάνει την εγκυρότητα από 73% σε 83%, ενώ στο Llama-3.2-3B-Instruct η αντίστοιχη μεταβολή είναι από 44% σε 65%.

Η σύγκριση της βασικής διαμόρφωσης με την καλύτερη επίδοση ανά μοντέλο συνοψίζεται στον Πίνακα 4.8.

Πίνακας 4.8: Σύγκριση της βασικής και της καλύτερης επίδοσης KG Valid για κάθε γλωσσικό μοντέλο.

Μοντέλο	**Baseline**	**Καλύτερη επίδοση**	**Μεταβολή**	**Διαμόρφωση**
Llama-3.1-8B-Instruct	88%	91%	$+3$ π.μ.	Gen Helper + Trajectory
Qwen-2.5-7B-Instruct	73%	83%	$+10$ π.μ.	CyVerACT All-In
Llama-3.2-3B-Instruct	44%	65%	$+21$ π.μ.	Gen Helper + Trajectory

Η μεγαλύτερη μεταβολή καταγράφεται στο Llama-3.2-3B-Instruct, όπου η βελτίωση ανέρχεται σε 21 ποσοστιαίες μονάδες. Αντίθετα, στο Llama-3.1-8B-Instruct, το οποίο ξεκινά από σαφώς υψηλότερη τιμή Baseline, η μέγιστη βελτίωση περιορίζεται στις 3 ποσοστιαίες μονάδες. Το αποτέλεσμα υποδεικνύει ότι, στο συγκεκριμένο πειραματικό σύνολο, οι μηχανισμοί μνήμης παρουσιάζουν μεγαλύτερο περιθώριο συνεισφοράς όταν η αρχική ικανότητα του μοντέλου να παράγει τεχνικά έγκυρα ερωτήματα είναι χαμηλότερη.

Παράλληλα, δεν προκύπτει μία ενιαία διαμόρφωση που να υπερέχει σε όλα τα μοντέλα. Η Gen Helper + Trajectory επιτυγχάνει την υψηλότερη εγκυρότητα στα δύο μοντέλα της οικογένειας Llama, ενώ στο Qwen-2.5-7B-Instruct η υψηλότερη τιμή καταγράφεται στη CyVerACT All-In. Ωστόσο, η διαφορά μεταξύ CyVerACT All-In και Gen Helper + Trajectory στο συγκεκριμένο μοντέλο είναι μόλις μία ποσοστιαία μονάδα (83% έναντι 82%).

Πέρα από τη μέγιστη επίδοση ανά μοντέλο, εξετάστηκε και η επίδραση των επιμέρους μηχανισμών. Οι δύο διαμορφώσεις στις οποίες χρησιμοποιείται Trajectory χωρίς History, Trajectory Only και Gen Helper + Trajectory, είναι οι μόνες που παρουσιάζουν θετική μεταβολή της KG Valid και στα τρία γλωσσικά μοντέλα. Επιπλέον, με σταθερή την παρουσία του Gen Helper, η χρήση Trajectory αντί History αυξάνει την εγκυρότητα από 87% σε 91% στο Llama-3.1-8B-Instruct, από 76% σε 82% στο Qwen-2.5-7B-Instruct και από 48% σε 65% στο Llama-3.2-3B-Instruct. Επιπρόσθετα, η προσθήκη του Gen Helper στη διαμόρφωση Trajectory Only βελτιώνει την KG Valid και στα τρία μοντέλα. Τα αποτελέσματα αυτά καθιστούν την ανάκτηση προηγούμενων τροχιών τον μηχανισμό με την πιο συνεπή θετική επίδραση στην τεχνική εγκυρότητα, ενώ ο συνδυασμός του Gen Helper ενισχύει περαιτέρω την εγκυρότητα.

Αντίθετα, το βραχυπρόθεσμο History δεν εμφανίζει αντίστοιχα συνεπή επίδραση. Η προσθήκη του στη Trajectory Only μειώνει την KG Valid και στα τρία μοντέλα, ενώ στο Llama-3.1-8B-Instruct όλες οι διαμορφώσεις που περιλαμβάνουν History βρίσκονται κάτω από τη Baseline. Μια πιθανή ερμηνεία είναι ότι, ειδικά σε ένα μοντέλο που εμφανίζει ήδη υψηλή βασική εγκυρότητα, το πρόσθετο ιστορικό αυξάνει την ποσότητα του διαθέσιμου πλαισίου χωρίς να παρέχει πάντοτε εξίσου χρήσιμη καθοδήγηση. Η ερμηνεία αυτή αποτελεί παρατήρηση της συγκεκριμένης πειραματικής συμπεριφοράς και δεν μπορεί να επιβεβαιωθεί αιτιακά από τα διαθέσιμα πειράματα.

4.4.2 Επίδραση στην Ορθότητα των Αποτελεσμάτων

Η επίδραση της μνήμης στις μετρικές αποτελέσματος είναι πιο περιορισμένη από τη μεταβολή που παρατηρείται στη KG Valid. Για την άμεση σύγκριση των τριών μοντέλων, στην παρούσα ανάλυση η Pass@1 Output εκφράζεται ως ποσοστό επί του κοινού συνόλου των 100 ερωτημάτων αξιολόγησης, σε αντίθεση με τα διαγράμματα της προηγούμενης ενότητας, όπου παρουσιάστηκε ως ποσοστό μόνο επί του υποσυνόλου των περιπτώσεων που κατέληξαν σε KG Valid ερώτημα Cypher.

Στο Llama-3.1-8B-Instruct, η βασική διαμόρφωση επιτυγχάνει πλήρη συμφωνία αποτελέσματος σε 19 ερωτήματα, ενώ η υψηλότερη επίδοση είναι 24, τόσο για τη Gen Helper + History όσο και για τη CyVerACT All-In. Στο Qwen-2.5-7B-Instruct η αντίστοιχη μεταβολή είναι από 20 σε 24 ερωτήματα, με καλύτερη διαμόρφωση τη Trajectory Only. Τέλος, στο Llama-3.2-3B-Instruct η Pass@1 Output αυξάνεται από 7 σε 10 ερωτήματα, με τη

Trajectory + History να εμφανίζει την υψηλότερη τιμή.

Πίνακας 4.9: Σύγκριση της βασικής και της καλύτερης επίδοσης Pass@1 Output επί των 100 ερωτημάτων αξιολόγησης.

Μοντέλο	**Baseline**	**Καλύτερη επίδοση**	**Μεταβολή**	**Διαμόρφωση**
Llama-3.1-8B-Instruct	19%	24%	+5 π.μ.	Gen Helper + History / All-In
Qwen-2.5-7B-Instruct	20%	24%	+4 π.μ.	Trajectory Only
Llama-3.2-3B-Instruct	7%	10%	+3 π.μ.	Trajectory + History

Η διαφορά μεταξύ των δύο πινάκων είναι χαρακτηριστική. Στο Llama-3.2-3B-Instruct, για παράδειγμα, η μέγιστη αύξηση της KG Valid είναι 21 ποσοστιαίες μονάδες, ενώ η αντίστοιχη αύξηση της Pass@1 Output είναι μόλις 3. Η διαφοροποίηση αυτή οφείλεται στο γεγονός ότι η KG Valid αξιολογεί την τεχνική εγκυρότητα του παραγόμενου ερωτήματος ως προς τη σύνταξη, το σχήμα και την πρόσβαση σε ιδιότητες, μέσω των ελεγκτών του CyVer, και όχι τη σημασιολογική αντιστοίχιση του ερωτήματος Cypher με την αρχική ερώτηση φυσικής γλώσσας. Συνεπώς, ένα ερώτημα μπορεί να είναι πλήρως έγκυρο ως προς τον γράφο γνώσης, χωρίς να εκφράζει απαραίτητα την επιθυμητή σημασιολογία ή να επιστρέφει το ίδιο αποτέλεσμα με το ερώτημα αναφοράς. Τα αποτελέσματα δείχνουν επομένως ότι η μνήμη ενισχύει ιδιαίτερα την τεχνική εγκυρότητα της παραγωγής, χωρίς η βελτίωση αυτή να μεταφέρεται στον ίδιο βαθμό στην ορθότητα του αποτελέσματος εκτέλεσης.

Η διαφοροποίηση αυτή είναι συνεπής και με τον σχεδιασμό της πρακτορικής ροής. Οι ενεργοί ελεγκτές του CyVer αξιολογούν τη συντακτική εγκυρότητα, τη συμβατότητα με το σχήμα και τη χρήση ιδιοτήτων, χωρίς να υπάρχει ξεχωριστός σημασιολογικός ελεγκτής που να συγκρίνει το τελικό ερώτημα Cypher με την πρόθεση της αρχικής ερώτησης ή με το αποτέλεσμα του ερωτήματος αναφοράς. Οι Pass@1 Output και Jaccard Output χρησιμοποιούνται επομένως ως εξωτερικές μετρικές αξιολόγησης και δεν παρέχουν ανατροφοδότηση στην ίδια την πρακτορική ροή. Οποιαδήποτε βελτίωση στις συγκεκριμένες μετρικές αποτελεί συνεπώς έμμεση συνέπεια της παραγωγής ή της διόρθωσης του ερωτήματος και όχι αποτέλεσμα μηχανισμού που βελτιστοποιεί άμεσα τη σημασιολογική αντιστοίχιση.

Αντίστοιχη διαφοροποίηση παρατηρείται στη Positive Jaccard Output. Χαρακτηριστική είναι η Gen Helper + Trajectory για το Llama-3.2-3B-Instruct: παρότι το ποσοστό θετικής Jaccard Output μέσα στο έγκυρο υποσύνολο είναι χαμηλότερο από εκείνο της Baseline, ο απόλυτος αριθμός θετικών περιπτώσεων αυξάνεται από 25 σε 31, καθώς έχει προηγουμένως αυξηθεί σημαντικά το πλήθος των έγκυρων ερωτημάτων. Η παρατήρηση αυτή αναδεικνύει τη σημασία της συνεξέτασης τόσο των αναλογιών όσο και των

απόλυτων αριθμών των επιτυχημένων περιπτώσεων.

4.4.3 Επίδραση στη Διαδικασία Διόρθωσης

Οι εξωτερικοί δακτύλιοι των διαγραμμάτων της Ενότητας 4.3 παρέχουν συμπληρωματική πληροφορία για το στάδιο της διορθωτικής διαδικασίας στο οποίο προκύπτουν οι επιτυχημένες περιπτώσεις. Συνολικά, τόσο στα διαγράμματα KG Valid όσο και στις περιπτώσεις Pass@1 Output και θετικής Jaccard Output, οι μεγαλύτερες συγκεντρώσεις επιτυχιών εμφανίζονται στις περισσότερες διαμορφώσεις κατά τις πρώτες διορθωτικές επαναλήψεις, κυρίως στις τιμές 0, 1 και 2. Οι μεταγενέστερες επαναλήψεις εξακολουθούν να οδηγούν σε πρόσθετες επιτυχίες, εμφανίζονται όμως γενικά με μικρότερη συχνότητα. Η τιμή 0 αντιστοιχεί στις περιπτώσεις στις οποίες η λύση προέκυψε χωρίς να απαιτηθεί ενεργοποίηση του Cypher Corrector στη συγκεκριμένη φάση.

Επιπλέον, σημαντικό μέρος των επιτυχημένων περιπτώσεων ολοκληρώνεται κατά τη χρήση του φιλτραρισμένου σχήματος, ενώ η μετάβαση στο πλήρες σχήμα επιτρέπει την επίλυση πρόσθετων περιπτώσεων που δεν ολοκληρώθηκαν επιτυχώς στην πρώτη φάση. Το μοτίβο αυτό υποδεικνύει ότι, όταν η πρακτορική ροή οδηγείται σε επιτυχημένη λύση, αυτή προκύπτει συχνότερα στα πρώτα στάδια της διορθωτικής διαδικασίας.

Πέρα από την κατανομή των επιτυχημένων περιπτώσεων ανά επανάληψη, εξετάστηκε και ο μέσος συνολικός αριθμός ενεργοποιήσεων του Cypher Corrector για τα ερωτήματα που κατέληξαν σε KG Valid αποτέλεσμα. Σε αντίθεση με τους εξωτερικούς δακτυλίους των προηγούμενων διαγραμμάτων, όπου οι διορθωτικές επαναλήψεις παρουσιάζονται διακριτά για τη φάση του φιλτραρισμένου και του πλήρους σχήματος, στον συγκεκριμένο υπολογισμό λαμβάνεται υπόψη ολόκληρη η διαδρομή εκτέλεσης του ερωτήματος. Επομένως, όταν ένα ερώτημα μεταβαίνει στο πλήρες σχήμα, συνυπολογίζονται οι ενεργοποιήσεις του Cypher Corrector τόσο πριν όσο και μετά την ενεργοποίηση του KG Schema Retriever. Ο υπολογισμός καλύπτει συνεπώς ολόκληρη τη διαδικασία μέχρι την επίτευξη έγκυρης λύσης.

Πίνακας 4.10: Μέσος συνολικός αριθμός ενεργοποιήσεων του Cypher Corrector για τα ερωτήματα που κατέληξαν σε KG Valid αποτέλεσμα, ανά πειραματική διαμόρφωση και γλωσσικό μοντέλο.

Διαμόρφωση	**Llama-3.1-8B Instruct**	**Qwen-2.5-7B Instruct**	**Llama-3.2-3B Instruct**
Baseline	1.78	2.88	2.45
Gen Helper + History	2.14	2.95	2.17
Gen Helper + Trajectory	**1.15**	2.06	2.25
History Only	2.34	**1.90**	2.08
Trajectory + History	2.08	2.52	**1.80**
Trajectory Only	1.34	2.42	1.93
CyVerACT All-In	2.25	2.59	2.82

Όπως φαίνεται στον Πίνακα 4.10, η επίδραση των μηχανισμών μνήμης στον αριθμό των διορθωτικών βημάτων διαφοροποιείται ανάλογα με το γλωσσικό μοντέλο και τη διαμόρφωση. Η Gen Helper + Trajectory παρουσιάζει τον χαμηλότερο μέσο αριθμό ενεργοποιήσεων στο Llama-3.1-8B-Instruct, με 1.15, έναντι 1.78 της Baseline. Στο Qwen-2.5-7B-Instruct η χαμηλότερη τιμή καταγράφεται από τη History Only, με 1.90, ενώ στο Llama-3.2-3B-Instruct από τη Trajectory + History, με 1.80.

Η διαμόρφωση με τον μικρότερο μέσο αριθμό διορθωτικών βημάτων δεν ταυτίζεται επομένως απαραίτητα με εκείνη που επιτυγχάνει τη μεγαλύτερη συνολική KG Valid. Επιπλέον, οι μέσοι όροι του Πίνακα 4.10 υπολογίζονται στο σύνολο των επιτυχημένων ερωτημάτων κάθε διαμόρφωσης χωριστά. Καθώς οι διαφορετικές διαμορφώσεις δεν οδηγούν κατ' ανάγκη τα ίδια ερωτήματα σε έγκυρη λύση, οι τιμές αυτές δεν πρέπει να ερμηνεύονται ως άμεση σύγκριση πάνω σε ταυτόσημο σύνολο περιπτώσεων.

Συνολικά, τα αποτελέσματα δεν αναδεικνύουν μία ενιαία επίδραση της μνήμης στον αριθμό των διορθωτικών επαναλήψεων για όλα τα μοντέλα. Σε ορισμένες διαμορφώσεις παρατηρείται σαφής μείωση των ενεργοποιήσεων του Cypher Corrector, ενώ σε άλλες ο αριθμός τους παραμένει παρόμοιος ή αυξάνεται. Η επίδραση στη διαδικασία διόρθωσης εξαρτάται επομένως τόσο από τον συνδυασμό των ενεργών μηχανισμών μνήμης όσο και από το χρησιμοποιούμενο γλωσσικό μοντέλο.

4.4.4 Επίδραση στον Χρόνο Εκτέλεσης

Για τη συμπληρωματική αξιολόγηση των πειραματικών διαμορφώσεων εξετάστηκε και ο χρόνος εκτέλεσής τους. Κάθε διαμόρφωση αξιολογήθηκε στο ίδιο σύνολο των 100 ερωτημάτων. Ο συνολικός καταγεγραμμένος χρόνος για κάθε εκτέλεση παρουσιάζεται στον Πίνακα 4.11.

Πίνακας 4.11: Συνολικός χρόνος εκτέλεσης των 100 ερωτημάτων για κάθε πειραματική διαμόρφωση και γλωσσικό μοντέλο.

Διαμόρφωση	Llama-3.1-8B Instruct	Qwen-2.5-7B Instruct	Llama-3.2-3B Instruct
Baseline	00:17:20	00:12:28	00:11:50
Gen Helper + History	00:36:11	00:36:29	00:41:12
Gen Helper + Trajectory	00:18:36	00:29:21	00:33:08
History Only	00:26:09	00:31:40	00:32:43
Trajectory + History	00:33:09	00:31:41	00:32:40
Trajectory Only	00:15:34	00:31:19	00:29:20
CyVerACT All-In	00:46:24	00:32:42	00:37:43

Συνολικά, οι 21 πειραματικές εκτελέσεις αντιστοιχούν σε περίπου 10 ώρες και 18 λεπτά καταγεγραμμένου χρόνου εκτέλεσης. Επειδή κάθε εκτέλεση περιλαμβάνει τον ίδιο αριθμό ερωτημάτων, για την ευκολότερη σύγκριση μεταξύ των διαμορφώσεων υπολογίστηκε επίσης ο μέσος χρόνος ανά ερώτημα, όπως παρουσιάζεται στον Πίνακα 4.12.

Πίνακας 4.12: Μέσος χρόνος εκτέλεσης ανά ερώτημα, σε δευτερόλεπτα, για κάθε πειραματική διαμόρφωση και γλωσσικό μοντέλο.

Διαμόρφωση	Llama-3.1-8B Instruct	Qwen-2.5-7B Instruct	Llama-3.2-3B Instruct
Baseline	10.41	7.48	7.10
Gen Helper + History	21.72	21.90	24.73
Gen Helper + Trajectory	11.17	17.61	19.89
History Only	15.69	19.00	19.63
Trajectory + History	19.89	19.02	19.61
Trajectory Only	9.35	18.80	17.61
CyVerACT All-In	27.84	19.63	22.64

Τα αποτελέσματα δείχνουν ότι ο χρόνος εκτέλεσης διαφοροποιείται ανάλογα με το μοντέλο και τη διαμόρφωση μνήμης. Στο Llama-3.1-8B-Instruct, η Gen Helper + Trajectory, η οποία επιτυγχάνει και την υψηλότερη KG Valid, παρουσιάζει μέσο χρόνο 11.17 δευτερολέπτων ανά ερώτημα, πολύ κοντά στα 10.41 δευτερόλεπτα της Baseline. Αντίθετα, η CyVerACT All-In παρουσιάζει αισθητά μεγαλύτερο χρόνο, φτάνοντας τα 27.84 δευτερόλεπτα ανά ερώτημα.

Στο Qwen-2.5-7B-Instruct, η CyVerACT All-In, η οποία καταγράφει την υψηλότερη KG Valid, αυξάνει τον μέσο χρόνο από 7.48 σε 19.63 δευτερόλεπτα. Αντίστοιχα, στο

Llama-3.2-3B-Instruct, η Gen Helper + Trajectory αυξάνει τη KG Valid από 44% σε 65%, ενώ ο μέσος χρόνος αυξάνεται από 7.10 σε 19.89 δευτερόλεπτα ανά ερώτημα.

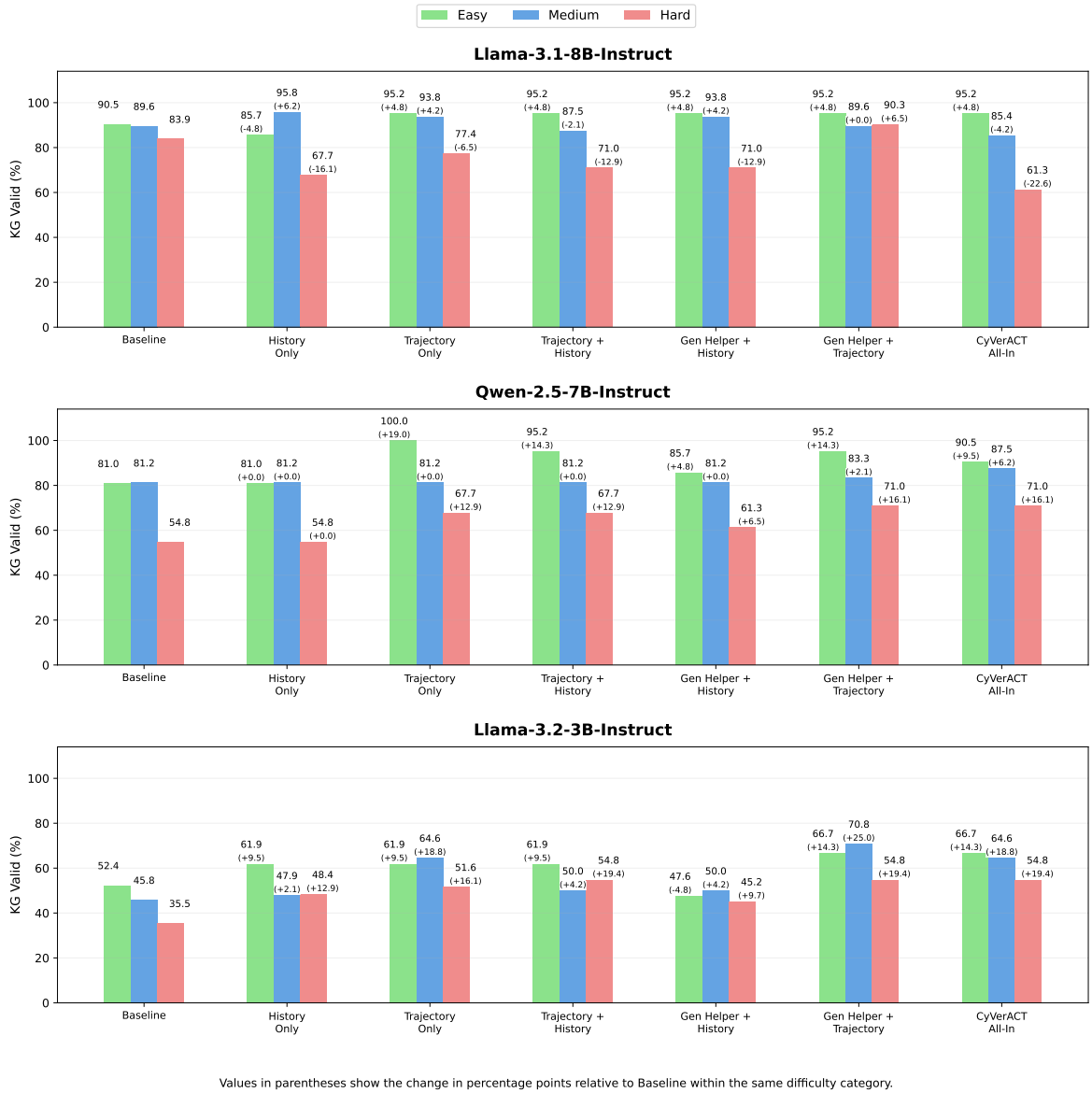
Για την καλύτερη ερμηνεία της χρονικής επιβάρυνσης πραγματοποιήθηκε συμπληρωματική χρονομέτρηση επιμέρους σταδίων της ροής. Η επεξεργασία του βραχυπρόθεσμου History και η αποδιπλοποίηση του ανακτημένου Trajectory παρουσίασαν πρακτικά αμελητέο κόστος. Αντίθετα, σημαντικότερη επιβάρυνση παρατηρήθηκε κατά την πρόσβαση στον γράφο μνήμης. Ενδεικτικά, η ανάκτηση FixRules παρουσίασε μέσο χρόνο της τάξης των 2.1 s ανά κλήση στις δοκιμές χρονομέτρησης, ενώ η ανάκτηση προηγούμενης τροχιάς κυμάνθηκε περίπου μεταξύ 0.1 και 0.4 s ανά κλήση. Η ίδια η αναζήτηση παρόμοιας επιτυχημένης ερώτησης στον Gen Helper ήταν σημαντικά ταχύτερη, ωστόσο ο συνολικός χρόνος του αντίστοιχου σταδίου μπορούσε να προσεγγίσει τα 2.1 s. Οι παρατηρήσεις αυτές υποδηλώνουν ότι σημαντικό μέρος της επιβάρυνσης συνδέεται με την αρχικοποίηση και διαχείριση των συνδέσεων και την επικοινωνία με τη βάση, και όχι αποκλειστικά με τους επιμέρους υπολογισμούς ανάκτησης.

Η επιβάρυνση μπορεί επιπλέον να αυξηθεί έμμεσα όταν μία διαμόρφωση οδηγεί σε περισσότερες ενεργοποιήσεις του Cypher Corrector, καθώς κάθε πρόσθετη επανάληψη συνεπάγεται νέα κλήση του γλωσσικού μοντέλου, επανεκτέλεση των ελεγκτών του CyVer και, ανάλογα με τη διαμόρφωση, νέες ανακτήσεις από τη μνήμη. Ωστόσο, οι εγγραφές νέας εμπειρίας στον γράφο μνήμης δεν συνεισφέρουν στους χρόνους της κύριας αξιολόγησης, καθώς η αποθήκευση ήταν απενεργοποιημένη κατά την εκτέλεση των 100 ερωτημάτων αξιολόγησης.

Τα αποτελέσματα υποδεικνύουν ότι η βελτίωση της τεχνικής εγκυρότητας μπορεί να συνοδεύεται από αύξηση του χρόνου εκτέλεσης, χωρίς όμως η σχέση αυτή να είναι ίδια σε όλα τα μοντέλα και τις διαμορφώσεις. Οι χρόνοι χρησιμοποιούνται επομένως ως συμπληρωματική ένδειξη του κόστους των διαφορετικών πειραματικών επιλογών και όχι ως ανεξάρτητη αξιολόγηση της απόδοσης των γλωσσικών μοντέλων.

4.4.5 Επίδραση ως προς τη Δυσκολία των Ερωτημάτων

Η ανάλυση ανά επίπεδο δυσκολίας επιτρέπει να εξεταστεί αν η επίδραση των μηχανισμών μνήμης μεταβάλλεται ανάλογα με την πολυπλοκότητα των ερωτημάτων. Τα αποτελέσματα παρουσιάζονται στο Σχήμα 4.12.



Σχήμα 4.12: Ποσοστό KG Valid ανά επίπεδο δυσκολίας (Easy, Medium, Hard) για τις επτά πειραματικές διαμορφώσεις και τα τρία γλωσσικά μοντέλα. Οι τιμές σε παρένθεση δηλώνουν τη μεταβολή σε ποσοστιαίες μονάδες ως προς τη Baseline της ίδιας κατηγορίας δυσκολίας.

Στο Llama-3.1-8B-Instruct, η Baseline καταγράφει KG Valid 90.5% στα Easy, 89.6% στα Medium και 83.9% στα Hard ερωτήματα. Στην κατηγορία Easy, η μέγιστη τιμή είναι 95.2% και επιτυγχάνεται από περισσότερες της μίας διαμορφώσεις. Στα Medium, η υψηλότερη τιμή καταγράφεται από τη History Only, με 95.8%, ενώ στα Hard η καλύτερη επίδοση επιτυγχάνεται από τη Gen Helper + Trajectory, με 90.3%. Η μεγαλύτερη βελτίωση ως προς τη Baseline εμφανίζεται επομένως στα Hard ερωτήματα, με αύξηση περίπου 6.5 ποσοστιαίων μονάδων, ενώ παρόμοια μεταβολή καταγράφεται και στα Medium. Στο Llama-3.1-8B-Instruct, το μέγιστο θετικό όφελος ανά κατηγορία δυσκολίας είναι σχετικά περιορισμένο, λόγω της υψηλής αρχικής επίδοσης. Ωστόσο, ορισμένες διαμορφώσεις εμφανίζουν αισθητή υποβάθμιση στα δύσκολα ερωτήματα, με τη CyVerACT All-In να μειώνεται από 83.9% σε 61.3%.

Στο Qwen-2.5-7B-Instruct, η Baseline επιτυγχάνει 81.0% στα Easy, 81.2% στα Medium και 54.8% στα Hard. Στα εύκολα ερωτήματα, η Trajectory Only φθάνει το 100%, παρουσιάζοντας τη μεγαλύτερη επίδοση της κατηγορίας, ενώ η Gen Helper + Trajectory φθάνει το 95.2% και υπερβαίνει τη CyVerACT All-In, η οποία καταγράφει 90.5%. Στα ερωτήματα μέτριας δυσκολίας, η υψηλότερη τιμή καταγράφεται από τη CyVerACT All-In, με 87.5%, έναντι 83.3% της Gen Helper + Trajectory. Στα δύσκολα ερωτήματα, αντίθετα, οι δύο διαμορφώσεις ισοβαθμούν στο 71.0%.

Η ανάλυση αυτή δείχνει ότι η οριακά υψηλότερη συνολική KG Valid της CyVerACT All-In στο Qwen-2.5-7B-Instruct (83% έναντι 82% της Gen Helper + Trajectory) δεν αντιστοιχεί σε γενικευμένη υπεροχή σε όλες τις κατηγορίες δυσκολίας. Η Gen Helper + Trajectory εμφανίζει καλύτερη επίδοση στα Easy, ισοδύναμη επίδοση στα Hard και υπολείπεται μόνο στα Medium. Επομένως, η μικρή συνολική υπεροχή της CyVerACT All-In φαίνεται να προκύπτει κυρίως από την πρόσθετη επίδοση στην κατηγορία Medium, χωρίς να παρέχει αντίστοιχο πρόσθετο όφελος στα δυσκολότερα ερωτήματα.

Ακόμη μεγαλύτερο περιθώριο βελτίωσης εμφανίζει το Llama-3.2-3B-Instruct. Η Baseline καταγράφει 52.4%, 45.8% και 35.5% στις κατηγορίες Easy, Medium και Hard, αντίστοιχα. Στα εύκολα ερωτήματα, οι διαμορφώσεις CyVerACT All-In και Gen Helper + Trajectory επιτυγχάνουν τη μέγιστη τιμή 66.7%. Στα ερωτήματα μέτριας δυσκολίας, η Gen Helper + Trajectory φθάνει το 70.8%, που αντιστοιχεί σε αύξηση 25.0 ποσοστιαίων μονάδων σε σχέση με τη Baseline. Στα δύσκολα ερωτήματα, η μέγιστη τιμή 54.8% επιτυγχάνεται από τις CyVerACT All-In, Gen Helper + Trajectory και Trajectory + History, έναντι 35.5% της Baseline. Η μεγαλύτερη ενίσχυση του μικρότερου μοντέλου εμφανίζεται επομένως στα ερωτήματα μέτριας δυσκολίας, ενώ σημαντική βελτίωση παρατηρείται και στην κατηγορία Hard.

Εξετάζοντας συνολικά τη μεταβολή ως προς τη Baseline ανά κατηγορία δυσκολίας, προκύπτει ότι το σημείο στο οποίο η μνήμη προσφέρει το μεγαλύτερο όφελος δεν είναι κοινό για όλα τα μοντέλα. Στο Llama-3.1-8B-Instruct η μεγαλύτερη βελτίωση εμφανίζεται στα Hard ερωτήματα, στο Qwen-2.5-7B-Instruct στα Easy, με σημαντική ενίσχυση και στα Hard, ενώ στο Llama-3.2-3B-Instruct η μεγαλύτερη αύξηση καταγράφεται στα Medium. Η επίδραση των μηχανισμών μνήμης εξαρτάται επομένως όχι μόνο από το χρησιμοποιούμενο γλωσσικό μοντέλο αλλά και από το είδος και την πολυπλοκότητα του ερωτήματος.

Η Gen Helper + Trajectory αναδεικνύεται ως η διαμόρφωση με την πιο σταθερά υψηλή επίδοση μεταξύ των τριών μοντέλων. Στα δύο μοντέλα της οικογένειας Llama επιτυγχάνει την υψηλότερη συνολική KG Valid, ενώ στο Qwen-2.5-7B-Instruct υπολείπεται της CyVerACT All-In κατά μόλις μία ποσοστιαία μονάδα συνολικά, χωρίς όμως να υστερεί στα δύσκολα ερωτήματα και με καλύτερη επίδοση στα εύκολα. Τα αποτελέσματα αυτά ενισχύουν την εικόνα ότι ο συνδυασμός ανάκτησης επιτυχημένου παραδείγματος στον Cypher Generator και προηγούμενης τροχιάς διόρθωσης αποτελεί την πιο σταθε-

ρά αποδοτική επιλογή στο συγκεκριμένο πειραματικό πλαίσιο, χωρίς να απαιτεί την ταυτόχρονη ενεργοποίηση όλων των διαθέσιμων μηχανισμών μνήμης.

4.5 Παρατηρήσεις επί των Αποτελεσμάτων

Η συνολική εικόνα των πειραμάτων δείχνει ότι η επίδραση της μνήμης εξαρτάται τόσο από το χρησιμοποιούμενο γλωσσικό μοντέλο όσο και από τον τρόπο με τον οποίο η προηγούμενη εμπειρία ενσωματώνεται στην πρακτορική ροή. Η πιο συνεπής θετική τάση παρατηρείται στην τεχνική εγκυρότητα των παραγόμενων ερωτημάτων, με την ανάκτηση προηγούμενων τροχιών να εμφανίζει τη σταθερότερη συμπεριφορά μεταξύ των εξεταζόμενων μηχανισμών. Το περιθώριο βελτίωσης είναι μεγαλύτερο στα μοντέλα με χαμηλότερη αρχική επίδοση, χωρίς όμως η αύξηση της KG Valid να συνοδεύεται πάντοτε από αντίστοιχη αύξηση στις μετρικές αποτελέσματος.

Παράλληλα, οι επιτυχημένες περιπτώσεις εμφανίζονται συχνότερα στις πρώτες διορθωτικές επαναλήψεις, ενώ ούτε ο συνολικός αριθμός ενεργοποιήσεων του Cypher Corrector ούτε ο χρόνος εκτέλεσης μεταβάλλονται με ενιαίο τρόπο σε όλες τις διαμορφώσεις. Συνεπώς, τα αποτελέσματα δεν αναδεικνύουν έναν μοναδικό μηχανισμό μνήμης ως καθολικά ανώτερο, αλλά δείχνουν ότι η αποτελεσματικότητά του εξαρτάται από το μοντέλο, το είδος της ανακτώμενης εμπειρίας και το στάδιο της πρακτορικής ροής στο οποίο αυτή αξιοποιείται.

4.6 Περιορισμοί της Πειραματικής Αξιολόγησης

Παρότι η πειραματική αξιολόγηση επιτρέπει τη συστηματική σύγκριση των διαφορετικών μηχανισμών μνήμης, τα αποτελέσματα πρέπει να ερμηνευθούν λαμβάνοντας υπόψη ορισμένους περιορισμούς του πειραματικού σχεδιασμού.

Αρχικά, η αξιολόγηση πραγματοποιήθηκε αποκλειστικά στη βάση Movies και σε ένα συγκεκριμένο σύνολο ερωτημάτων Text-to-Cypher. Το τελικό σύνολο περιλαμβάνει 100 ερωτήματα αξιολόγησης, ενώ επιπλέον 50 ερωτήματα χρησιμοποιήθηκαν για την αρχική τροφοδότηση της μακροπρόθεσμης μνήμης. Παρότι η δειγματοληψία διατήρησε την παρουσία και την αναλογία διαφορετικών επιπέδων δυσκολίας, το συγκεκριμένο μέγεθος και η χρήση ενός μόνο σχήματος γράφου περιορίζουν τη δυνατότητα γενίκευσης των αποτελεσμάτων σε άλλους γράφους γνώσης, διαφορετικά πεδία εφαρμογής ή σημαντικά μεγαλύτερα σύνολα ερωτημάτων.

Επιπλέον, εξετάστηκαν τρία γλωσσικά μοντέλα, τα Llama-3.1-8B-Instruct, Qwen-2.5-7B-Instruct και Llama-3.2-3B-Instruct. Η επιλογή αυτή επιτρέπει τη σύγκριση μοντέλων διαφορετικής κλίμακας και οικογένειας, δεν επιτρέπει όμως τη γενίκευση των παρατηρούμενων τάσεων στο σύνολο των μεγάλων γλωσσικών μοντέλων. Ειδικότερα, το

εύρημα ότι οι μεγαλύτερες μεταβολές παρατηρούνται στο μοντέλο με τη χαμηλότερη αρχική επίδοση αφορά το συγκεκριμένο πειραματικό πλαίσιο.

Οι πειραματικές εκτελέσεις πραγματοποιήθηκαν με `temperature=0`, ώστε να περιοριστεί η στοχαστικότητα της παραγωγής και να διατηρηθούν όσο το δυνατόν σταθερότερες συνθήκες σύγκρισης. Δεν πραγματοποιήθηκαν, ωστόσο, πολλαπλές ανεξάρτητες επαναλήψεις κάθε συνδυασμού μοντέλου και διαμόρφωσης. Κατά συνέπεια, δεν είναι διαθέσιμη εκτίμηση της διακύμανσης των μετρικών μεταξύ διαφορετικών εκτελέσεων.

Ένας επιπλέον περιορισμός αφορά τον διαχωρισμό μεταξύ του συνόλου μνήμης και του συνόλου αξιολόγησης. Ο διαχωρισμός πραγματοποιήθηκε έτσι ώστε κάθε εγγραφή του επιλεγμένου συνόλου να ανήκει αποκλειστικά είτε στο σύνολο αξιολόγησης είτε στο σύνολο μνήμης. Δεν πραγματοποιήθηκε, ωστόσο, πρόσθετος έλεγχος για λεκτικά ή σημασιολογικά παρόμοιες ερωτήσεις μεταξύ των δύο υποσυνόλων. Κατά μεταγενέστερο έλεγχο εντοπίστηκαν δύο περιπτώσεις στις οποίες το ίδιο λεκτικό ερώτημα φυσικής γλώσσας εμφανίζεται και στα δύο υποσύνολα, παρότι αντιστοιχεί σε διαφορετικές εγγραφές και διαφορετικά ερωτήματα αναφοράς Cypher. Το ζήτημα αυτό αφορά κυρίως τις διαμορφώσεις που χρησιμοποιούν τον Gen Helper, καθώς μια τέτοια περίπτωση μπορεί να παρουσιάζει πολύ υψηλή σημασιολογική ομοιότητα κατά την ανάκτηση. Ο συγκεκριμένος περιορισμός πρέπει επομένως να λαμβάνεται υπόψη κατά την ερμηνεία των αντίστοιχων αποτελεσμάτων.

Οι παράμετροι των μηχανισμών ανάκτησης, όπως τα κατώφλια ομοιότητας και η στάθμιση των επιμέρους συνιστωσών της τροχιάς, διατηρήθηκαν σταθερές σε όλες τις πειραματικές διαμορφώσεις. Η επιλογή αυτή εξασφαλίζει τη συγκρισιμότητα των πειραμάτων, δεν εξετάστηκε όμως εξαντλητικά αν διαφορετικές τιμές των παραμέτρων θα μπορούσαν να βελτιώσουν την απόδοση συγκεκριμένων μοντέλων ή κατηγοριών ερωτημάτων. Επομένως, τα αποτελέσματα αξιολογούν τις συγκεκριμένες υλοποιημένες στρατηγικές και όχι το θεωρητικά βέλτιστο σύνολο παραμέτρων για κάθε μηχανισμό μνήμης.

Οι συνολικοί χρόνοι εκτέλεσης αποτελούν μετρήσεις πραγματικού χρόνου της πλήρους πρακτορικής ροής και επηρεάζονται, εκτός από την εσωτερική επεξεργασία του συστήματος, και από την καθυστέρηση των απομακρυσμένων κλήσεων προς τα γλωσσικά μοντέλα και τις βάσεις δεδομένων. Η συμπληρωματική χρονομέτρηση επιμέρους σταδίων χρησιμοποιήθηκε διαγνωστικά για τον εντοπισμό των κυριότερων πηγών χρονικής επιβάρυνσης και δεν επαναλήφθηκε συστηματικά για όλους τους συνδυασμούς μοντέλων και διαμορφώσεων. Οι επιμέρους χρόνοι πρέπει επομένως να ερμηνεύονται ως ενδεικτικές μετρήσεις και όχι ως ανεξάρτητο benchmark απόδοσης των αντίστοιχων μηχανισμών.

Τέλος, οι χρησιμοποιούμενες μετρικές αποτυπώνουν διαφορετικές αλλά μερικές όψεις της ποιότητας ενός παραγόμενου ερωτήματος Cypher. Η KG Valid αξιολογεί την τεχνική συμβατότητα με τον γράφο, ενώ οι μετρικές αποτελέσματος εξετάζουν

τη συμφωνία με την έξοδο του ερωτήματος αναφοράς Cypher. Παρότι ο συνδυασμός τους παρέχει πληρέστερη εικόνα από μία μεμονωμένη μετρική, δεν εξαλείφει πλήρως το πρόβλημα της αξιολόγησης σημασιολογικά ισοδύναμων αλλά διαφορετικά διατυπωμένων ερωτημάτων Cypher. Για τον λόγο αυτό, οι μετρικές ομοιότητας της μορφής του ερωτήματος χρησιμοποιήθηκαν συμπληρωματικά και τα βασικά συμπεράσματα της αξιολόγησης στηρίζονται κυρίως στη συνεξέταση της τεχνικής εγκυρότητας και των αποτελεσμάτων εκτέλεσης.

Οι παραπάνω περιορισμοί δεν αναιρούν τις συγκριτικές παρατηρήσεις που προέκυψαν υπό τις κοινές πειραματικές συνθήκες, οριοθετούν όμως το πλαίσιο μέσα στο οποίο μπορούν να ερμηνευθούν τα αποτελέσματα της παρούσας αξιολόγησης.

4.7 Σύνοψη Κεφαλαίου

Στο παρόν κεφάλαιο παρουσιάστηκε η πειραματική αξιολόγηση των μηχανισμών μνήμης που ενσωματώθηκαν στην πρακτορική ροή CyVerACT. Εξετάστηκαν η τεχνική εγκυρότητα των παραγόμενων ερωτημάτων, η ορθότητα των αποτελεσμάτων εκτέλεσης, η διαδικασία διορθωτικών επαναλήψεων, ο χρόνος εκτέλεσης και η επίδραση της δυσκολίας των ερωτημάτων. Παρουσιάστηκαν επίσης οι βασικοί περιορισμοί του πειραματικού σχεδιασμού. Τα συνολικά συμπεράσματα που προκύπτουν από τα παραπάνω αποτελέσματα παρουσιάζονται στο επόμενο κεφάλαιο.

Κεφάλαιο 5

Συμπεράσματα και Μελλοντικές Επεκτάσεις

Στο παρόν κεφάλαιο συνοψίζονται τα βασικά συμπεράσματα που προέκυψαν από τον σχεδιασμό, την υλοποίηση και την πειραματική αξιολόγηση της προτεινόμενης επέκτασης μνήμης για την πρακτορική ροή CyVerACT. Ακόμη, παρουσιάζονται κατευθύνσεις που μπορούν να αξιοποιηθούν για την περαιτέρω ανάπτυξη του συστήματος και για την αξιολόγησή του σε ευρύτερα και πιο απαιτητικά πειραματικά περιβάλλοντα.

5.1 Συμπεράσματα

Η παρούσα εργασία επικεντρώθηκε στη διερεύνηση της αξιοποίησης μνήμης σε έναν πρακτορικό μηχανισμό Text-to-Cypher, με στόχο τη χρήση προηγούμενης εμπειρίας τόσο κατά την αρχική παραγωγή ενός ερωτήματος Cypher όσο και κατά την επαναληπτική διαδικασία διόρθωσής του. Ως βάση χρησιμοποιήθηκε η πρακτορική αρχιτεκτονική CyVerACT, η οποία επεκτάθηκε με έναν αρθρωτό μηχανισμό μνήμης που επιτρέπει την αξιοποίηση διαφορετικών μορφών προηγούμενης εμπειρίας σε διακριτούς συνδυασμούς.

Η προτεινόμενη σχεδίαση συνδυάζει πληροφορία της τρέχουσας εκτέλεσης με μακροπρόθεσμη επεισοδιακή εμπειρία. Το βραχυπρόθεσμο History παρέχει στον Cypher Corrector τις προηγούμενες αποτυχημένες προσπάθειες του ίδιου ερωτήματος, ενώ ο γράφος μνήμης της Ενότητας 3.4 διατηρεί προηγούμενες ερωτήσεις, λύσεις, προσπάθειες και σφάλματα. Η μακροπρόθεσμη εμπειρία αξιοποιείται τόσο πριν από την αρχική παραγωγή, μέσω της ανάκτησης παρόμοιου επιτυχημένου παραδείγματος από τον Gen Helper, όσο και κατά τη διόρθωση, μέσω της ανάκτησης προηγούμενων τροχιών. Με τον τρόπο αυτό η μνήμη μπορεί να υποστηρίζει διαφορετικά στάδια της πρακτορικής ροής, χωρίς να απαιτείται η ταυτόχρονη ενεργοποίηση των επιμέρους μηχανισμών.

Η πειραματική αξιολόγηση του Κεφαλαίου 4 πραγματοποιήθηκε στη βάση Movies,

χρησιμοποιώντας τρία γλωσσικά μοντέλα και επτά πειραματικές διαμορφώσεις. Η σύγκριση πραγματοποιήθηκε σε κοινό σύνολο 100 ερωτημάτων αξιολόγησης, ενώ 50 επιπλέον ερωτήματα χρησιμοποιήθηκαν για την αρχική τροφοδότηση της μακροπρόθεσμης μνήμης. Με τον τρόπο αυτό ήταν δυνατή η σύγκριση της βασικής πρακτορικής ροής με διαμορφώσεις στις οποίες οι διαφορετικοί μηχανισμοί μνήμης ενεργοποιούνταν είτε μεμονωμένα είτε συνδυαστικά.

Το βασικότερο πειραματικό εύρημα είναι ότι η προσθήκη μνήμης μπορεί να αυξήσει αισθητά το ποσοστό των ερωτημάτων για τα οποία η πρακτορική ροή καταλήγει σε τεχνικά έγκυρο ερώτημα Cypher. Οι καλύτερες διαμορφώσεις αύξησαν τη KG Valid κατά 3, 10 και 21 ποσοστιαίες μονάδες στα Llama-3.1-8B-Instruct, Qwen-2.5-7B-Instruct και Llama-3.2-3B-Instruct, αντίστοιχα. Η μεγαλύτερη βελτίωση καταγράφηκε επομένως στο μικρότερο μοντέλο, όπου η εγκυρότητα αυξήθηκε από 44% σε 65%. Το αποτέλεσμα αυτό υποδεικνύει ότι, στο συγκεκριμένο πειραματικό πλαίσιο, η αξιοποίηση προηγούμενης εμπειρίας μπορεί να προσφέρει μεγαλύτερο περιθώριο βελτίωσης όταν η αρχική ικανότητα παραγωγής τεχνικά έγκυρων ερωτημάτων είναι χαμηλότερη.

Η βελτίωση της τεχνικής εγκυρότητας δεν συνοδεύτηκε, ωστόσο, από αντίστοιχου μεγέθους αύξηση σε όλες τις μετρικές αποτελέσματος. Η μέγιστη Pass@1 Output αυξήθηκε κατά 5, 4 και 3 ποσοστιαίες μονάδες στα Llama-3.1-8B-Instruct, Qwen-2.5-7B-Instruct και Llama-3.2-3B-Instruct, αντίστοιχα. Οι αντίστοιχες μέγιστες αυξήσεις της KG Valid ήταν 3, 10 και 21 ποσοστιαίες μονάδες. Επομένως, στα Qwen-2.5-7B-Instruct και Llama-3.2-3B-Instruct η βελτίωση της πλήρους συμφωνίας αποτελέσματος ήταν μικρότερη από τη βελτίωση της τεχνικής εγκυρότητας, ενώ στο Llama-3.1-8B-Instruct η σχέση αυτή αντιστράφηκε. Οι μέγιστες τιμές κάθε μετρικής μπορεί να αντιστοιχούν σε διαφορετικές διαμορφώσεις. Το αποτέλεσμα αυτό αναδεικνύει μία βασική διάκριση της αξιολόγησης: η παραγωγή ενός συντακτικά και δομικά έγκυρου ερωτήματος αποτελεί απαραίτητη αλλά όχι επαρκή συνθήκη για την ορθή απάντηση στην αρχική ερώτηση. Η παρούσα υλοποίηση αξιοποιεί προηγούμενη εμπειρία κυρίως για την υποστήριξη της παραγωγής και της τεχνικής διόρθωσης των ερωτημάτων. Μία φυσική επέκταση είναι επομένως η αξιοποίηση αντίστοιχων μηχανισμών μνήμης και για περιπτώσεις σημασιολογικής απόκλισης, ώστε η προηγούμενη εμπειρία να μπορεί να υποστηρίζει όχι μόνο την παραγωγή ερωτημάτων που είναι έγκυρα ως προς τον γράφο, αλλά και την καλύτερη αντιστοίχισή τους με την πρόθεση της αρχικής ερώτησης.

Ως προς τους επιμέρους μηχανισμούς, η ανάκτηση προηγούμενων τροχιών παρουσίασε την πιο συνεπή θετική επίδραση στην τεχνική εγκυρότητα. Ειδικότερα, οι διαμορφώσεις Trajectory Only και Gen Helper + Trajectory, στις οποίες η τροχιά χρησιμοποιείται χωρίς το βραχυπρόθεσμο History, παρουσίασαν θετική μεταβολή της KG Valid και στα τρία γλωσσικά μοντέλα. Αντίθετα, η επίδραση του History παρουσίασε μεγαλύτερη εξάρτηση από το μοντέλο και τον συνδυασμό στον οποίο χρησιμοποιήθηκε.

Επισημαίνεται ότι η διαμόρφωση Gen Helper + Trajectory αναδείχθηκε ως η δια-

μόρφωση με την πιο σταθερά υψηλή επίδοση ως προς την τεχνική εγκυρότητα. Στα δύο μοντέλα της οικογένειας Llama πέτυχε την υψηλότερη συνολική KG Valid, ενώ στο Qwen-2.5-7B-Instruct υπολείφθηκε της CyVerACT All-In κατά μόλις μία ποσοστιαία μονάδα. Η ανάλυση ανά επίπεδο δυσκολίας έδειξε επιπλέον ότι η μικρή αυτή διαφορά δεν αντιστοιχεί σε καθολική υπεροχή της CyVerACT All-In: η Gen Helper + Trajectory παρουσίασε καλύτερη επίδοση στα εύκολα ερωτήματα και ισοδύναμη επίδοση στα δύσκολα, ενώ η διαφορά υπέρ της CyVerACT All-In εμφανίστηκε κυρίως στα ερωτήματα μέτριας δυσκολίας.

Παρ΄ όλα αυτά, οι καλύτερες διαμορφώσεις ως προς τη Pass@1 Output δεν συνέπιπταν πάντοτε με εκείνες που παρήγαγαν τα περισσότερα τεχνικά έγκυρα ερωτήματα. Το γεγονός ότι η ταυτόχρονη ενεργοποίηση όλων των διαθέσιμων μηχανισμών, όπως στη CyVerACT All-In, δεν οδηγεί κατ΄ ανάγκη στη βέλτιστη απόδοση υποστηρίζει τη σχεδιαστική επιλογή της ανεξάρτητης ενεργοποίησης και συνδυαστικής αξιολόγησής τους. Η μεγαλύτερη ποσότητα διαθέσιμης πληροφορίας μνήμης δεν είναι επομένως απαραίτητα ωφέλιμη σε κάθε μοντέλο ή ερώτημα.

Η ανάλυση της διορθωτικής διαδικασίας δεν ανέδειξε μία ενιαία επίδραση της μνήμης στον αριθμό των ενεργοποιήσεων του Cypher Corrector. Οι επιτυχημένες περιπτώσεις συγκεντρώνονται συχνότερα στις πρώτες διορθωτικές επαναλήψεις, κυρίως στις τιμές 0–2, χωρίς όμως οι διαμορφώσεις με την υψηλότερη KG Valid να απαιτούν πάντοτε και τις λιγότερες διορθώσεις. Αντίστοιχα, ο χρόνος εκτέλεσης διαφοροποιήθηκε ανάλογα με το μοντέλο και τη διαμόρφωση, καθώς οι πρόσθετες ανακτήσεις από τον γράφο μνήμης και οι επαναλαμβανόμενες ενεργοποιήσεις της πρακτορικής ροής μπορούν να αυξήσουν το υπολογιστικό κόστος.

Συνολικά, τα αποτελέσματα της εργασίας δείχνουν ότι η διατήρηση και η επιλεκτική ανάκτηση προηγούμενης εμπειρίας μπορούν να αποτελέσουν χρήσιμο συμπλήρωμα μιας πρακτορικής ροής Text-to-Cypher. Η μνήμη δεν υποκαθιστά την ικανότητα του γλωσσικού μοντέλου ούτε εγγυάται από μόνη της τη σημασιολογική ορθότητα του παραγόμενου ερωτήματος. Μπορεί όμως, υπό κατάλληλες συνθήκες, να προσφέρει πρόσθετο πλαίσιο από προηγούμενες εκτελέσεις, να αξιοποιεί επιτυχημένα παραδείγματα και τροχιές διόρθωσης και να βοηθά το σύστημα να καταλήγει συχνότερα σε τεχνικά έγκυρες λύσεις. Η δυνατότητα αυτή είναι χρήσιμη ιδίως για πρακτορικά συστήματα στα οποία οι εκτελέσεις δεν αντιμετωπίζονται ως ανεξάρτητες, αλλά ως πηγή εμπειρίας που μπορεί να αξιοποιηθεί σε επόμενα ερωτήματα.

5.2 Μελλοντικές Επεκτάσεις

Οι περιορισμοί που παρουσιάστηκαν στην Ενότητα 4.6, καθώς και οι παρατηρήσεις που προέκυψαν από την πειραματική αξιολόγηση, δημιουργούν αρκετές δυνατότητες για περαιτέρω επέκταση της παρούσας εργασίας.

Μία πρώτη κατεύθυνση αφορά τη διεύρυνση της πειραματικής αξιολόγησης. Η χρήση ενός μόνο γράφου γνώσης δεν επιτρέπει να διαπιστωθεί σε ποιον βαθμό οι παρατηρούμενες τάσεις γενικεύονται σε διαφορετικά σχήματα και θεματικά πεδία. Η αξιολόγηση σε πολλαπλές βάσεις γνώσης, με μεγαλύτερα και πιο ετερογενή σύνολα ερωτημάτων, θα επέτρεπε την εξέταση της συμπεριφοράς των μηχανισμών μνήμης σε διαφορετικές δομές γράφων, οντότητες, σχέσεις και τύπους ερωτημάτων. Αντίστοιχα, η χρήση περισσότερων γλωσσικών μοντέλων και μοντέλων διαφορετικής κλίμακας θα παρείχε πληρέστερη εικόνα για τη σχέση μεταξύ της βασικής ικανότητας του μοντέλου και του οφέλους που μπορεί να προκύψει από την προσθήκη μνήμης.

Η πειραματική μεθοδολογία μπορεί επίσης να ενισχυθεί μέσω πολλαπλών ανεξάρτητων εκτελέσεων για κάθε διαμόρφωση. Παρότι στην παρούσα εργασία χρησιμοποιήθηκε μηδενική θερμοκρασία για τον περιορισμό της στοχαστικότητας, η επανάληψη των πειραμάτων θα επέτρεπε τον υπολογισμό διακύμανσης και διαστημάτων εμπιστοσύνης και, κατά συνέπεια, ισχυρότερη στατιστική αξιολόγηση των διαφορών μεταξύ των μηχανισμών μνήμης.

Σημαντική βελτίωση μπορεί να επιτευχθεί και στον τρόπο δημιουργίας των συνόλων μνήμης και αξιολόγησης. Εκτός από τον διαχωρισμό σε επίπεδο εγγραφών, μπορεί να πραγματοποιείται έλεγχος για λεκτικά ή σημασιολογικά παρόμοιες ερωτήσεις πριν από τον τελικό διαχωρισμό. Με τον τρόπο αυτό θα περιορίζεται η πιθανότητα μία ερώτηση του συνόλου αξιολόγησης να έχει σχεδόν ταυτόσημο παράδειγμα στο σύνολο που χρησιμοποιήθηκε για την αρχική τροφοδότηση της μνήμης και θα επιτυγχάνεται αυστηρότερη αξιολόγηση της ικανότητας γενίκευσης του μηχανισμού ανάκτησης.

Μία ιδιαίτερα σημαντική κατεύθυνση αφορά την επέκταση της μνήμης ώστε να υποστηρίζει όχι μόνο την τεχνική εγκυρότητα αλλά και τη σημασιολογική ορθότητα των παραγόμενων ερωτημάτων Cypher. Στην παρούσα υλοποίηση, η αποθηκευμένη εμπειρία και η διορθωτική ανατροφοδότηση σχετίζονται κυρίως με σφάλματα σύνταξης, σχήματος και ιδιοτήτων. Μελλοντικά, θα μπορούσαν να αποθηκεύονται και περιπτώσεις στις οποίες ένα τεχνικά έγκυρο ερώτημα δεν αποδίδει σωστά την πρόθεση της ερώτησης φυσικής γλώσσας ή επιστρέφει διαφορετικό αποτέλεσμα από το αναμενόμενο. Η ανάκτηση αντίστοιχων προηγούμενων περιπτώσεων, σε συνδυασμό με έναν μηχανισμό σημασιολογικής αξιολόγησης, θα μπορούσε να παρέχει στο σύστημα στοχευμένη καθοδήγηση και για τη διόρθωση σημασιολογικών αποκλίσεων. Με τον τρόπο αυτό, η μνήμη θα μπορούσε να αξιοποιείται όχι μόνο για την αποφυγή τεχνικών σφαλμάτων αλλά και για τη σταδιακή βελτίωση της αντιστοίχισης μεταξύ φυσικής γλώσσας και παραγόμενου ερωτήματος Cypher.

Μία ακόμη ενδιαφέρουσα κατεύθυνση αφορά τον συνδυασμό της επεισοδιακής μνήμης που εξετάστηκε στην παρούσα εργασία με μηχανισμούς μνήμης συνομιλίας. Η υφιστάμενη προσέγγιση διατηρεί εμπειρία από προηγούμενες εκτελέσεις της πρακτορικής ροής, χωρίς όμως να μοντελοποιεί τη συνέχεια μιας αλληλεπίδρασης πολλαπλών

γύρων με τον χρήστη. Αυτή η μορφή μνήμης είναι διακριτή από το βραχυπρόθεσμο History των αποτυχημένων προσπαθειών της ίδιας εκτέλεσης που χρησιμοποιήθηκε στην παρούσα εργασία.

Σε ένα συνομιλιακό περιβάλλον Text-to-Cypher, η διατήρηση του ιστορικού προηγούμενων ερωτήσεων και απαντήσεων της ίδιας συνομιλίας θα μπορούσε να υποστηρίξει ερωτήματα συνέχειας, αναφορές σε οντότητες που έχουν ήδη αναφερθεί και σταδιακή εξειδίκευση της πληροφοριακής ανάγκης του χρήστη. Ο συνδυασμός αυτού του συνομιλιακού πλαισίου με τη μακροπρόθεσμη επεισοδιακή εμπειρία του συστήματος θα επέτρεπε τη διερεύνηση μιας πληρέστερης αρχιτεκτονικής μνήμης για συνομιλιακή πρόσβαση σε γράφους γνώσης.

Μία επιπλέον κατεύθυνση αφορά την προσαρμοστική επιλογή της πληροφορίας μνήμης. Τα πειραματικά αποτελέσματα έδειξαν ότι η αποτελεσματικότητα των διαφορετικών μηχανισμών μεταβάλλεται ανάλογα με το γλωσσικό μοντέλο και τη δυσκολία του ερωτήματος. Μελλοντικά, το σύστημα θα μπορούσε επομένως να αποφασίζει δυναμικά αν είναι προτιμότερο να αξιοποιήσει προηγούμενη τροχιά, επιτυχημένο παράδειγμα, βραχυπρόθεσμο ιστορικό ή κάποιον συνδυασμό τους, αντί να εφαρμόζει μία προκαθορισμένη διαμόρφωση σε όλα τα ερωτήματα. Αντίστοιχα, τα κατώφλια και οι παράμετροι ανάκτησης θα μπορούσαν να προσαρμόζονται στη διαθέσιμη πληροφορία και στα χαρακτηριστικά της τρέχουσας περίπτωσης.

Η διαδικασία ανάκτησης μπορεί επίσης να επεκταθεί με πλουσιότερους μηχανισμούς κατάταξης. Η παρούσα προσέγγιση βασίζεται στη σημασιολογική ομοιότητα ερωτήσεων και σφαλμάτων και, στην περίπτωση των τροχιών, στη συνδυαστική βαθμολογία που περιγράφηκε στην Ενότητα 3.6. Μια μελλοντική υλοποίηση θα μπορούσε να συνδυάζει περισσότερα χαρακτηριστικά της τρέχουσας κατάστασης, όπως την ερώτηση φυσικής γλώσσας, το παραγόμενο ερώτημα, το είδος του σφάλματος και τη θέση της εκτέλεσης μέσα στην πρακτορική ροή. Η ταυτόχρονη αξιοποίηση αυτών των χαρακτηριστικών ενδέχεται να επιτρέψει ακριβέστερη επιλογή της εμπειρίας που είναι πραγματικά χρήσιμη για την τρέχουσα περίπτωση.

Επιπλέον, αξιόλογη θα ήταν η περαιτέρω διαχείριση του κύκλου ζωής της μακροπρόθεσμης μνήμης. Καθώς ο αριθμός των εκτελέσεων αυξάνεται, ο γράφος μνήμης μπορεί να συγκεντρώσει μεγάλο αριθμό παρόμοιων ή επαναλαμβανόμενων εμπειριών. Για τον λόγο αυτό θα μπορούσαν να εξεταστούν μηχανισμοί εντοπισμού και συγχώνευσης διπλότυπης πληροφορίας, περιοδικής συμπύκνωσης της μνήμης, διαγραφής παρωχημένων ή χαμηλής χρησιμότητας εγγραφών και ανάδειξης των σημαντικότερων εμπειριών. Η γραφική δομή της μνήμης που παρουσιάστηκε στην Ενότητα 3.4 επιτρέπει επίσης τη μελλοντική διερεύνηση μετρικών κεντρικότητας ή άλλων γραφοκεντρικών τεχνικών για την εκτίμηση της σημασίας μιας αποθηκευμένης εμπειρίας.

Ως προς την αποδοτικότητα του συστήματος, στην παρούσα εργασία καταγράφηκε και συγκρίθηκε ο συνολικός χρόνος εκτέλεσης των διαφορετικών πειραματικών διαμορ-

φώσεων και πραγματοποιήθηκε συμπληρωματική χρονομέτρηση επιμέρους σταδίων της ροής. Μελλοντικά, η ανάλυση αυτή θα μπορούσε να πραγματοποιηθεί συστηματικά για όλους τους συνδυασμούς μοντέλων και διαμορφώσεων, διαχωρίζοντας το κόστος των κλήσεων προς το γλωσσικό μοντέλο, των ελέγχων του CyVer και των διαφορετικών μηχανισμών ανάκτησης από τη μνήμη. Επιπρόσθετα, θα μπορούσαν να αναλυθούν η κατανάλωση prompt και completion tokens και το συνολικό κόστος των κλήσεων. Η μελέτη των μεγεθών αυτών καθώς αυξάνεται ο γράφος μνήμης θα επέτρεπε πληρέστερη αξιολόγηση της σχέσης μεταξύ του οφέλους που προσφέρει η μνήμη και του πρόσθετου υπολογιστικού κόστους.

Συμπληρωματικά προς την επέκταση της ίδιας της μνήμης με σημασιολογική εμπειρία, μπορούν να εξεταστούν και πλουσιότεροι τρόποι αξιολόγησης της σημασιολογικής ορθότητας των παραγόμενων ερωτημάτων. Η σύγκριση των αποτελεσμάτων εκτέλεσης αποτελεί ισχυρότερο κριτήριο από την απλή λεκτική ομοιότητα των δύο ερωτημάτων, ωστόσο εξακολουθούν να υπάρχουν περιπτώσεις στις οποίες διαφορετικές διατυπώσεις Cypher απαιτούν πιο σύνθετη αξιολόγηση. Μελλοντικά θα μπορούσαν να συνδυαστούν έλεγχοι ισοδυναμίας αποτελεσμάτων, περισσότερα σύνολα δεδομένων αναφοράς και, όπου είναι εφικτό, χειροκίνητη αξιολόγηση επιλεγμένων περιπτώσεων, ώστε να διαχωρίζονται με μεγαλύτερη ακρίβεια οι τεχνικά έγκυρες από τις πραγματικά ορθές λύσεις.

Συνολικά, οι παραπάνω επεκτάσεις μπορούν να οδηγήσουν από τον πειραματικό μηχανισμό που εξετάστηκε στην παρούσα εργασία σε ένα πιο προσαρμοστικό σύστημα μνήμης, το οποίο δεν αποθηκεύει απλώς προηγούμενη εμπειρία, αλλά αξιολογεί τη χρησιμότητά της, διαχειρίζεται δυναμικά τον κύκλο ζωής της και επιλέγει πότε και με ποιον τρόπο πρέπει να την επαναχρησιμοποιήσει. Η ενσωμάτωση σημασιολογικής και συνομιλιακής πληροφορίας θα μπορούσε επιπλέον να επεκτείνει τον ρόλο της μνήμης πέρα από την τεχνική διόρθωση μεμονωμένων ερωτημάτων, προς συστήματα που διατηρούν εμπειρία τόσο μεταξύ διαφορετικών εκτελέσεων όσο και κατά τη διάρκεια μιας συνεχούς αλληλεπίδρασης με τον χρήστη. Η κατεύθυνση αυτή μπορεί να συμβάλει στη δημιουργία πρακτορικών συστημάτων Text-to-Cypher που βελτιώνουν τη συμπεριφορά τους μέσα από προηγούμενες εκτελέσεις, διατηρώντας παράλληλα ελεγχόμενο και ερμηνεύσιμο τον μηχανισμό μνήμης.

Παράρτημα Α

Προτροπές των Πειραματικών και Τελικών Ροών

Στο παρόν παράρτημα παρατίθενται οι βασικές προτροπές που χρησιμοποιήθηκαν στην υλοποίηση της παρούσας εργασίας. Οι αρχικές προτροπές του Cypher Generator και του Cypher Corrector βασίζονται στην αρχιτεκτονική του CyVerACT [8] και επεκτάθηκαν στο πλαίσιο της παρούσας εργασίας με πρόσθετα τμήματα για την αξιοποίηση πληροφορίας μνήμης. Παράλληλα, παρατίθενται οι προτροπές που χρησιμοποιήθηκαν στην πειραματική ενσωμάτωση του Cognee, η οποία προηγήθηκε της τελικής προσαρμοσμένης υλοποίησης.

Για λόγους αναπαραγωγιμότητας, διατηρείται η αγγλική διατύπωση των προτροπών όπως χρησιμοποιήθηκε στην υλοποίηση. Οι τιμές που παράγονται δυναμικά κατά την εκτέλεση αποδίδονται με αγκύλες της μορφής `{variable}`. Στις βασικές προτροπές του Cypher Generator και του Cypher Corrector, τα τμήματα που προστέθηκαν στο πλαίσιο της παρούσας εργασίας επισημαίνονται με **έντονη γραφή**.

Α.1 Προτροπή του Cypher Generator

Η βασική δομή της προτροπής του Cypher Generator ακολουθεί την αντίστοιχη προτροπή του αρχικού CyVerACT [8]. Στην παρούσα εργασία επεκτάθηκε με το δυναμικό πεδίο `episodic_memory`, μέσω του οποίου μπορούν να εισαχθούν προηγούμενα επιτυχημένα παραδείγματα που ανακτώνται από τη μνήμη.

```
[SYSTEM]

Task: Generate Cypher statement to query a graph database.
Instructions: Use only the provided relationship types and properties in the schema.
Do not use any other relationship types or properties that are not provided in the
schema.
Do not include any explanations or apologies in your responses.
```

```
Do not respond to any questions that might ask anything else than for you to construct
a Cypher statement.
Do not include any text except the generated Cypher statement.



[HUMAN]

Generate a Cypher statement to query a graph database, as plain text without any code
block formatting or backticks.
Use only the provided relationship types and properties in the schema.

Schema: {schema}

Question: {question}

{episodic_memory}

Cypher statement:
```

A.2 Προτροπή του Cypher Corrector

Αντίστοιχα, η βασική προτροπή του Cypher Corrector βασίζεται στην αρχική υλοποίηση του CyVerACT [8], όπου η αρχική ερώτηση, το παραγόμενο ερώτημα Cypher και τα σφάλματα του CyVer χρησιμοποιούνται για την επαναληπτική διόρθωση της λύσης. Στην παρούσα εργασία η προτροπή επεκτάθηκε με τα δυναμικά πεδία `history`, `trajectory_context` και `rules_context`, ανάλογα με την εκάστοτε πειραματική διαμόρφωση.

```
[SYSTEM]

You are a Cypher expert reviewing a statement.
Given an input Cypher statement and the errors it has, correct the Cypher statement.
No pre-amble.
Do not wrap the response in any backticks or anything else. Respond with a Cypher
statement only!



[HUMAN]

Fix the Cypher statement based on the given errors.
The Cypher statement should be valid for the given knowledge graph schema.
Use only the provided labels and relationship types and properties in the schema.
Do not use any other node labels, relationship types or properties that are not
provided in the schema.

Schema:
```

```
{schema}

Note: Do not include any explanations or apologies in your responses.
Do not wrap the response in any backticks or anything else.
Respond with a Cypher statement only!

The question is:
{question}

The Cypher statement is:
{cypher}

The errors for the Cypher statement are:
{errors}

Previous Attempts and Their Outcomes:
{history}

{trajectory_context}

{rules_context}

Corrected Cypher statement:
```

A.3 Δυναμικά Συστατικά Μνήμης των Προτροπών

Τα πεδία μνήμης των παραπάνω προτροπών δεν αποτελούν σταθερό κείμενο, αλλά κατασκευάζονται δυναμικά κατά την εκτέλεση. Στις ακόλουθες ενότητες παρουσιάζεται η μορφή με την οποία κάθε κατηγορία πληροφορίας ενσωματώνεται στην αντίστοιχη προτροπή.

A.3.1 Ανακτημένο Επιτυχημένο Παράδειγμα

Όταν η σημασιολογική ομοιότητα της τρέχουσας ερώτησης με προηγούμενη επιτυχημένη ερώτηση υπερβαίνει το προκαθορισμένο κατώφλι, η ανακτημένη περίπτωση προστίθεται στην προτροπή του Cypher Generator ως δομικό παράδειγμα.

```
Successful historical query patterns are provided below.
Reuse their structural patterns and aggregation logic whenever relevant:
{retrieved_memory_record}
```

Το `retrieved_memory_record` περιλαμβάνει την προηγούμενη ερώτηση, το αντίστοιχο επιτυχημένο ερώτημα Cypher και τη βαθμολογία ομοιότητας της ανακτημένης περίπτωσης.

A.3.2 Συμπιεσμένο Βραχυπρόθεσμο Ιστορικό

Το ιστορικό των αποτυχημένων προσπαθειών ομαδοποιείται με βάση την ίδια συλλογή κανονικοποιημένων σφαλμάτων που παρήγαγε κάθε προσπάθεια. Η συμπιεσμένη αναπαράσταση που εισάγεται στον Cypher Corrector ακολουθεί την παρακάτω μορφή.

```
[MEMORY MANAGER] Compressed Error History:

The following attempts all produced the EXACT SAME sequence of errors:
  Attempt {attempt_num}: {cypher}
  ...

Errors generated by the above attempts:
  - {error}
  ...
----------------------------------------
```

A.3.3 Ανακτημένη Τροχιά Διόρθωσης

Για κάθε επιλεγμένη προηγούμενη τροχιά παρέχονται η προηγούμενη πρόθεση, το αποτυχημένο ερώτημα, το αντίστοιχο σφάλμα και το επιτυχημένο ερώτημα που προέκυψε μετά τη διόρθωση.

```
Review the following context and structural guidance to resolve the error:

--- Trajectory Strategy (Relevance: {combined_score}) ---
Past Intent: {past_question}
Past Failure: {failed_cypher}
Resulting Engine Error: {error_message}
Successful Structural Fix: {fixed_cypher}

CRITICAL INSTRUCTION: Analyze how the 'Successful Structural Fix' altered the syntax
of the 'Past Failure' to resolve the engine error. Apply this exact same structural
transformation to your current failing query.
```

A.3.4 Γενικοί Κανόνες Διόρθωσης (FixRules)

Εκτός από πληροφορία που προέρχεται από προηγούμενες εκτελέσεις, η υλοποίηση περιλαμβάνει ένα σύνολο προκαθορισμένων κανόνων διόρθωσης (`FixRules`). Κάθε κανόνας συνδέεται με μία κατηγορία σφάλματος (`ErrorType`) και ανακτάται όταν η αντίστοιχη κατηγορία εμφανιστεί κατά την αξιολόγηση ενός ερωτήματος από το CyVer.

Οι κατηγορίες σφαλμάτων ακολουθούν τους ελέγχους που χρησιμοποιούνται από το CyVerACT [8], ενώ οι αντίστοιχοι κανόνες διόρθωσης ορίστηκαν στο πλαίσιο της

παρούσας εργασίας ως πρόσθετη μορφή καθοδήγησης για τον Cypher Corrector. Οι κανόνες είναι ανεξάρτητοι από συγκεκριμένα προηγούμενα ερωτήματα και αποσκοπούν στην παροχή γενικεύσιμων οδηγιών ανά κατηγορία σφάλματος.

Όταν ανακτηθούν ένας ή περισσότεροι σχετικοί κανόνες, ενσωματώνονται στην προτροπή του Cypher Corrector με την ακόλουθη μορφή:

```
Generic Repair Principles:
- {rule_1}
- {rule_2}
...
```

Ο Πίνακας Α.1 παρουσιάζει το σύνολο των προκαθορισμένων κανόνων που χρησιμοποιήθηκαν στην υλοποίηση.

Πίνακας Α.1: Προκαθορισμένοι κανόνες διόρθωσης για τους τύπους σφαλμάτων του CyVer.

ErrorType	**FixRule**
ExecutionFailed	Generate only structurally sound queries that can be successfully parsed and executed by the database engine without failing at runtime.
SyntaxError	Strictly adhere to valid query syntax (e.g., standard Cypher). Do not hallucinate unsupported clauses, keywords, or invalid operators.
ParameterNotProvided	If utilizing parameterized variables in the query (e.g., $param), verify that you are not referencing parameters that have not been explicitly defined or passed to the environment.
UnsatisfiableRelationship TypeExpression	Construct logical relationship patterns. Avoid creating impossible, contradictory, or mutually exclusive relationship type constraints within a single match clause.
ProcedureNotFound	Only invoke built-in database functions or verified, standard plugin procedures. Do not guess or invent procedure names.
ArithmeticError	Implement mathematical safety checks within the query; ensure that dynamic or static divisors cannot evaluate to zero to prevent division-by-zero runtime errors.
ConflictingVariables Error	Maintain strict variable scope and typing. Never assign conflicting node labels or relationship types to the same variable name within the same query.
UnknownLabelWarning	Only utilize node labels that are explicitly confirmed to exist within the provided knowledge graph schema.
UnknownRelationship TypeWarning	Ensure all relationship types exactly match the defined relationships in the database schema. Do not guess relationship names.

ErrorType	FixRule
UnknownPathWarning	Validate that the structural combination of nodes and relationships forming a path actually exists in the schema's topological layout before querying it.
WrongDirectionPath Warning	Pay strict attention to relationship directionality (e.g., (a)-[:REL]->(b) vs. (a)<-[:REL]-(b)). Ensure the query aligns perfectly with the directed nature of the graph.
VariableLengthPath Warning	When querying paths of unknown depth, use the unbounded wildcard operator (*) safely, rather than assuming specific fixed bounds (like *X..Y), unless exact depth bounds are strictly verified by the schema.
WrongDirectionPath& VariableLengthPath Warning	Double-check both the directionality arrows and the variable-length traversal syntax simultaneously to ensure the multi-hop path accurately reflects how the relationships traverse the graph.
UnknownPropertyKey Warning	Only query or filter by property keys that are explicitly defined in the schema dictionary.
InvalidPropertyAccessBy LabelWarning	Cross-reference property keys with their correct node labels or relationship types. Never attempt to access or return a property from an entity that does not inherently support it.

A.4 Πειραματικές Προτροπές της Ενσωμάτωσης Cognee

Κατά την πειραματική διερεύνηση του Cognee χρησιμοποιήθηκαν ξεχωριστές προτροπές για την κατασκευή του γράφου μνήμης, την αποθήκευση μιας αποτυχημένης προσπάθειας και την ανάκτηση σχετικών κανόνων διόρθωσης. Οι προτροπές αυτές αφορούν την πειραματική διαμόρφωση που προηγήθηκε της τελικής προσαρμοσμένης λύσης και δεν αποτελούν μέρος του τελικού μηχανισμού μνήμης.

A.4.1 Προτροπή Κατασκευής του Γράφου Μνήμης

Η ακόλουθη προσαρμοσμένη προτροπή χρησιμοποιήθηκε στη διαδικασία `cognify()` για την εξαγωγή των οντοτήτων και σχέσεων του πειραματικού γράφου.

```
You MUST extract ALL of the following entities:

1. CypherErrorType:
- name

2. CypherError:
```

```
- name
- description
- is_type → CypherErrorType

3. FixRule:
- description
- fixes → CypherError

4. QueryExample:
- user_query
- bad_cypher
- error_message
- related_error → CypherError

Rules:
- ALWAYS create a FixRule
- ALWAYS create a QueryExample
- FixRule must be general and reusable
- QueryExample must contain exact values

Relationships:
- FixRule fixes CypherError
- QueryExample relates to CypherError

Output MUST include all entity types.
```

A.4.2 Προτροπή Αποθήκευσης Αποτυχημένης Προσπάθειας

Για την εισαγωγή μιας αποτυχημένης προσπάθειας στη μνήμη κατασκευαζόταν δυναμικά το ακόλουθο κείμενο.

```
Cypher Error Type: {error_type}

Cypher Error:
- Name: {error_message}
- Description: The agent attempted to execute invalid Cypher against the Neo4j
database.

Fix Rule:
- Description: Analyze the following error and create a single, highly specific, and
reusable Cypher rule to prevent it in the future.
Error Context to analyze: '{error_message}'.
Do NOT just copy the error. Write a definitive rule (e.g., 'Always use WITH before
COUNT', 'Reverse the relationship arrow for ACTED_IN').

Query Example:
```

```
- User Query: {user_query}
- Bad Cypher: {bad_cypher}
- Error Message: {error_message}
```

A.4.3 Προτροπή Ανάκτησης Κανόνων

Η αναζήτηση στο γράφο του Cognee πραγματοποιούνταν μέσω της λειτουργίας GRAPH_COMPLETION, με την ακόλουθη προτροπή.

```
Cypher query debugging rules.

User request:
{query}

STRICT INSTRUCTIONS:
- Return ONLY the most relevant rules
- If Cognee memory is empty, return ONLY the phrase 'No results'
- Maximum length: 200 words
- Prefer concise bullet points
- DO NOT repeat similar rules
- DO NOT include long explanations
- DO NOT include full examples unless critical

Focus on:
- aggregation mistakes (WITH, COUNT)
- relationship direction
- schema issues (labels, properties)
- duplicate results (DISTINCT)
```

A.4.4 Ενσωμάτωση του Ανακτημένου Πλαισίου στη Ροή

Στην πειραματική διαμόρφωση που παρουσιάζεται στο Σχήμα 3.2, η ανακτημένη πληροφορία προστίθεται ως επιπλέον πλαίσιο στα γλωσσικά στάδια της ροής. Ειδικότερα, η πειραματική προτροπή του Cypher Generator περιλάμβανε τις ακόλουθες πρόσθετες οδηγίες και μεταβλητές:

```
Instructions: If memory_context contains previous mistakes for this question, avoid
making the same mistake again. Do not repeat previously known invalid Cypher
statements. Respond with the corrected or new Cypher statement only.

{procedural_context}
{episodic_context}
```

Αντίστοιχα, στο στάδιο διόρθωσης η ανακτημένη πληροφορία εισαγόταν ως επιπλέ-

ον σύνολο προηγούμενων σφαλμάτων και καθοδήγησης:

```
Previous Similar Mistakes & Lessons:
{memory_context}
```

Οι παραπάνω προσθήκες αφορούν αποκλειστικά την πειραματική ενσωμάτωση του Cognee. Στην τελική αρχιτεκτονική αντικαταστάθηκαν από τους ρητά ελεγχόμενους μηχανισμούς μνήμης που παρουσιάστηκαν στις προηγούμενες ενότητες του παρόντος παραρτήματος.

Βιβλιογραφία

[1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," *arXiv preprint arXiv:2310.08560*, 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[2] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan, "Rethinking Memory in LLM based Agents: Representations, Operations, and Emerging Topics," *arXiv preprint arXiv:2505.00675*, 2025. [Online]. Available: https://arxiv.org/abs/2505.00675

[3] M. Pink, Q. Wu, V. A. Vo, J. Turek, J. Mu, A. Huth, and M. Toneva, "Position: Episodic Memory is the Missing Piece for Long-Term LLM Agents," *arXiv preprint arXiv:2502.06975*, 2025. [Online]. Available: https://arxiv.org/abs/2502.06975

[4] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic Memory for LLM Agents," *arXiv preprint arXiv:2502.12110*, 2025. [Online]. Available: https://arxiv.org/abs/2502.12110

[5] V. Markovic, L. Obradovic, L. Hajdu, and J. Pavlovic, "Optimizing the Interface Between Knowledge Graphs and LLMs for Complex Reasoning," *arXiv preprint arXiv:2505.24478*, 2025. [Online]. Available: https://arxiv.org/abs/2505.24478

[6] P. Anokhin, N. Semenov, A. Sorokin, D. Evseev, A. Kravchenko, M. Burtsev, and E. Burnaev, "AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents," *arXiv preprint arXiv:2407.04363*, 2024. [Online]. Available: https://arxiv.org/abs/2407.04363

[7] I. Mandilara, C.-M. Androna, E. Fotopoulou, A. Zafeiropoulos, and S. Papavassiliou, "Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?" *IEEE Access*, vol. 13, pp. 80 981–81 001, 2025.

[8] C.-M. Androna, I. Mandilara, E. Arkadopoulou, E. Fotopoulou, A. Zafeiropoulos, and S. Papavassiliou, "CyVerACT: An Agentic Cypher Translation Workflow over Knowledge Graphs," *Information Processing & Management*, vol. 63, p. 104836, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S030645732600227X

[9] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan, "Memory in the Age of AI Agents," *arXiv preprint arXiv:2512.13564*, 2025, version 2, revised January 2026. [Online]. Available: https://arxiv.org/abs/2512.13564v2

[10] H. Yu, T. Chen, J. Feng, J. Chen, W. Dai, Q. Yu, Y.-Q. Zhang, W.-Y. Ma, J. Liu, M. Wang, and H. Zhou, "MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent," *arXiv preprint arXiv:2507.02259*, 2025, version 2, revised July 2026. [Online]. Available: https://arxiv.org/abs/2507.02259v2

[11] LangChain, "Memory Overview," LangChain Documentation, 2026, accessed: 2026-08-07. [Online]. Available: https://docs.langchain.com/oss/python/concepts/memory

[12] A. Gilmore, "Modeling Agent Memory," Neo4j Developer Blog, March 2025, accessed: 2026-08-07. [Online]. Available: https://neo4j.com/blog/developer/modeling-agent-memory/

[13] J. Kang, M. Ji, Z. Zhao, and T. Bai, "Memory OS of AI Agent," *arXiv preprint arXiv:2506.06326*, 2025. [Online]. Available: https://arxiv.org/abs/2506.06326

[14] R. Fang, Y. Liang, X. Wang, J. Wu, S. Qiao, P. Xie, F. Huang, H. Chen, and N. Zhang, "Memp: Exploring Agent Procedural Memory," *arXiv preprint arXiv:2508.06433*, 2025. [Online]. Available: https://arxiv.org/abs/2508.06433

[15] Y. Wang and X. Chen, "MIRIX: Multi-Agent Memory System for LLM-Based Agents," *arXiv preprint arXiv:2507.07957*, 2025. [Online]. Available: https://arxiv.org/abs/2507.07957

[16] J. Luo, Y. Tian, C. Cao, Z. Luo, H. Lin, K. Li, C. Kong, R. Yang, and J. Ma, "From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms," *arXiv preprint arXiv:2605.06716*, 2026. [Online]. Available: https://arxiv.org/abs/2605.06716

[17] M. Suzgun, M. Yuksekgonul, F. Bianchi, D. Jurafsky, and J. Zou, "Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory," *arXiv preprint arXiv:2504.07952*, 2025. [Online]. Available: https://arxiv.org/abs/2504.07952

[18] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://arxiv.org/abs/2410.10813

[19] D. Wu, Z. Ji, A. Kawatkar, B. Kwan, J.-C. Gu, N. Peng, and K.-W. Chang, "LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues," *arXiv preprint arXiv:2605.12493*, 2026. [Online]. Available: https://arxiv.org/abs/2605.12493

[20] W. Ma, J. Nan, W. Wu, and Y. Chen, "What Deserves Memory: Adaptive Memory Distillation for LLM Agents," *arXiv preprint arXiv:2508.03341*, 2026, version 4, April 2026; first submitted August 2025. [Online]. Available: https://arxiv.org/abs/2508.03341v4

[21] J. Fang, X. Deng, H. Xu, Z. Jiang, Y. Tang, Z. Xu, S. Deng, Y. Yao, M. Wang, S. Qiao, H. Chen, and N. Zhang, "LightMem: Lightweight and Efficient Memory-Augmented Generation," *arXiv preprint arXiv:2510.18866*, 2025, version 4, revised February 2026. [Online]. Available: https://arxiv.org/abs/2510.18866v4

[22] J. Liu, Y. Su, P. Xia, S. Han, Z. Zheng, C. Xie, M. Ding, and H. Yao, "SimpleMem: Efficient Lifelong Memory for LLM Agents," *arXiv preprint arXiv:2601.02553*, 2026. [Online]. Available: https://arxiv.org/abs/2601.02553v3

[23] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," *arXiv preprint arXiv:2504.19413*, 2025. [Online]. Available: https://arxiv.org/abs/2504.19413

[24] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, J. E. Labra Gayo, S. Kirrane, S. Neumaier, A. Polleres, R. Navigli, A.-C. Ngonga Ngomo, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, 2021. [Online]. Available: https://arxiv.org/abs/2003.02320

[25] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, "Cypher: An Evolving Query Language for Property Graphs," in *Proceedings of the 2018 International Conference on Management of Data*. ACM, 2018, pp. 1433–1445.

[26] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," *arXiv preprint arXiv:2501.13956*, 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[27] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410/

[28] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models," *arXiv preprint arXiv:2506.05176*, 2025. [Online]. Available: https://arxiv.org/abs/2506.05176v3

[29] B. Jiménez Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, "From RAG to Memory: Non-Parametric Continual Learning for Large Language Models," *arXiv preprint arXiv:2502.14802*, 2025. [Online]. Available: https://arxiv.org/abs/2502.14802

[30] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, 2024. [Online]. Available: https://arxiv.org/abs/2306.08302

[31] S. Piao, J. Lee, and S. Park, "LitE-SQL: A Lightweight and Efficient Text-to-SQL Framework with Vector-based Schema Linking and Execution-Guided Self-Correction," *arXiv preprint arXiv:2510.09014*, 2025. [Online]. Available: https://arxiv.org/abs/2510.09014

[32] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, "Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects," *arXiv preprint arXiv:2512.12818*, 2025. [Online]. Available: https://arxiv.org/abs/2512.12818

[33] W. Lyon, "Meet Lenny's Memory: Building Context Graphs for AI Agents," Neo4j Developer Blog, February 2026, accessed: 2026-08-07. [Online]. Available: https://neo4j.com/blog/developer/meet-lennys-memory-building-context-graphs-for-ai-agents/

[34] D. Chalef, "Graphiti: Knowledge Graph Memory for an Agentic World," Neo4j Developer Blog, March 2025, accessed: 2026-08-07. [Online]. Available: https://neo4j.com/blog/developer/graphiti-knowledge-graph-memory/

[35] Cognee, "Cognify: Transforming Ingested Data into a Knowledge Graph with Embeddings, Chunks, and Summaries," Cognee Documentation, 2026, accessed: 2026-08-25. [Online]. Available: https://docs.cognee.ai/core-concepts/main-operations/legacy-operations/cognify

[36] ——, "Search: Querying AI Memory with Vectors, Graphs, and LLMs," Cognee Documentation, 2026, accessed: 2026-08-25. [Online]. Available: https://docs.cognee.ai/core-concepts/main-operations/legacy-operations/search

[37] H. Kafkas, "LangGraph x cognee: Enhancing Agents with Persistent, Queryable Memory," Cognee Blog, October 2025, accessed: 2026-08-07. [Online]. Avail-

able: https://www.cognee.ai/blog/integrations/langgraph-cognee-integration-build-langgraph-agents-with-persistent-cognee-memory

[38] LangChain, “LangChain Overview,” LangChain Documentation, 2026, accessed: 2026-08-07. [Online]. Available: https://docs.langchain.com/oss/python/langchain/overview

[39] Sentence Transformers, “all-MiniLM-L6-v2,” Hugging Face Model Card, accessed: 2026-09-16. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[40] Neo4j, “Neo4j Text2Cypher (2024) Dataset,” Hugging Face Datasets, 2024, accessed: August 2026. [Online]. Available: https://huggingface.co/datasets/neo4j/text2cypher-2024v1